Wollo University

Kombolcha Institute of Technology (KIoT)

Computer Programming Module (ECEg 1052)

Prepared By:

1. Abel Teshome (MSc., Computer Science Department)
2. Chimdessa Assaba (MSc., Computer Science Department)
3. Hassen Ibrahim (MSc., Information Systems Department)
4. Tadele Asitatikie (MSc., Information Technology Department)
5. Tewodros Asefa (MSc., Electrical Engineering Department)

Editor:

1. Wondatir Teka (MSc., Electrical Engineering Department)

Kombolcha, Ethiopia

January 2021

TABLE OF CONTENTS

LIST OF TABLES VIII

LIST OF FIGURES IX

CHAPTER ONE: INTRODUCTION TO COMPUTER 1

1.1. THE NATURE OF INFORMATION 1
1.2. DEFINITION OF COMPUTER 3
1.3. CHARACTERISTICS OF COMPUTER 4
1.4. ADVANTAGES AND LIMITATIONS OF USING COMPUTERS 5
1.5. APPLICATIONS OF COMPUTERS 6
1.6. GENERATION OF COMPUTER 7
1.6.1. Parts of a Computer System 9
1.6.2. Hardware Components 10
1.6.3. Software Components 21
1.7. CLASSIFICATION OF COMPUTERS 27
1.7.1. Classification by Size 27
1.7.2. Classification by Type 28
1.7.3. Classification by Purpose 29
REVIEW QUESTIONS 29
REFERENCES 30

CHAPTER TWO: NUMBERING SYSTEM AND DATA REPRESENTATION 31

2.1. BINARY SYSTEMS 31
2.2. NUMBER BASE CONVERSIONS 34
2.3. DATA REPRESENTATION 39
EXERCISE 40
REFERENCES 41

CHAPTER THREE: INTRODUCTION TO PROGRAMMING 42

3.1. WHAT IS COMPUTER PROGRAMMING? 42
3.2. WHAT SKILLS DO WE NEED TO BE A PROGRAMMER? 43

3.3. REASONS TO STUDY PROGRAMMING 43
3.4. WHAT IS A PROGRAMMING LANGUAGE? 44
3.5. GENERATION OF PROGRAMMING LANGUAGE 45
3.5.1. First Generation (Machine languages, 1940's) 46
3.5.2. Second Generation (Assembly languages, early 1950's) 46
3.5.3. Third Generation (High-level languages, 1950's to 1970s) 46
3.5.4. Fourth Generation (since the late 1970s) 47
3.5.5. Fifth Generation (1990's) 47
3.6. MAJOR PROGRAMMING PARADIGM 47
3.6.1. Procedural Programming Paradigms 48
3.6.2. Structured Programming Paradigms 48
3.6.3. Object-Oriented Programming Paradigms 49
3.7. PROBLEM SOLVING PROCESS AND SOFTWARE ENGINEERING 49
3.7.1. Software Engineering 49
3.7.2. Software Development Life Cycle (SDLC) 49
3.7.3. Software Development Life Cycle Models 54
3.7.4. Problem Solving Process 63
EXERCISES 66
REFERENCES 67

CHAPTER FOUR: ALGORITHM DEVELOPMENT AND PROBLEM-SOLVING TECHNIQUES 68

4.1. COMPUTATIONAL PROBLEMS 68
4.2. INTRODUCTION ABOUT ALGORITHM 69
4.3. PSEUDO-CODE 72
4.4. FLOW CHART 75
4.4.1. General Rules for Flowcharting 77
4.4.2. Flowcharting Tips 78
4.4.3. Different Basic Flowcharts 78
4.5. QUALITIES/FEATURES/PROPERTIES OF A GOOD ALGORITHM 83
LAB 87

REVIEW QUESTIONS 87
EXERCISE 88
REFERENCES 89

CHAPTER FIVE: FUNDAMENTALS OF C++ PROGRAMMING LANGUAGE 90

5.1. MECHANICS OF CREATING A PROGRAM (C++) 90
5.2. COMPILING PROGRAMS 92
5.3. IDE TOOLS 93
5.3.1. Setting Up the "Hello World!" Project 95
5.3.2. To add a new source file to the project 98
5.3.3. Running the Code 100
5.4. THE STRUCTURE OF C++ PROGRAMS 100
5.5. COMMENTS 103
5.6. C++ BASIC INPUT/OUTPUT 105
5.6.1. I/O Library Header Files 105
5.6.2. The Standard Output Stream (cout) 106
5.6.3. The Standard Input Stream (cin) 106
5.7. CONSTANTS, KEYWORDS, VARIABLES, DATA TYPES AND OPERATORS 107
5.7.1. Keywords (reserved words) 107
5.7.2. Variable 108
5.7.3. Identifiers 109
5.7.4. Data Type 110
5.7.5. typedef Declarations 111
5.7.6. Variable Declaration 112
5.7.7. C++ CONSTANTS/LITERALS 114
5.7.8. C++ Operator 118
5.8. OPERATORS PRECEDENCE IN C++ 128
5.9. SIMPLE TYPE CONVERSION 129
5.10. STATEMENTS 130
5.10.1. Null Statement 130
5.10.2. The Block Statements 130

5.10.3. The Assignment Statements. *131*
LAB 131
EXERCISE 134
REFERENCES 140
CHAPTER SIX: CONTROL FLOW STATEMENTS 141
6.1. INTRODUCTION 141
6.2. CONDITIONAL STATEMENTS 142
6.2.1. The if Statement *142*
6.2.2. The if…. else Statement *145*
6.2.3. The if...else if...else Statement: *147*
6.2.4. Nested if statement *150*
6.2.5. C++ switch Statement *153*
6.2.6. C++ Nested switch Statements *160*
6.3. LOOPING STATEMENTS 161
6.3.1. The 'for' Statement *161*
6.3.2. The 'while' Statement *165*
6.3.3. The 'do…while' Statement *168*
6.4. PITFALLS IN WRITING LOOPING STATEMENTS 169
6.5. OTHER STATEMENTS 171
6.5.1. The 'continue' Statement *171*
6.5.2. The 'break' Statement *172*
6.5.3. The 'goto' Statement *173*
6.5.4. The 'return' Statement *174*
LAB 175
EXERCISE 177
REFERENCES 178
CHAPTER SEVEN: FUNCTION 179
7.1. INTRODUCTION TO FUNCTION 179
7.1.1. Defining a Function *181*

7.1.2. Function Declarations *183*
7.1.3. Calling a Function *185*
7.2. FUNCTION ARGUMENTS 186
7.2.1. Call by value *186*
7.2.2. Call by reference *188*
7.3. SCOPE OF VARIABLES 189
7.3.1. Local Variables *189*
7.3.2. Global Variables *191*
7.4. RETURN VALUES 193
7.5. DEFAULT VALUES FOR PARAMETERS: 194
7.6. FUNCTION OVERLOADING 195
7.7. RECURSION 198
7.8. RECURSION VERSUS ITERATION 200
LAB 202
EXERCISE 203
REFERENCES 205

CHAPTER EIGHT: ARRAY AND STRING 206

8.1. INTRODUCTION TO ARRAY 206
8.1.1. Declaring arrays *207*
8.1.2. Initializing arrays *208*
8.2. ACCESSING AND PROCESSING ARRAY ELEMENTS 210
8.3. TWO-DIMENSIONAL ARRAYS 213
8.3.1. Accessing two-dimensional array elements *214*
8.4. ARRAYS AS PARAMETERS 234
8.5. INTRODUCTION TO STRINGS 236
8.5.1. What are the Strings? *236*
8.5.2. Initialization of Strings *237*
8.5.3. Reading a String from the Keyboard *238*
8.5.4. Functions to manipulate strings *242*
8.5.5. Converting strings to other types *247*

LAB 250
REVIEW QUESTION 251
EXERCISE 253
REFERENCES 255

List of Tables

Table 1:Numbers with Different Bases..... 37

Table 2: Common symbols in flowchart design 76

Table 3: Test for the example one algorithm. 87

Table 4: I/O Library Header Files..... 105

Table 5: Reserved words in C++ 107

Table 6: Data Types, Sizes, and Ranges 110

Table 7: Escape sequence codes for C++ 116

Table 8: The common arithmetic operators supported by C++ language..... 119

Table 9: The common relational operators supported by C++ language..... 120

Table 10: The common logical operators supported by C++ language. 120

Table 11: The common bitwise operators supported by C++ language 121

Table 12: The common assignment operators supported by C++ language..... 122

Table 13: The common assignment operators supported by C++ language..... 126

Table 14: The miscellaneous operators supported by C++ language 127

Table 15: Precedence of operators in C++..... 128

Table 16: Loop condition example 165

Table 17: Types of function calls..... 186

Table 18: Two-dimensional array..... 213

List of Figures

Figure 1:A typical wired keyboard from DELL 11

Figure 2: A typical wired mouse from DELL..... 11

Figure 3: A wireless trackball 12

Figure 4: A laptop touchpad 12

Figure 5: A light pen for a tablet..... 12

Figure 6: A bar code reader 13

Figure 7: A character recognizing device 13

Figure 8: MIDI board connected to a laptop..... 13

Figure 9: Different types of sensors..... 14

Figure 10: Epson Perfection V850 Pro Scanner 14

Figure 11: Sony DSC-W830 Digital Camera (Silver) 14

Figure 12: Shure SM58-LC Dynamic Microphone 15

Figure 13:Different RAMs..... 18

Figure 14: A hard disk drive 20

Figure 15: Operating System and other software and hardware layers 23

Figure 16: Flow of activities in waterfall model software development life cycle 55

Figure 17. The flow of activities in the V-shaped model software development life cycle 56

Figure 18. Flow of activities in Prototype model of software development Life cycle.... 58

Figure 19. Flow of activities in Spiral model software development life cycle 59

Figure 20. Flow of activities in Iterative Incremental model software development life cycle 60

Figure 21. The flow of activities in an Agile model software development life cycle..... 62

Figure 22: Algorithm development steps..... 71

Figure 23: Phase of C++ program..... 91

Figure 24: Code Blocks interface 94

Figure 25: Creating a New Project..... 95

Figure 26: Console application interface (welcome screen)..... 96

Figure 27: Console application interface (selected language) 96

Figure 28: Console application interface project title selection..... 97

Figure 29: Starting the HelloWorld Application 97

Figure 30: C++ new source file creation 98

Figure 31: Adding a New Source File to a project 99

Figure 32: The source file before typing code 99

Figure 33: The source file after typing code 100

Figure 34: C++ program outputting “Hello World!” to the screen..... 100

Figure 35. Syntax of an identifier 109

Figure 36. A general from of a typical decision-making structure 142

Figure 37. Control flow of a simple if statement block in C++..... 143

Figure 38. Control flow of an if else block in C++..... 145

Figure 39. Control flow of a switch statement in C++ 155

Chapter One

1. Introduction to Computer

Objectives: Software programs (applications) are developed by computers and are run on computers. Thus, students are supposed to have a basic understanding of computers before diving into the world of computer programming.

At the end of this chapter, students are expected to:

- *be able to describe the word computer differently*
- *list the advantages of using computers over the non-computer world*
- *identify the limitations of computers that users need to take note of*
- *explain the five computer generations that human-being has passed through*
- *identify the basic hardware and software components of a computer*
- *differentiate computers by size, type, purpose*

1.1. The Nature of Information

Information is the core of computers and communications technology. Let us have a brief look at different terminologies related to information.

A. Data: Data is specific numerical or symbolic representations of facts about the world. Data is the element to be input, stored, and manipulated by the computer. A data that adequately describes the reality it was meant to be referred to as valid. One can think of data as a "raw material" that needs to be processed before being turned into something useful. Data processing is, therefore, the manipulation and transformation of data into a more meaningful form, information.

B. Information: Information is a useful organization and selection of facts, not the number of facts available. Information is organized data. It involves relationships among the represented facts. Computers process information when they store, retrieve, or rearrange relationships among data. For example, a telephone book contains data representing the names, addresses, and telephone numbers of people in a region. The

information in the phone book is the relationship between name and phone number and the alphabetical order of the names. If you know someone's name, you can use that information to find his or her phone number. There is less information to help you find the name of someone whose phone number you have; you might have to look at each item of numerical data before you could use the information relating name to number. A phone book for the whole world would contain more data than a city directory, but it would not necessarily have more information. Indeed, it might contain too much data for you to find the information you want. If you are trying to reach only Mr. Alebachew Kebede's family in your town, it would not be helpful to see how many Alebachew people with that name have telephones.

C. **Knowledge**: Knowledge involves the evaluation and understanding of information. It refers to the meaning of information concerning human interests and purposes. You could have a great deal of information available to you, for example in an encyclopedia, without understanding what it meant or how to apply it to your situation. In the telephone book example, your knowledge includes an understanding that the names refer to people and the number is a code enabling you to operate a machine to speak with them. Your understanding of how to use a telephone is a form of knowledge. So is the scientific knowledge of how telephone circuits operate and the cultural knowledge that you should call your mother on New Year's Day. The expression garbage in; garbage out applies to computerized information in several ways. If the data are not valid, no amount of careful organization can make them represent reality. If the data are valid, but our arrangement is at fault, then we will not have accurate information. Finally, if we have valid data organized to provide excellent information about things of no interest or value to us, we have not contributed positively to our knowledge.

D. **Wisdom**: Wisdom is the ability to know when and how to apply knowledge. It is the ability to imagine the consequences of our actions for ourselves and others. It is a capacity for good judgment that we acquire through experience. Just as we are only now beginning to understand the effects that our industrial society has had on the world's environment, we sometimes acquire wisdom through a long period of painful mistakes. The wise use of computers involves a careful study of their context and

consequences, not only their inner workings. Yet, because our present knowledge about the effects of computers is incomplete, we may find in the future that some of our choices about computerization were unwise. Acquiring wisdom doesn't mean that we stop making errors, only that we never stop learning from them.

1.2. Definition of Computer

A computer is a device that can compute. However, a computer can do more than a computing task and it can be defined in different ways. Here are provided three definitions of a computer.

Definition 1: A computer is an electronic machine, operating under the control of instruction stored in its memory, that can accept data (input) manipulate the data according to specified rules (Process), produce results (output), and store the result for future use.

Definition 2: A computer can be defined as an electronic device that accepts data as an input, process data as information, and gives the information as the output.

Definition 3: A computer can be defined as a machine which works under the control of a well-structured set of instruction which accepts raw fact (input), process the raw facts (process) and produce meaningful information (output).

From these definitions, one can identify that a computer accepts input, processes data, stores data, and produces output.

A. ***A computer accepts input:*** Computer input is whatever data that is fed into a computer. Furthermore, the input can also refer to the process of feeding data into a computer. Examples for input to a computer include a character from the keyboard, audio from the microphone, hardcopy documents from the scanner, and others. An input device gathers and translates the input into a form that the computer can process.
B. ***A computer processes data:*** A computer manipulates input data in some way depending on the required aspect. Such manipulation of the input is called data

processing. Data processing examples are the mathematical computation of input numbers, alphabetical sorting of names.

C. ***A computer stores data:*** A computer stores data either permanently or temporarily in its storage devices. Storing could be necessary to store processed data, to store system files permanently, or to store the temporary demanded file for short time. Storage types are discussed in the computer components section below.

D. ***A computer produces the output:*** Computer output is whatever data is generated by a computer system. Furthermore, the output can also refer to the process of showing the processed data out of a computer system. Examples of outputs could be a character inserted from a keyboard is shown in the computer screen and a document is printed to a hardcopy document

1.3. Characteristics of Computer

The characteristics of a computer show the capability and the potential of the computer for processing data. This saves time, space, money, labor, etc. These characteristics answer the question of why computers are used. The following are the main characteristics of a computer.

- **Automatic**: Once a task is started on a computer, the computer carries on the task until it is finished, normally without any human intervention.

- **Speed**: The ability of computers to carry out their instructions in a very short period is one of the main reasons for their popularity. Computers can perform within a matter of seconds or minutes tasks that would be impossible for a person to complete by hand in a lifetime.

- **Accuracy**: Nowadays, computers are being used in life-and-death situations (For example, jet pilots rely on computer computations for guidance, Hospitals rely on patient-monitoring systems in critical–care units) which needs almost a hundred percent accuracy. From this, we can understand that the computer is accurate and

consistent. Unless there is an error in the input data or unreliable program the computer processes very accurately.

- **Reliability**: Computers can carry out the same type of work repeatedly without throwing up errors due to tiredness or boredom, which are very common among humans.

- **Versatility**: Computers can carry out a wide range of work from data entry and ticket booking to complex mathematical calculations and continuous astronomical observations. If you can input the necessary data with correct instructions, the computer will do the processing.

- **Storage Capacity**: Computers can store a very large amount of data at a fraction of the cost of traditional storage of files. Also, data is safe from normal wear and tear associated with paper.

1.4. Advantages and Limitations of Using Computers

Having the above-mentioned characteristics, a computer is found very advantages due to the following capabilities:

- Computers can do the same task repetitively with great ease, speed, and reliability
- Computers do not get tired or bored
- Computers can take up routine tasks while releasing human resource for more intelligent functions
- Store and process large amount of information with high speed and accuracy
- Transmit information across continents via communication channels
- Simulate events
- Perform complex mathematical computations and make comparisons
- Monitor ongoing industrial operations

However, computers have also limitations like:

- Computers can perform tasks that are only programmed to do
- Computers operate on data that are provided by users
- Computers cannot give any conclusion without going through immediate instructions
- Computers have limited use in qualitative conditions
- Computers require regular checking

1.5. Applications of Computers

Computers have become important tools in our day-to-day activities. Some of the area's computers are used are briefly discussed below.

A. ***Education and Research:*** Computer is used in educational institutions as a teaching aid, online teaching. It helps in providing a lot of facilities in the education system. Learning could be controlled, delivered, and evaluated using computer-based education. The Internet contains a lot of information for researchers, scholars, and teachers.

B. ***Business:*** Computers are needed to perform business operations that require the handling of large amounts of data. Some of the applications in this area are accounting and financial management, inventory control, database management, and statistical analysis

C. ***Engineering:*** Computers are used for design work and analysis & control of physical systems. Examples are space technology, meteorological observatory systems, astronomical investigations, design of machines, and control of the manufacturing process.

D. ***Electronic banking and service:*** Computers are found very important in getting banking service easily. Examples are online banking (a bank customer can use his/her computer to check account balances, transfer money or pay for bills), shopping from home (an individual may shop by the computer in the comfort of his/her home), and Automatic Teller Machines (customers are issued cards that permit them to use other banks' teller machines)

E. ***Communication:*** The integration of computers and communication technologies is what is being referred to as ICT. This merger has resulted in more efficient communication using computers and other handheld devices such as mobile phones used to facilitate the sending and receiving of messages over the internet. Computers also control many telecommunications equipment.

F. ***Manufacturing:*** Computers have made industrial processes more efficient through optimization scheduling, process control, and using robots to perform automated manufacturing process control which is otherwise dangerous to human beings.

G. ***Transportation:*** Computers have affected almost every kind of transportation. Many aircraft can fly under the control of the computer; in this situation, the captain simply serves as a manager by telling the computer what to do. In Cars, computers have provided functional controls such as spark and fuel control.

H. ***Medical and Health Care:*** Computers have long been used by hospitals for routine record keeping. Today, however, many people owe their lives to the computer. Computers are used in hospitals as sensors (devices that detect changes in blood pressure, heart rate, temperature), testing (scan the body and provide 3-D figures), patient treatment.

I. ***Consultant (Expert system):*** An Expert system is a computer program, which can solve problems from a specific knowledge base. These systems don't replace expert humans because the knowledge base of an expert system is given by the skilled specialist. Example: MYCIN (a medical diagnostic program by using a sophisticated decision-making process).

1.6. Generation of Computer

The computer technology we use today is not a result of one-night innovation, instead, it has evolved through different stages. The major characteristics that distinguish the various generations are the following:

- The dominant type of electronic circuit elements used
- Major secondary storage media
- Computer language used

- Type or characteristics of the operating system used
- Memory access time

As we advance from the first generation to the fifth generation, computers get smaller in size, more powerful, and less expensive. In this section, five different generations of computers are presented starting from the earliest one.

a. First Generation Computer (1946 -1959)

The first-generation computers used **vacuum tubes** for circuitry and magnetic drums for memory, and were enormous, taking up entire rooms. They were very expensive to operate and used high electricity, generated a lot of heat, which cause malfunctions.

First-generation computers relied on machine language which is the lowest-level programming language understood by computers, to perform operations, and they could only solve one problem at a time.

b. Second Generation Computer (1959-1965)

In the second generation of computers, transistors replaced vacuum tubes; the transistor was invented in 1947 but did not see widespread use in computers until the late 1950s. The transistor was greater than the vacuum tube, allowing computers to become smaller, faster, cheaper, more energy-efficient, and more reliable than their first-generation predecessors. Though the transistor still generated a great deal of heat that subjected the computer to damage, it was a great improvement over the vacuum tube. High-level programming languages were also being developed at this time, such as early versions of COBOL and FORTRAN. These were also the first computers that stored their instructions in their memory, which moved from a magnetic drum to magnetic core technology.

c. Third Generation Computer (1965-1971)

In the third generation, the development of the **integrated circuit** was the hallmark of the third generation of computers. Transistors were miniaturized and placed on silicon chips, called semiconductors, which drastically increased the speed and efficiency of computers.

Instead of punched cards and printouts, users interacted with third generation computers through keyboards and monitors and interfaced with an operating system, which allowed the device to run many different applications at one time with a central program that monitored the memory. Computers for the first time became accessible to a mass audience because they were smaller and cheaper than their predecessors.

d. Fourth Generation Computer (1971 -1980)

The microprocessor brought the fourth generation of computers, as thousands of integrated circuits (Very Large-Scale Integrated microprocessor) were built onto a single silicon chip. What is the first generation filled an entire room could now fit in the palm? The Intel 4004 chip, developed in 1971, located all the components of the computer from the central processing unit and memory to input/output controls on a single chip.

e. Fifth Generation Computer (1980-onwards)

Fifth-generation computing devices, based on artificial intelligence, are still in development, though there are some applications, such as voice recognition, that are being used today. The use of parallel processing and superconductors is helping to make artificial intelligence a reality. Quantum computation and molecular and nanotechnology will completely change the face of computers in years to come. The goal of fifth-generation computing is to develop devices that respond to natural language input and are capable of learning and self-organization

1.6.1. Parts of a Computer System

Every single full-functional computer contains various components. A computer generally consists of two major components called hardware components and software components. In this section, these components are briefly discussed considering the Desktop personal computer.

1.6.2. Hardware Components

Hardware components of a computer refer to those physical components of a computer that could be touched and be felt. These components could be used to input data to the computer, to process the input data for any sort of data manipulation, to see an output of the computer, or to store the output data for further usage. Generally, hardware components of a computer system could be categorized as an input unit, processing unit, output unit, and storage unit.

1.6.2.1. Input Unit

Data and instructions must enter the computer system before any computation can be performed on the supplied data. This task is performed by the input unit that links the external environment with the computer system. Data and instructions enter input units in forms that depend upon the particular device used. For example, data are entered from a keyboard like typing and this differs from how data are entered through a scanner, which is another type of input device. However, regardless of the form in which they receive their inputs, all input devices must provide a computer with data that are transformed into the binary codes that the primary memory of a computer is designed to accept. This transformation is accomplished by units called input interfaces. Input interfaces are designed to match the unique physical or electrical characteristics of input devices to the requirements of the computer system. The common input devices are briefly described below.

A. **Keyboard**: The keyboard is a typewriter-style device that can use the alphanumeric keys or buttons for the input given by the user to the computer system. There are various types of keyboards available nowadays, which are designed with a focus on the specific features according to the user's need. The keyboard includes different types of sections such as control keys, function keys, LED indicators, wrist pad, arrow keys, and keypad. Mostly the keyboards are similar to each other, but maybe one or more sections are missing. The Smartphone and tablet keyboard do not come with the physical existence of the keyboard.

Figure 1:A typical wired keyboard from DELL

B. **Mouse**: A mouse is a pointing device that fits comfortably under the palm of your hand. It is the most widely used pointing device on Desktop computers. The mouse is used to move the cursor on your computer screen, to give instructions to your computer, and to run programs and applications. It can be used to select menu commands, move icons, size windows, start programs, close windows, and others.

Figure 2: A typical wired mouse from DELL

The most commonly used mouse actions are click, right-click, double-click, and drag & drop.

- Click: Click means press and release the left mouse button once. It is used to select an item.
- Right Click: Right-click means to press and release the right mouse button once. It is used to display a set of alternative commands from which one can choose a command.
- Double Click: Double click means quickly press the left mouse button twice in a fast sequence. It is used to open a document or a program.
- Drag and Drop: place the cursor over an item on the screen and then press and hold down the Left Mouse Button. Holding down the button, move the cursor to where you want to place the item, and then release the button.

C. **Trackball**: This is a ball that is like an inverted mouse, which is rotated with the fingers or palm.

Figure 3: A wireless trackball

D. **Touchpad**: The touchpad is a small, flat surface. With this device, the fingers can be used to move the mouse pointer to various locations

Figure 4: A laptop touchpad

E. **Light Pen**: The light pen is a light-sensitive stylus, or pen-like device, connected by a wire to the computer terminal. A user brings the pen to the desired point on the display screen and presses the pen button, which identifies the screen location of the computer.

Figure 5: A light pen for a tablet

F. **Bar-code Readers**: Bar codes are the vertical zebra-striped marks you see on most manufactured retail products. Bar-code readers are photoelectric scanners that translate the bar-code symbols into digital code.

Figure 6: A bar code reader

G. **Character Recognizing Devices**: These devices are used in the bank to read to the numbers at the bottom of the check, to read to the number plates of the cars, and generally printed characters when scanning.

Figure 7: A character recognizing device

H. **MIDI Board:** MIDI stands for Musical Instrument Digital Interface. It provides a standard for the interchange of musical information between musical instruments, synthesizers, and computers. MIDI keyboards can be plugged into the computers so that the music can be input into the computers and processed.

Figure 8: MIDI board connected to a laptop

I. **Sensor**: A sensor is an input device that collects specific kinds of data directly from the environment and transmits it to a computer. Sensors are found very useful in earthquake detection, tracking the depth of the sea, and lots of other areas.

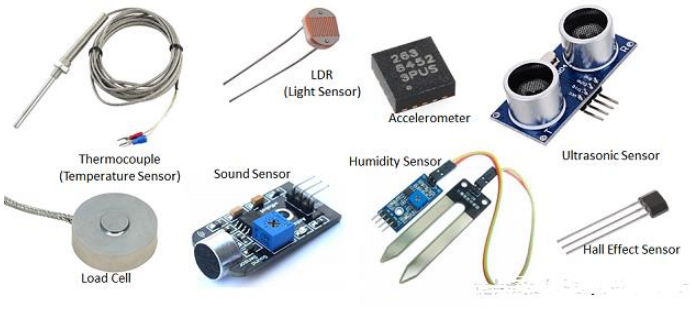


Figure 9: Different types of sensors

J. **Scanner**: Scanner is an input device that can read text or illustrations printed on paper & translate the information into a form that a computer can use.

Figure 10: Epson Perfection V850 Pro Scanner

K. **Digital Camera**: Digital camera is an electronic device that is used to capture and store photographs electronically in a digital format that can be read by a computer. The image can be transferred directly from the camera to a computer

Figure 11: Sony DSC-W830 Digital Camera (Silver)

L. **Microphone**: A microphone is a device that can be used to input sound information to a computer system from the environment. It can be used with a voice recognition system to record sound.

Figure 12: Shure SM58-LC Dynamic Microphone

1.6.2.2. Processing Unit (Central Processing Unit-CPU)

Once data or instruction is fed to the computer system, it is the processing unit that is responsible for any sort of data manipulation or execution of instruction so that appropriate information can be displayed and/or required data can be stored to the desired destination. Generally, the CPU is known also as the brain of a computer as it controls all processing inside a computer. CPU has the following fundamental features:

- It is part of computer hardware that executes program instructions.
- The CPU is like the human brain and it oversees and controls all of the activities of the computer.
- The CPU manages and coordinates the entire computer including the input and output units.
- The CPU is the collection of electronic circuits made up of millions of transistors placed into integrated circuits.
- Integrated circuits are also called chips or microchips
- Each transistor is an electrical switch that can be in one of the two states: “on” or “off”.
- Small transistors allow more transistors to be packed in one single chip, thus eliminating the need for separate chips

CPU has two major components: The Arithmetic Logic Unit and the Control Unit.

A. **The Arithmetic Logic Unit (ALU):** The arithmetic logic unit (ALU) of a computer system is the place where the actual execution of the instructions takes place during the processing operation. Calculations are performed and all comparisons are made in the ALU. The data and instructions stored in the primary storage before processing, are transferred as and when needed to the ALU where processing takes place. No

processing is done in the primary storage unit. Intermediate results generated in the ALU are temporarily transferred back to the primary storage until needed at a later time. Data may thus move from primary storage to ALU and back again to storage many times before the processing is over. After the completion of processing the final results, which are stored in the storage unit are released to an output device.

The type and number of arithmetic and logic operations that a computer can perform are determined by the engineering design of the ALU. However, almost all ALUs are designed to perform the four basic arithmetic operations (add, subtract, multiply, and divide) and logic operations or comparisons such as less than, equal to, and greater than, and others.

B. The Control Unit: The control unit maintains the order of tasks and directs the operation of the entire system by selecting, interpreting, and seeing the execution of program instructions. In other words, it acts as a central nervous system for the other components of the computer system as it manages and coordinates the entire computer system. It obtains instructions from the program stored in the main memory, interprets the instructions, and issues signals that cause other units of the system to execute them.

1.6.2.3. Output Unit

The job of an output unit is just the reverse of that of an input unit. It supplies the information obtained as a result of data processing to the outside world. Thus, it links the computer with the external environment. As computers work with binary code, the results produced are also in the binary form. Hence, before supplying the results to the outside world, it must be converted to a human-readable form. This task is accomplished by units called output interfaces. Output interfaces are designed to match the unique physical or electrical characteristics of output devices to the requirements of the external environment. The output could be in softcopy or hard copy format. Softcopy format is electronic whereas hardcopy format is paper-based.

The common output hardware components of a computer system are briefly discussed below.

A. **Monitor**: It displays the output of a computer. There are different types of monitors depending on different criteria. Based on the color types monitors could support, a monitor could be monochrome, grayscale, or color monitor. Monochrome monitors show one-color text/image on a single background color. The black and white monitor is one example of such type of monitors. The grayscale monitor shows shades of gray on a white background. Color monitors, on the contrary, can display different colors.

   Depending on the technology monitors are made of, monitors could be cathode ray tubes (CRT), liquid crystal display (LCD), electroluminescent, or XGA.

B. **Speech Synthesizer**: This device converts digital data into speech-like sounds.

C. **Video Conferencing system:** Videoconferencing is a method whereby people in different geographical locations can have a meeting and see and hear one another – using computers and communications.

D. **Printers:** Printers are used to produce "hard copy" of documents, programs, and data. Some printers use serial communication (RS232 standard) while others use parallel communication to transfer information from the PC to the printer. In serial communication, data is transferred one bit at a time while in the case of parallel communication; one byte (8 bits) is transferred at one time.

1.6.2.4. Storage Unit

The data and instruction that are entered into the computer system through input units have to be stored inside that computer before the actual processing starts. Similarly, the results produced by the computer after processing must also be kept somewhere inside the computer system before being passed on to the output units. Moreover, the intermediate results produced by the computer must also be preserved for ongoing processing. The storage unit of a computer system is designed to cater to all these needs. It provides space for storing data and instructions, space for intermediate results, and also for the final results.

The storage unit of all computers is comprised of the following two types of storage:

1. **Primary Storage**: The primary storage, also known as main memory, is used to hold pieces of program instructions and data, intermediate results of processing, and recently

produced results of processing of the jobs that the computer system is currently working on. These pieces of information are represented electronically in the main memory chip's circuitry, and while it remains in the main memory, the central processing unit can access it directly at a very fast speed. However, the primary storage can hold information only while the computer system is on. As soon as the computer system is switched off or reset, the information held in the primary storage disappears. Additionally, the primary storage normally has limited storage capacity because it is very expensive. The primary storage of modern computer systems is made up of semiconductor devices. Primary storage could be read-only memory or random-access memory.

- **Read-Only Memory (ROM):** Nonvolatile chips always hold the same data; the data in them cannot be changed except through a special process that overwrites the data. It is usually done at the factory. During normal use, the data in these chips are only read and used -not changed- so the memory is called read-only memory (ROM). One important reason a computer needs ROM is that it must know what to do when the power is first turned on. Among other things, ROM contains a set of start-up instructions, which ensures that the rest of the memory is functioning properly, checks for hardware devices, and checks for an operating system on the computer's disk drives.
- **Random Access Memory (RAM):** A Memory that can be instantly changed is called read-write memory or random-access memory (RAM). When people talk about computer memory in connection with microcomputers, they usually mean the volatile RAM. It is best simply to remember that the data in ROM does not change while the data in RAM changes constantly. RAM can hold information only while the computer system is on. As soon as the computer system is switched off or reset, the information held in the RAM disappears. This nature of RAM is known as volatile.

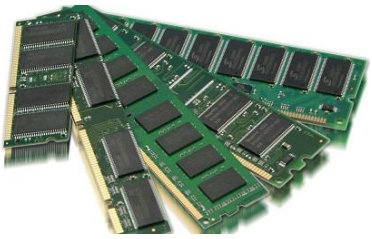


Figure 13:Different RAMs

2. **Secondary Storage**: The secondary storage, also known as auxiliary storage, is used to take care of the limitations of the primary storage. That is, it is used to supplement the limited storage capacity and the volatile characteristic of primary storage. This is because secondary storage is much cheaper than primary storage and it can retain information even when the computer system is switched off or reset. The secondary storage is normally used to hold the program instructions, data, and information of those jobs on which the computer system is not working currently but needs to hold them for processing at a later time. There exist two technologies for secondary storage: magnetic storage devices and optical storage devices.
    - **Magnetic Storage Devices**: the medium contains iron particles, which can be polarized, i.e., given a magnetic charge in one of two directions. Each particle's direction represents a 1 or a 0, representing each bit of data that the CPU can recognize. A disk drive uses read/write heads containing electromagnets to create magnetic charges on the medium.

      Formatting maps the disk's surface and determines how data will be stored. During formatting, the drive creates circular tracks around the disk's surface and then divides each into sections called sectors. The OS organizes sectors into groups, called clusters, and then tracks each file's location according to the cluster it occupies. Formatting also deletes any information that exists on the disk. Floppy disk, hard disk, magnetic tapes are examples of magnetic storage.
        - Hard disk drive:
            - Inside the case of the hard disk drive are a set of rigid rotating disks, called platters, stacked on a spindle. As the platters rotate at high speed, Read/Write heads move back and forth recording or reading magnetic information on both sides of the platters. The speed with which the heads can find data is called "average access time".
            - Hard disk drives are normally installed inside the computer case although external hard disk drives, which connect to the computer by a cable, are also available.
            - The hard disk drive is selected based on its storage capacity (measured in "Gigabytes" (GB) or greater) and its access time

(measured in milliseconds (ms)). The larger the capacity and the smaller the access time, the better. Operating system and application software normally reside in hard disk drives. Documents created using application software may also be placed on hard disk drives.

Figure 14: A hard disk drive

- **Optical Storage Devices:** offer high-capacity storage medium. Optical drives use reflected light to read data. The disk's metal surface is covered with dents (pits) and flat spots (lands), which cause light to be reflected differently. Light cannot be reflected back from pits (data = 0) and lands reflect light back to its source (data = 1). CD-ROM, DVD-ROM, CD-R, CD-RW are examples of optical storage.
    - **CD–R** is a standard and technology allowing you to write to and read from a Compact Disc, but not erase or change what you record. This technology is compatible with existing CDs, i.e., you can read these discs in existing CD–players. Today, we have speeds as much as 52X. A single CD–R can hold up to 650MB of information**.**

    - **CD–RW** is a rewritable CD–ROM technology. CD–R/W drives can also be used to write CD–R discs, and they can read CD–ROMs. But, CD–R/W disks have lower reflectivity than CD–ROMs, and CD–ROM drives must be of the newer multi-read variety to read them. CD–R/W was initially known as CD–E (or CD–Erasable).

    - **Digital Video Disc (DVD),** also called Digital Versatile Disc, is similar to CD–ROMs but able to store far more music, video, or data. DVDs are of the same physical size as today's CDs but can store as much as 17GB, compared to the 650MB on today's CD–ROMs.

- **DVD:** The DVD represents a new generation of high-density CD-ROM disks, which are read by a laser and which have both write-once and re-writable capabilities. It stands for Digital Video Disk.

- **Flash Memory Cards:** Flash-memory cards consist of circuitry on PC cards that can be inserted into slots connecting to motherboards.

1.6.3. Software Components

Software is the set of instructions that can be stored and run by hardware. In other words, the software is a collection of data or computer instructions that tell the computer how to work. Hardware is typically directed by the software to execute any command or instruction. You cannot see or touch the software, although you can see the packages it comes in. The software can be divided into three kinds depending upon the nature of work this software does: Application software, System software, and Language software.

A. Application Software: It is a program or group of programs designed for end-users. Most of the applications we use in our daily life such as photo editing tools, games, web browsers, file viewers, music players are typical examples of application software. Application software is also known as either application or app. There are five categories of application software.

- **Entertainment Software**: Atomic Bomber man, Challenger, or any other video game is enjoyed by many youngsters. Even though it seems not important, in reality, these games software gain lot of profit in the market than some of the software. For example, it generated 3.5 billion dollars in sales in the US in 1996. It has become so popular that there are colleges that offer a four-year degree program in video game and computer animation programming. This software involves a lot of animation programming required which means the software developer should know how to make an object move and fly and run to make the software.

- **Home/Personal Software**: Application software that is needed for personal needs like travel planner, greetings designing, recipes guide, etc. is called Home or

Personal Software. This software mainly focuses on Home and Personal needs. Some of them are home repair, home decoration, floor planning, etc.

- **Educational/Reference Software**: Computer software has become so effective in such a way that it can teach various subjects. Some educational software can be used by a person who doesn't even know more about computers. These kinds of software come in CDs. The learner just needs to insert the CD and the lesson begins. This educational software uses multimedia technology a lot so that the user (learner) can feel easy to learn. From Children's learning software to encyclopedia software there is a lot of software available. There is software that will enable distant learning very easy. One such software is WebCT developed by British Columbia University is widely used nowadays.
- **Productivity Software:** Productivity software consists of programs found in most offices, homes, almost all computers. The purpose of this software is simply to make users more productive at performing particular tasks. This software is used by most people on a day-to-day basis. Some of them are word processing software, spreadsheet software, database server, financial software, and web-based software
- **Specialty Software**: For every profession, there can be related software found. Whether it is medical, mechanical engineering, farming, architecture, space research, or games there is software developed for these professions so that with the help of the software the efficiency, quality and fastness can be achieved. So, software developed to achieve a particular task especially is called specialty software.

B. System Software: System software is next level closer to the hardware of the computer. It helps to make the instructions given by the application software understandable to the computer. It is a software that enables the application software to run on the computer. It controls the allocation and usage of hardware resources and enables the applications software to run. System software can further be classified as the operating system, programming language, and utilities.

Operating System: The operating system acts as the master control program that runs the computer. It handles such activities as running and storing programs and storing and processing data. The purpose of the operating system is to allow applications to operate by

standardizing access to shared resources such as disks and memory. Examples of operating systems are MS-DOS, Windows-2000, XP, Windows 7, Windows 8.1, Windows 10, Unix, and Macintosh operating system. The operating system (OS) consists of the master system of programs that manage the basic operations of the computer. These programs provide resource management services of many kinds, handling such matters as the control and use of hardware resources, including disk space, memory, CPU, time allocation, and peripheral devices.

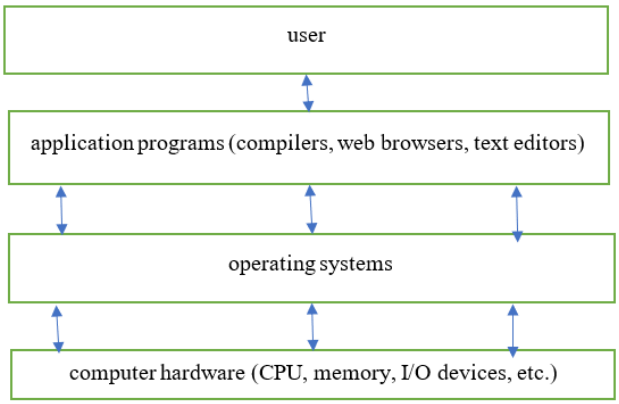


Figure 15: Operating System and other software and hardware layers

The operating system is responsible for booting, managing storage media, user interface, managing computer resources, managing files, and managing tasks.

Booting: When the computer is switched on, the startup process is carried out which is called booting. This includes checking the system, devices, etc., and enabling them so that the computer can be ready to work with.

Managing storage media: OS manages storage devices so that they can be efficiently utilized. Recognizing the storage hardware, enabling them so that they can be used include this process.

User Interface: Once the system is started the user should be able to interact with the computer. OS provides a user-friendly capability so that the user can interact with the computer. This is called the User Interface. With this, the user can set up software, printer, scanner, or do any OS-related work. This is also called Graphical User Interface (GUI)

Managing computer resources: Providing resources can be different from managing them. Resources such as Random-Access Memory should be managed properly to run the computer efficiently. There can be more than one process or one work requiring the same resource. For example, MS Word and Calculator running on the computer at the same time. The user may require the Calculator to do some calculations so that he can enter those numbers in the Word file. So, two applications should share the same RAM. These kinds of things are managed by the OS.

Managing Files: Files are stored in the hard disks or floppy disks and other secondary storage devices. The operating system helps to find them and manage them. With the help of this, we can manage the files. We can rename a file, delete, copy, move, etc.

Managing Tasks: A computer is required to perform various tasks at the same time. For example, the computer can print a document, scan a picture, and search for a file. An operating system manages these tasks so that the computer can do all these tasks efficiently.

Types of Operating Systems

Operating systems can be classified depending on the number of programs they can handle and the number of users that can be working on the system at a time: -

1. **Single-user operating system**: A single-user operating system normally only allows one user to access the system at a time. Operating systems of this type is used by many microprocessor systems, Example of a single user operating system include MS_DOS Microsoft Disk Operating System.
2. **Bach-mode Operating systems**: Batch processing refers in general to a computer system in which jobs (programs +data) are entered and then run one or more a time under the control of the operation system. In this system, the operating system schedules the submitted jobs according to the required resources and priority of the jobs and produces the result at some time later. It is the oldest operating system.
3. **Time-sharing Operating system**: A time-sharing system is an interactive system in which several users at terminals all have access to the computer at the same time. An operating system for such a system is called timesharing. UNIX is the most popular time-sharing operating system.

4. **Real-time operating system:** A real-time system is a system that is capable of processing data so quickly that the results are available to influence the activity currently taking place. Examples include airplane seat reservations and a computer-controlled plant.
5. **Multitasking**: Executing more than one program concurrently. (one user) How does a computer handle more than one program? The answer is that the operating system directs the processor (CPU) to spend a predetermined amount of time executing the instruction for each program, one at a time. A small amount of each program is processed and then the processor moves to the remaining programs, one at a time, processing small parts of each. This cycle is repeated until processing is complete. The processor speed is usually so fast that it may seem as if all the programs are being executed at the same time. However, the processor is still executing only one instruction at a time, no matter how it may appear to the user.
6. **Multiprogramming**: Executing different users' programs concurrently. (Multiple users) As with multitasking, the CPU spends a certain amount of time executing each user's program, but it works so quickly that it seems as though all the programs are being run at the same time.
7. **Multiprocessing**: Simultaneous processing of two or more programs by multiple computers. (Single or multiple users) With multiprocessing, two or more computers or processors linked together to perform work simultaneously, meaning at precisely the same time. This can entail processing instructions from different programs or different instructions from the same program. One type of multiprocessing is parallel processing, whereby several CPUs work together on the same tasks, sharing memory. This is used in large computer systems designed to keep running if one of the CPUs fails. This kind of system is at work in important places like space stations, online reservation systems, etc.

Language Software: Software is written in a programming language. A program is a set of instructions that tells the computer what to do and how to do certain tasks. To write these instructions, there are a set of rules and conditions. The instructions that are written are abided by these rules and conditions. Otherwise, when these instructions are executed, the computer will give an error. This is called a programming language. For example, the

English language has a set of rules and conditions that we call Grammar. In the same way, the language that is used to write programs is called *programming language.*

When these instructions/programs are written in a programming language, it has to be translated to a known instruction to the Operating System. Again, the software is needed to do this job. This software is called Language software. So, a software that helps to translate the written programs or instructions into an operating system level instruction is called a language software. For example, C is a programming language and 'C compiler' is the language software.

Sometimes to write the instructions or programs in these languages is tedious. So, there is another set of software that makes the writing of instructions/programs easy. For example, to make a box or a window in C++, you have to write instructions. Sometimes it may be tedious. The same can be done by drawing the same box or window with the software. This software is called IDE. Integrated Development Environment.

Programming languages can generally be classified as low-level languages and high-level languages. Low-level language may be machine languages and assembly languages.

- **Machine language**: Machine language is the natural language of the computer. A program written in machine language is ready for immediate execution. Machine language is very difficult to understand for humans. A program written in machine language takes the form of strings of zeros and ones. Even the simplest program is complex and lengthy. Moreover, each computer type has its machine language and thus a program written in machine language for one computer may not run in another computer, i.e., machine language programs are not portable.
- **Assembly language**: Assembly language consists of English-like abbreviations. It is easier to understand than machine language. Programs written in assembly language need to be translated into machine language before they can be executed. The program that translates assembly language programs to machine language is called an assembler. High-level languages are English-like instructions. They are easier to learn than low-level languages. Programs written in a high-level language are called source codes and they must be translated into machine language before they can run on a computer. After the translation, the program in machine language is called object code.

There exist two types of translators: interpreters and compilers. Interpreters translate each line of the program for immediate execution while compilers translate the entire source code into object code and then save the object code.

Utility Software: Utility software is programs that are intended to perform tasks related to the maintaining of your computer's health, either hardware or data. Some utility software is included with the operating system. Examples of utility programs include file defragmentation, data configuration, backup, antivirus, and screen saver programs.

1.7. Classification of Computers

1.7.1. Classification by Size

Based on size and performance, computers can be classified into four types:

1. ***Micro Computer:*** A microcomputer is a computer whose CPU is a microprocessor. A microprocessor is a processor all of whose components are on a single integrated circuit chip. Personal computers are kind of microcomputer. Personal computers are called so because they are designed for the personal use of the individual or individual small business units' office automation unit or professionals. Pc can be used for a variety of applications like computer literacy, fun, and games, business applications, programming, etc.
2. ***Mini Computer:*** They are the smaller version of the mainframes. Generally, they offer the same computing power as their bigger counterparts. The most important advantage of a mini-computer over the mainframe is that it is cheaper smaller in size and reliable. It does not require air conditioning and can be operated at room temperature. The main use of these systems is in education in local government word processing etc. in business they are being used for involving stock payroll etc. it is generally used as a server system on networks with personal computers as nodes.
3. ***Mainframe Computer:*** They are very big and offer maximum computing power. A large number of peripherals can be attached to them. They are generally used in large networks of computers with the mainframe being the model point of the network. They

used satellites for networking. A typical application is the airline system. It has a mainframe computer at their head office where information of all the fights is stored. Small computers are installed at the booking offices are attached to the central data bank, so that up-to-date information about all flights is always available.

4. ***Super Computer:*** They are the most expensive of all the computers. These computers are big general-purpose computers capable of executing more than 10,000 million instructions per second and have storage capacities of millions of bits per chip. These computers are used to solve multi-variate mathematical problems such as atomic nuclear and plasma physics seismology, aerodynamics, etc. The supercomputer is typically capable of handling hundreds of millions of floating points. Operations per second (MFLOPS). The speed of supercomputers is generally measured in "FLOPS" (Floating Point Operations Per Second). Supercomputers are used for highly calculation-intensive tasks such as weather forecasting, climate research, molecular modeling, physical simulation, and cryptanalysis, and military and Scientific agencies are heavy users.

1.7.2. Classification by Type

Based on the working principle, the computer can be classified into three types:

- ***Analog Computer:*** process data that vary continuously with time, such as variations in temperature, speed, the chemical composition of petroleum products, or current flowing through a conductor. Analog computers operate by measuring. They deal with continuous variables. They do not compete directly with numbers, rather, they operate by measuring physical magnitude such as pressure-temperature, voltage, current, etc. Generally, they are computers designed for special purposes. E.g., Thermometer, voltmeter, speedometer
- ***Digital Computer:*** process digital data. All the PCs currently in wide use are digital computers. Digital computers deal with discrete variables. They operate by counting rather than measuring. They operate directly upon numbers (or digits) that represent numbers, letters, or other special symbols. A digital computer is a computing device in which data is represented by discrete numerical quantities. It is represented by discrete

voltage states (1s and 0s). They have a very high accuracy and speed than analog ones. E.g., Personal Computers

- ***Hybrid Computer:*** Hybrid computer utilizes the best qualities of both analog and digital computers. They are suited for situations where digital processing of data collected in analog form is desirable. For example – in a hospital, intensive care unit analog device may measure a patient's heart function, temperature, etc. these measurements may then be converted into numbers and supplied to digital devices. Other areas are guided missile system new aircraft design etc.

1.7.3. Classification by Purpose

Purpose wise computer can be classified into two types: general-purpose computer and a specific purpose computer.

- ***General-purpose computer:*** This computer can store different programs and can thus be used in countless applications. It can perform any kind of job with equal efficiency simply by changing the application program stored in the main memory.
- ***Special-purpose computer:*** It is the one that is designed to perform only one special task. The program or instruction set is permanently stored in such a machine. It cannot be used for any other purpose. These computers are often used to perform specific functions such as controlling a manufacturing process or directing communications.

Review Questions

1. Define what does computer means.
2. List the characteristics of the computer.
3. Differentiate the advantages and limitations of computers.
4. Explain the five generations of computers.
5. List and explain the major hardware components of the computer.
6. List and explain the major software components of the computer.
7. What are the different types of computer-based computer size?
8. What are the different types of computer-based on computer type?

9. What are the different types of computer-based on computer purpose?

References

[1] R. Khanna, Basics of Computer Science, New Age International (P) Ltd, 2008.

[2] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Laurie Rosatone , 2012.

[3] P. Nortons, Introduction to Computers, McGRAW-HILL, 2002.

Chapter Two

2. Numbering System and Data Representation

Objectives: Inside the computer, every information is represented in binary data. A character can be represented in different numbering formats. Therefore, computer programmers need to have basic knowledge of numbering systems and data representation.

At the end of this chapter, students should be able to:

- *identify the various numbering systems*
- *convert a number from one numbering system to another numbering system*
- *understand how data is represented inside a computer*

2.1. Binary Systems

Digital systems have such a prominent role in everyday life that we refer to the present technological period as the digital age. Digital systems are used in communication, business transactions, traffic control, space guidance, medical treatment, weather monitoring, the Internet, and many other commercials, industrial, and scientific enterprises. We have digital telephones, digital television, digital versatile discs, digital cameras, and of course, digital computers. The most striking property of the digital computer is its generality. It can follow a sequence of instructions, called a program that operates on given data. The user can specify and change the program or the data according to the specific need. Because of this flexibility, general-purpose digital computers can perform a variety of information processing tasks that range over a wide spectrum of applications.

Digital systems manipulate discrete quantities of information that are represented in binary form. Operands used for calculations may be expressed in the binary number system. Other discrete elements, including the decimal digits, are represented in binary codes. Data processing is carried out through binary logic elements using binary signals. Quantities are stored in binary storage elements.

A decimal number such as 7,392 represents a quantity equal to 7 thousand plus 3 hundred, plus 9 tens, plus 2 units. The thousands, hundreds, etc. are powers of 10 implied by the position of the coefficients. To be more exact, 7,392 should be written as:

$$7x10^3 + 3x10^2 + 9x10^1 + 2x10^0$$

However, the convention is to write only the coefficients and from their position deduce the necessary powers of 10. In general, a number with a decimal point is represented by a series of coefficients as follows:

$$d^5\ d^4\ d^3\ d^2\ d^1\ d^0.d^{-1}\ d^{-2}\ d^{-3}$$

The d_j coefficients are any of the 10 digits (0, 1, 2... 9), and the subscript value; gives the place value and, hence, the power of 10 by which the coefficient must be multiplied. This can be expressed as

$$10^5d^5 + 10^4d^4 + 10^3d^3 + 10^2d^2 + 10^1d^1+10^0d^0 +10^{-1}d^{-1} + 10^{-2}d^{-2} + 10^{-3}d^{-3}$$

The decimal number system is said to be off base or radix, 10 because it uses 10 digits and the coefficients are multiplied by powers of 10. The binary system is a different number system. The coefficients of the binary numbers system have only two possible values: 0 or 1. Each coefficient d is multiplied by 2^n. For example, the decimal equivalent of the binary number 11010.11 is 26.75, as shown from the multiplication of the coefficients by powers of 2:

$$1x2^4 + 1x2^3 + 0x2^2 + 1x2^1 + 0x2^0 + 1x2^{-1} + 1x2^{-2} = 26.75$$

In general, a number expressed in a base `r` system has coefficients multiplied by powers of r:

$$d^nxr^n+d^{n-1}xr^{n-1}+ \ldots +d^2xr^2 + d^1xr^1 + d^0xr^0+d^{-1}xr^{-1} + d^{-2}xr^{-2} + \ldots + d^{-m}xr^{-m}$$

Where d^0 to d^n digits are before the radix point and d^{-1} to d_{-m} digits are after the radix point. To distinguish between numbers of different bases, we enclose the coefficients in parentheses and write a subscript equal to the base used (except sometimes for decimal

numbers, where the content makes it obvious that it is decimal). An example of a base-5 number is

$$(4021.2)_5 = 4x5^3 + 0x5^2 + 2x5^1 + 1x5^0 + 2x5^{-1} = (511.4)_{10}$$

The coefficient values for base 5 can be only 0, 1, 2, 3, and 4. The octal number system is a base- 8 system that has eight digits: 0, 1, 2, 3, 4, 5, 6, 7. An example of an octal number is 127.4. To determine its equivalent decimal value, we expand the number in a power series with a base of 8:

$$(127.4)_8 = 1x8^2 + 2x8^1 + 7x8^0 + 4x8^{-1} = (87.5)_{10}$$

Note that the digits 8 and 9 cannot appear in an octal number. It is customary to borrow the needed r digits for the coefficients from the decimal system when the base of the number is less than 10. The letters of the alphabet are used to supplement the 10 decimal digits when the base of the number is greater than 10. For example, in the hexadecimal (base 16) number system, the first ten digits are borrowed from the decimal system.

The letters A, B, C, D, E, and F are used for digits 10, 11, 12, 13, 14, and 15, respectively. An example of a hexadecimal number is:

$$(B65F)_{16} = 11 \text{ x } 16^3 + 6 \text{ x } 16^2 + 5 \text{ x } 16^1 + 15 \text{ x } 16^0 = (46,687)_{10}$$

As noted, before, the digits in a binary number are called bits. When a bit is equal to 0, it does not contribute to the sum during the conversion. Therefore, the conversion from binary to decimal can be obtained by adding the numbers with powers of two corresponding to the bits that are equal to 1. For example,

$$(110101)_2 = 32 + 16 + 4 + 1 = (53)_{10}$$

There are four 1's in the binary number. The corresponding decimal number is the sum of the four powers of two numbers. In computer work, 2^{10} is referred to as K (kilo), 2^{20} as M (mega), 2^{30} as G (Giga), and 2^{40} as T (Tera). Thus 4K = 2^{12} = 4096 and 16M = 2^{24} = 16,777,216. Computer capacity is usually given in bytes. A byte is equal to

eight bits and can accommodate one keyboard character. A computer hard disk with 4 gigabytes of storage has a capacity of 4G = 2^{32} bytes (approximately 10 billion bytes).

Consider base 10- and 2-digit numbers, we have 00 to 99 (100 different numbers). With 3 digits we have 000 to 999, i.e., 1000 numbers. We can say, there are r^n different combinations of numbers for n digits with radix r, by which we can normally represent

0 – (r^n-1). The minimum number is 0 and the maximum number is (r^n -1). If M is several n digits with radix r; we can write,

$$(r^n - 1) \geq M$$

$$r^n \geq M+1$$

$$n\log r \geq \log(M+1)$$

$$n \geq \frac{\log(M+1)}{\log r}$$

$$n_{min} = \frac{\log(M+1)}{\log r}$$

To represent a number, M in any base, r we can compute the minimum digits required to express the number. Say, for example, we want to represent $(729)_{10}$ in base 2 and base 16. We need 10 and 3 digits respectively.

$$n_2 = \left\lceil \frac{\log(729+1)}{\log 2} \right\rceil$$
$$n_2 = \left\lceil \frac{\log 730}{\log 2} \right\rceil$$
$$n_2 = \lceil 9.5117 \rceil$$
$$n_2 = 10$$

$$n_{16} = \left\lceil \frac{\log(729+1)}{\log 16} \right\rceil$$
$$n_{16} = \left\lceil \frac{\log 730}{\log 16} \right\rceil$$
$$n_{16} = \lceil 2.3779 \rceil$$
$$n_{16} = 3$$

We observe that as the radix increases, the number of required digits decreases.

2.2. Number Base Conversions

The conversion of a number in base `r` to decimal is done by expanding the number in a power series and adding all the terms as shown previously. We now present a general procedure for the reverse operation of converting a decimal number to a number in base `r`. If the number includes a radix point, it is necessary to separate the number into an integer

part and a fraction part, since each part must be converted differently. The conversion of a decimal integer to a number in base `r` is done by dividing the number and all successive quotients by r and accumulating the remainders. This procedure is best illustrated by an example.

Example 1.1: Convert decimal 41 to binary.

First, 41 is divided by 2 to give an integer quotient of 20 and a remainder of ½. The quotient is again divided by 2 to give a new quotient and remainder. This process is continued until the integer quotient becomes 0. The coefficients of the desired binary number are obtained from the remainders as follows:

	Integer Quotient		Remainder	Coefficient
$41/2 =$	20	+	$\frac{1}{2}$	$a_0 = 1$
$20/2 =$	10	+	0	$a_1 = 0$
$10/2 =$	5	+	0	$a_2 = 0$
$5/2 =$	2	+	$\frac{1}{2}$	$a_3 = 1$
$2/2 =$	1	+	0	$a_4 = 0$
$1/2 =$	0	+	$\frac{1}{2}$	$a_5 = 1$

Therefore, the answer is $(41)_{10} = (a_5a_4a_3a_2a_1a_0)_2 = (101001)_2$

The arithmetic process can be manipulated more conveniently as follows:

Integer	Remainder
41	
20	1
10	0
5	0
2	1
1	0
0	1

101001 = answer

The conversion from decimal integers to any base `r` system is similar to the example, except that division is done by r instead of 2.

Example 1.2: Convert decimal 153 to octal.

The required base `r` is 8. First, 153 is divided by 8 to give an integer quotient of 19 and a remainder of 1. Then 19 is divided by 8 to give an integer quotient of 2 and a

remainder of 3. Finally, 2 is divided by 8 to give a quotient of 0 and a remainder of 2. This process can be conveniently manipulated as follows:

153		
19	1	
2	3	
0	2	$= (231)_8$

The conversion of a decimal fraction to binary is accomplished by a method similar to that used for integers. However, multiplication is used instead of division, and integers are accumulated instead of remainders. Again, the method is best explained by example.

Example 1.3: Convert $(0.6875)_{10}$ to binary.

First, 0.6875 is multiplied by 2 to give an integer and a fraction. The new fraction is multiplied by 2 to give a new integer and a new fraction. This process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy. The coefficients of the binary number are obtained from the integers as follows:

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$

Therefore, the answer is $(0.6875)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 = (0.1011)_2$

To convert a decimal fraction to a number expressed in the base `r`, a similar procedure is used. Multiplication is by r instead of 2, and the coefficients found from the integers may range in value from 0 to r - 1 instead of 0 and 1.

Example 1.4: Convert $(0.513)_{10}$ to octal.

$$0.513 \quad X \quad 8 \quad = \quad 4.104$$

$$0.104 \quad X \quad 8 \quad = \quad 7.872$$

The answer, to seven significant figures, is obtained from the integer part of the products

$$(0.513)_{10} = (0.406517\ldots)_8$$

The conversion of decimal numbers with both integer and fraction parts is done by converting the integer and the fraction separately and then combining the two answers. Using the results of Examples 1-1 and 1-3, we obtain

$$(41.6875)_{10} = (101001.1011)_2$$

From Examples 1-2 and 1-4, we have:

$$(153.513)10 = (231.406517)_8$$

The conversion from and to binary, octal, and hexadecimal plays an important role in digital computers. Since $2^3 = 8$ and $2^4 = 16$, each octal digit corresponds to three binary digits and each hexadecimal digit corresponds to four binary digits. The first 16 numbers in the decimal, binary, octal, and hexadecimal number systems are listed in Table 1.

The conversion from binary to octal is easily accomplished by partitioning the binary number into groups of three digits each, starting from the binary point and proceeding to the left and the right. The corresponding octal digit is then assigned to each group. The following example illustrates the procedure:

$$(10\ 110\ 001\ 101\ 011\ 111\ 100\ 000\ 110)_2 = (26153.7460)_8$$

2 6 1 5 3 7 4 0 6

Table 1:Numbers with Different Bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7

08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	101	15	D
14	1110	16	E
15	1111	17	F

Conversion from binary to hexadecimal is similar, except that the binary number is divided into groups of four digits:

$$(10\ 1100\ 0110\ 1011.\ 1111\ 0010)_2 = (2C6B.F2)_{16}$$

2 C 6 B . F 2

The corresponding hexadecimal (or octal) digit for each group of binary digits is easily remembered after studying the values listed in Table 1. Conversion from octal or hexadecimal to binary is done by reversing the preceding procedural. Each octal digit is converted to its three-digit binary equivalent. Similarly, each hexadecimal digit is converted to its four-digit binary equivalent. This is illustrated in the following example:

$$(306.D)_{16} = (0011\ 0000\ 0110\ 1101)_2$$

3 0 6 D

Binary numbers are difficult to work with because they require three or four times as many digits as their decimal equivalent. For example, the binary number 111111111111 is equivalent to decimal 4095. However, digital computers use binary numbers and it is sometimes necessary for the human operator or user to communicate directly with the machine through binary numbers.

One scheme that retains the binary system in the computer, but reduces the number of digits the human must consider, utilizes the relationship between the binary number system and the octal or hexadecimal system. By this method, the human thinks in terms of octal or hexadecimal numbers and performs the required conversion by inspection when direct communication with the machine is necessary. Thus, the binary number

111111111111 has 12 digits and is expressed in octal as 7777 (four digits) or in hexadecimal as FFF (three digits).

During communication between people (about binary numbers in the computer), the octal or hexadecimal representation is more desirable because it can be expressed more compactly with a third or a quarter of the number of digits required for the equivalent binary number. Thus, most computer manuals use either octal or hexadecimal numbers to specify binary quantities. The choice between them is arbitrary, although the hexadecimal tends to win out since it can represent a byte with two digits.

2.3. Data Representation

We enter data into a computer or review (see) output data from a computer using the letter of the alphabet, various special symbols, and the numerals in the decimal number system. But since a computer is an electronic device that understands electrical flow (signal) there is no letter, symbol, or number inside the computer. The computer works with binary numbers. As a semiconductor is conducting or isn't conducting; a switch is closed or opened. Therefore, every data and instruction inside computers is represented using two symbols – 0 and 1. Computers use the binary number system to represent numbers. This system uses only two digits: 0 and 1. Humans use the decimal system which uses ten symbols: 0, 1,9. The binary number system works the same way as the decimal system but has only two available symbols (digits) rather than ten.

Digital systems use signals that have two distinct values and circuit elements that have two stable states. There is a direct analogy among binary signals, binary circuit elements, and binary digits. A binary number of n digits, for example, maybe represented by n binary circuit elements, each having an output signal equivalent to 0 or 1. Digital systems represent and manipulate not only binary numbers but also many other discrete elements of information. Any discrete element of information distinct among a group of quantities can be represented with a binary code. The codes must be in binary because computers can only hold 1's and 0's. It must be realized that binary codes merely change the symbols, not the meaning of the elements of information that they represent. If we inspect the bits of a

computer at random, we will find that most of the time they represent some type of coded information rather than binary numbers.

An n-bit binary code is a group of n bits that assume up to 2n distinct combinations of 1's and 0's, with each combination representing one element of the set that is being coded. A set of four elements can be coded with two bits, with each element assigned one of the following bit combinations: 00, 01, 10, 11. A set of eight elements requires a 3-bit code and a set of 16 elements requires a 4-bit code. The bit combination of a w-bit code is determined from the count in binary from 0 to 2" - 1. Each element must be assigned a unique binary bit combination and no two elements can have the same value; otherwise, the code assignment will be ambiguous.

Although the minimum number of bits required to code 2n distinct quantities is n, there is no maximum number of bits that may be used for a binary code. For example, the 10 decimal digits can be coded with 10 bits, and each decimal digit can be assigned a bit combination of nine 0's and a 1. In this particular binary code, the digit 6 is assigned the bit combination 0001000000.

Exercise

1. How do you convert binary numbers to hexadecimal?
2. Find the octal equivalent of 1100101.001010.
3. Convert the hexadecimal number 0x2301 to its binary equivalent
4. Convert (22)8 into its corresponding decimal number
5. Convert (5401)8 to hexadecimal.
6. Convert (52)16 into its decimal equivalent
7. Find hexadecimal Addition of (3A5)16 and (1B2)16
8. Convert the hexadecimal number B2 to binary
9. Convert the hexadecimal number 2C to decimal
10. Convert the decimal number 168 to the binary equivalent

References

1. Ifrah, G. (2000). The universal history of numbers. London: Harvill.
2. Thurston, H. A. (2012). The number system. Courier Corporation.
3. Sakk, E. (2018). Understanding Binary Numbers: Concepts in Computer Systems (Volume 1).

Chapter Three

3. Introduction to Programming

Objectives: Software is written using a programming language. There are several programming languages from which programmers could choose one. Software development has different phases that could follow a certain paradigm.

At the end of this chapter, students should be able to:

- *explain computer programming*
- *explain programming language*
- *identify the various types of programming languages*
- *identify the major programming paradigms*
- *identify the various software development life cycle models*

3.1. What is computer programming?

In essence, the word programming refers to giving directions for the computer to accomplish a task. Computer programming (often shortened to programming or coding) is the process of writing, testing, debugging, and maintaining the source code of computer programs [1]. Programming is a skill that can be acquired by a computer professional that gives him/her the knowledge of making the computer perform the required operation or task [2, 3]. Writing computer programs means writing instructions that will make the computer follow and run a program based on those instructions. Each instruction is relatively simple, yet because of the computer's speed, it can run millions of instructions in a second.

Programming Language: is a set of different categories of written symbols that instruct computer hardware to perform specified operations required by the designer.

3.2. What Skills Do We Need to Be a Programmer?

For someone to be a programmer, in addition to basic skills in computer, he/she needs to have the following major skills [4]:

- **Programming Language Skill:** knowing one or more programming languages to talk to the computer and instruct the machine to perform a task.
- **Problem Solving Skill:** skills on how to solve the real-world problem and represent the solution in an understandable format.
- **Algorithm Development Skill**: the skill of coming up with the sequence of a simple and human-understandable set of instructions showing the step of solving the problem. Those set of steps should not be dependent on any programming language or machine.

In every programming language, there are sets of rules that govern the symbols to be used in the programming language. This set of rules determine how the programmer can make the computer hardware to perform a specific operation. These sets of rules are called syntax.

3.3. Reasons to Study Programming

- Increased capacity to express programming concepts
- Increased ability to learn new languages
- Understand the significance of implementation
- Increased ability to design new languages
- Mastering different programming paradigms
- To solve problems

3.4. What is a Programming Language?

Programming Language is a set of different categories of written symbols that instruct computer hardware to perform specified operations required by the designer. Programming Language can be dividing into three major categories.

a. Machine Languages

Within a computer, all data is represented by microscopic electronic switches that can be either off or on. The off switch is designated by a 0, and the on switch is designated by a 1. Because computers can understand only these on and off switches, the first programmers had to write the program instructions using nothing but combinations of 0s and 1s. For example, a program might contain the instruction 00101 10001 10000. Instructions written in 0s and 1s are called machine language or machine code. The machine languages (each type of machine has its language) represent the only way to communicate directly with the computer. As you can imagine, programming in machine language is very tedious and error-prone and requires highly trained programmers [4].

The advantage of machine language is the fact that it is translation free and high speed and while its disadvantage is that machine language is machine-dependent, complex, and error-prone.

b. Assembly Languages

Assembly language is close to a one-to-one correspondence between symbolic instruction and executable machine code and was created to use letters instead of 0s and 1s. For example, most assembly languages use the mnemonic ADD to represent an add operation and the mnemonic MUL to represent a multiply operation. An example of an instruction written in an assembly language is ADD BX, AX. Programs written in an assembly language require an assembler, which also is a program, that converts the assembly instructions into machine code the 0s and 1s the computer can understand.

The advantage of assembly language is that it is easy to understand and use, less error-prone. Its disadvantages are its machine dependency and harder to learn natures.

c. High-Level Languages

High-level languages represent the next major development in programming languages. High-level languages are a vast improvement over machine and assembly languages because they allow the programmer to use instructions that more closely resemble the English language. An example of an instruction written in a high-level language is,

*grossPay = hours * rate.*

Besides, high-level languages are more machine-independent than are machine and assembly languages. As a result, programs written in a high-level language can be used in many different types of computers. Programs are written in a high-level language usually require a compiler, which also is a program, to convert the English-like instructions into the 0s and 1s the computer can understand.

Advantages of high-level language are its readability, machine independence, easy debugging, easier maintenance, and low development cost nature. Its disadvantages, on the other hand, are its poor control in hardware and less efficiency

3.5. Generation of Programming Language

Programming languages are categorized into five generations, namely 1st, 2nd, 3rd, 4th, and 5th generation languages.

- These programming languages can also be categorized into two broad categories: low level and high-level languages.
- Low-level languages are machine specific or dependent.
- High-level languages like COBOL and BASIC are machine independent and can run on a variety of computers.

- From the five categories of programming languages, first- and second-generation languages are low-level languages and the remaining categories are high-level programming languages.
- The higher the level of a language, the easier it is to understand and use by programmers.
- Languages after the fourth generation are referred to as a very high-level language.

3.5.1. First Generation (Machine languages, 1940's)

- Difficult to write applications with.
- Dependent on machine languages of the specific computer being used.
- Machine languages allow the programmer to interact directly with the hardware, and it can be executed by the computer without the need for a translator.
- Is more powerful in utilizing the resources of the computer.
- They execute very quickly and use memory very efficiently.

3.5.2. Second Generation (Assembly languages, early 1950's)

- Uses symbolic names for operations and storage locations.
- A system program called an assembler translates a program written in assembly language to machine language.
- Programs written in assembly language are not portable. That is different computer architectures have their machine and assembly languages.
- They are highly used in system software development.

3.5.3. Third Generation (High-level languages, 1950's to 1970s)

- Uses English like instructions and mathematicians were able to define variables with statements such as Z = A + B
- Such languages are much easier to use than assembly language.
- Programs written in high-level languages need

- The use of common words (reserved words) within instructions makes it easier to learn.
- All third-generation programming languages are procedural languages.
- In procedural languages, the programmer is expected to specify what is required and how to perform it.

3.5.4. Fourth Generation (since the late 1970s)

- Have a simple, English language like syntax rules; commonly used to access databases.
- Fourth-generation languages have a minimum number of syntax rules. This saves time and free professional programmers for more complex tasks.
- Some examples of 4GL are structured query languages (SQL), report generators, application generators, and graphics languages.

3.5.5. Fifth Generation (1990's)

- These are used in artificial intelligence (AI) and expert systems; also used for accessing databases.
- 5GLs are "natural" languages whose instruction closely resembles human speech.
- 5GLs require very powerful hardware and software because of the complexity involved in interpreting commands in human language.

3.6. Major programming Paradigm

The major landmarks in the programming world are the different kinds of features or properties observed in the development of programming languages. As might be expected in a dynamic and evolving field, there is no single standard for classifying programming languages. Another most fundamental way programming languages are characterized (categorized) is by programming paradigm. A programming paradigm provides the programmer's view of code execution. Among these, the following are worth mentioning: *Procedural, Structured, and Object-Oriented Programming Paradigms* [3].

3.6.1. Procedural Programming Paradigms

Procedural programming specifies a list of operations that the program must complete reaching the desired state [1, 2]. Each program has a starting state, a list of operations to complete, and an ending point. This approach is also known as imperative programming. Integral to the idea of procedural programming is the concept of a procedure call.

Procedures, also known as functions, subroutines, or methods, are small sections of code that perform a particular function. A procedure is effectively a list of computations to be carried out. Procedural programming can be compared to unstructured programming, where all of the code resides in a single large block [5]. By splitting the programmatic tasks into small pieces, procedural programming allows a section of code to be re-used in the program without making multiple copies. It also makes it easier for programmers to understand and maintain the program structure.

Two of the most popular procedural programming languages are FORTRAN and BASIC.

3.6.2. Structured Programming Paradigms

Structured programming is a special type of procedural programming. It provides additional tools to manage the problems that larger programs were creating. Structured programming requires that programmers break program structure into small pieces of code that are easily understood. It also frowns upon the use of global variables and instead uses variables local to each subroutine. One of the well-known features of structural programming is that it does not allow the use of the GOTO statement. It is often associated with a "top-down" approach to design. The top-down approach begins with an initial overview of the system that contains minimal details about the different parts. Subsequent design iterations then add increasing detail to the components until the design is complete.

The most popular structured programming languages include C, Ada, and Pascal.

3.6.3. Object-Oriented Programming Paradigms

Object-oriented programming is one of the recent and most powerful paradigms. In object-oriented programs, the designer specifies both the data structures and the types of operations that can be applied to those data structures. This pairing of a piece of data with the operations that can be performed on it is known as an object. A program, thus, becomes a collection of cooperating objects, rather than a list of instructions. Objects can store state information and interact with other objects, but generally, each object has a distinct, and limited role.

3.7. Problem Solving Process and Software Engineering

3.7.1. Software Engineering

Software engineering is the profession that creates and maintains software applications by applying technologies and practices from computer science, project management, engineering, application domain, and other fields. The method used in solving problems in computer science and/or information systems is called the software development life cycle (SDLC).

The software development life cycle has the following phases.

- *Feasibility study, requirement gathering, and analysis*
- *Design*
- *Implementation or coding*
- *Testing*
- *Deployment and maintenance*

3.7.2. Software Development Life Cycle (SDLC)

SDLC is a process that defines the various stages involved in the development of software for delivering a high-quality product. SDLC stages cover the complete life cycle of

software i.e., from inception to retirement of the product. The purpose of SDLC is to deliver a high-quality product that is as per the customer's requirement.

SDLC has defined its phases as, Requirement gathering, Designing, Coding, Testing, and Maintenance. It is important to adhere to the phases to provide the Product systematically.

For Example, the software has to be developed and a team is divided to work on a feature of the product and is allowed to work as they want. One of the developers decides to design first whereas the other decides to code first and the other on the documentation part.

This will lead to project failure because of which it is necessary to have a good knowledge and understanding among the team members to deliver an expected product.

3.7.2.1. Feasibility study, Requirement Gathering and Analysis

The first step in software development is to identify a need for the new system. This will include determining whether a business problem or opportunity exists, conducting a feasibility study to determine if the proposed solution is cost-effective, and developing a project plan.

Ideally, the process occurs in tandem with a review of the organization's strategic plan to ensure that IT is being used to help the organization achieve its strategic objectives. Management may need to approve concept ideas before any money is budgeted for its development.

A preliminary analysis, determining the nature and scope of the problems to be solved is carried out. Possible solutions are proposed, describing the cost and benefits. Finally, a preliminary plan for decision making is produced.

The process of developing a large information system can be very costly, and the investigation stage may require a preliminary study called a feasibility study, which includes e.g., the following components:

a. Organizational Feasibility
   - How well the proposed system supports the strategic objectives of the organization.

b. Economic Feasibility
   - Cost savings
   - Increased revenue
   - Decreased investment
   - Increased profits

c. Technical Feasibility
   - Hardware, software, and network capability, reliability, and availability

d. Operational Feasibility
   - End-user acceptance
   - Management support
   - Customer, supplier, and government requirements

Requirements analysis is the process of analyzing the information needs of the end-users, the organizational environment, and any system presently being used, developing the functional requirements of a system that can meet the needs of the users. Also, the requirements should be recorded in a document, email, user interface storyboard, executable prototype, or some other form. The requirements documentation should be referred to throughout the rest of the system development process to ensure the developing project aligns with user needs and requirements.

End-users must be involved in this process to ensure that the new system will function adequately and meets their needs and expectations.

During the requirement gathering and analysis phase, all the relevant information is collected from the customer to develop a product as per their expectation. Any ambiguities must be resolved in this phase only.

The business analyst and Project Manager set up a meeting with the customer to gather all the information like what the customer wants to build, who will be the end-user, what is

the purpose of the product. Before building a product a core understanding or knowledge of the product is very important.

For example, a customer wants to have an application that involves money transactions. In this case, the requirement has to be clear like what kind of transactions will be done, how it will be done, in which currency it will be done, etc.

Once the requirement gathering is done, an analysis is done to check the feasibility of the development of a product. In case of any ambiguity, a call is set up for further discussion.

Once the requirement is clearly understood, the SRS (Software Requirement Specification) document is created. This document should be thoroughly understood by the developers and also should be reviewed by the customer for future reference.

3.7.2.2. Design

After the requirements have been determined, the necessary specifications for the hardware, software, people, and data resources, and the information products that will satisfy the functional requirements of the proposed system can be determined. The design will serve as a blueprint for the system and helps detect problems before these errors or problems are built into the final system.

The created system design, but must be reviewed by users to ensure the design meets users' needs.

In this phase, the requirement gathered in the SRS document is used as an input, and software architecture that is used for implementing system development is derived.

3.7.2.3. Implementation or Coding

The real code is written here. Systems implementation is the construction of the new system and its delivery into production or day-to-day operation. The key to understanding the implementation phase is to realize that there is a lot more to be done than programming. Implementation requires programming, but it also requires database creation and population, and network installation and testing. You also need to make sure the people are

taken care of with effective training and documentation. Finally, if you expect your development skills to improve over time, you need to conduct a review of the lessons learned.

3.7.2.4. Testing

Testing starts once the coding is complete and the modules are released for testing. In this phase, the developed software is tested thoroughly and any defects found are assigned to developers to get them fixed. Retesting, regression testing is done until the point at which the software is as per the customer's expectation.

a. ***Unit testing:*** Testers refer to the SRS document to make sure that the software is as per the customer's standard. Normally programs are written as a series of individual modules, these subject to separate and detailed tests.
b. ***Integration and System testing:*** Brings all the pieces together into a special testing environment, then checks for errors, bugs, and interoperability. The system is tested to ensure that interfaces between modules work (integration testing), the system works on the intended platform and with the expected volume of data (volume testing) and that the system does what the user requires (acceptance/beta testing).

3.7.2.5. Deployment and Maintenance

What happens during the rest of the software's life: changes, correction, additions, moves to a different computing platform, and more. This, the least glamorous and perhaps most important step of all, goes on seemingly forever. Once the product is tested, it is deployed in the production environment or the first UAT (User Acceptance testing) is done depending on the customer expectation.

In the case of UAT, a replica of the production environment is created and the customer along with the developers does the testing. If the customer finds the application as expected, then sign off is provided by the customer to go live.

After the deployment of a product in the production environment, maintenance of the product i.e., if any issue comes up and needs to be fixed or any enhancement is to be done is taken care of by the developers.

3.7.3. Software Development Life Cycle Models

A software life cycle model is a descriptive representation of the software development cycle. SDLC models might have a different approach but the basic phases and activity remain the same for all the models [6].

3.7.3.1. Waterfall Model

The waterfall model is the very first model that is used in SDLC. It is also known as the linear sequential model. In this model, the outcome of one phase is the input for the next phase. Development of the next phase starts only when the previous phase is complete.

- First, Requirement gathering and analysis is done. Once the requirement is freezing then only the System Design can start. Herein, the SRS document created is the output for the Requirement phase and it acts as an input for the System Design.
- In System Design Software architecture and Design, documents that act as an input for the next phase are created i.e., Implementation and coding.
- In the Implementation phase, coding is done and the software developed is the input for the next phase i.e., testing.
- In the testing phase, the developed code is tested thoroughly to detect the defects in the software. Defects are logged into the defect tracking tool and are retested once fixed. Bug logging, Retest, Regression testing go on until the time the software is in a go-live state.
- In the Deployment phase, the developed code is moved into production after the sign off is given by the customer.
- Any issues in the production environment are resolved by the developers which come under maintenance.

Advantages of the Waterfall Model:

- The waterfall model is the simple model that can be easily understood and is the one in which all the phases are done step by step.
- Deliverables of each phase are well defined, and this leads to no complexity and makes the project easily manageable.

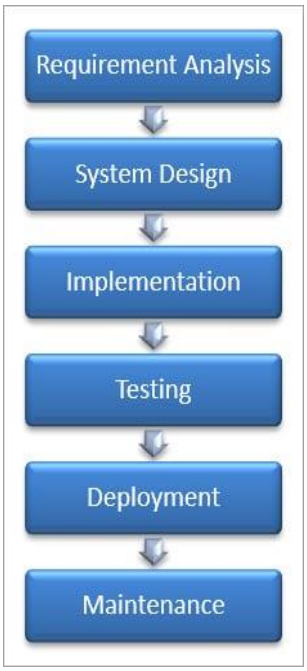


Figure 16: Flow of activities in waterfall model software development life cycle

Disadvantages of Waterfall model:

- The waterfall model is time-consuming & cannot be used in the short duration projects as in this model a new phase cannot be started until the ongoing phase is completed.
- Waterfall model cannot be used for the projects which have uncertain requirement or wherein the requirement keeps on changing as this model expects the requirement to be clear in the requirement gathering and analysis phase itself and any change in the later stages would lead to cost higher as the changes would be required in all the phases.

3.7.3.2. V-Shaped Model

V- Model is also known as Verification and Validation Model. In this model Verification & Validation go hand in hand i.e., development and testing go parallel. V model and waterfall model are the same except that the test planning and testing start at an early stage in V-Model.

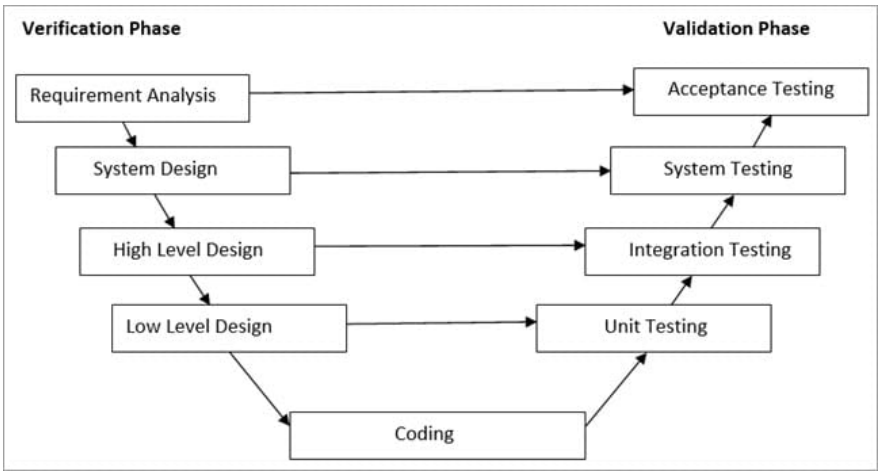


Figure 17. The flow of activities in the V-shaped model software development life cycle

e. Verification Phase:

a. **Requirement Analysis:** In this phase, all the required information is gathered & analyzed. Verification activities include reviewing the requirements.

b. **System Design:** Once the requirement is clear, a system is designed i.e., architecture, components of the product are created and documented in a design document.

c. **High-Level Design:** High-level design defines the architecture/design of modules. It defines the functionality between the two modules.

d. **Low-Level Design:** Low-level Design defines the architecture/design of individual components.

e. **Coding**: Code development is done in this phase.

f. Validation Phase:

(i) **Unit Testing:** Unit testing is performed using the unit test cases that are designed and are done in the Low-level design phase. Unit testing is performed by the developer itself. It is performed on individual components which leads to early defect detection.

(ii) **Integration Testing:** Integration testing is performed using integration test cases in the High-level Design phase. Integration testing is the testing that is done on integrated modules. It is performed by testers.

(iii) **System Testing:** System testing is performed in the System Design phase. In this phase, the complete system is tested i.e., the entire system functionality is tested.

(iv) **Acceptance Testing:** Acceptance testing is associated with the Requirement Analysis phase and is done in the customer's environment.

Advantages of V – Model:

- It is a simple and easily understandable model.
- V –model approach is good for smaller projects wherein the requirement is defined and it freezes in the early stage.
- It is a systematic and disciplined model that results in a high-quality product.

Disadvantages of V-Model:

- The V-shaped model is not good for ongoing projects.
- Requirement change at the later stage would cost too high.

3.7.3.3. Prototype Model

The prototype model is a model in which the prototype is developed before the actual software.

Prototype models have limited functional capabilities and inefficient performance when compared to the actual software. Dummy functions are used to create prototypes. This is a valuable mechanism for understanding the customers' needs.

Software prototypes are built before the actual software to get valuable feedback from the customer. Feedbacks are implemented and the prototype is again reviewed by the customer for any change. This process goes on until the model is accepted by the customer.

Once the requirement gathering is done, the quick design is created and the prototype which is presented to the customer for evaluation is built.

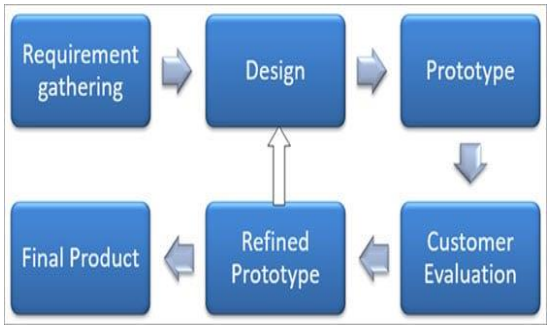


Figure 18. Flow of activities in Prototype model of software development Life cycle

Customer feedback and the refined requirement is used to modify the prototype and is again presented to the customer for evaluation. Once the customer approves the prototype, it is used as a requirement for building the actual software. The actual software is build using the Waterfall model approach.

Advantages of Prototype Model:

- The prototype model reduces the cost and time of development as the defects are found much earlier.
- Missing feature or functionality or a change in requirement can be identified in the evaluation phase and can be implemented in the refined prototype.
- Involvement of a customer from the initial stage reduces any confusion in the requirement or understanding of any functionality.

Disadvantages of Prototype Model:

- Since the customer is involved in every phase, the customer can change the requirement of the end product which increases the complexity of the scope and may increase the delivery time of the product.

3.7.3.4. Spiral Model

The Spiral Model includes an iterative and prototype approach. Spiral model phases are followed in the iterations. The loops in the model represent the phase of the SDLC process i.e., the innermost loop is of requirement gathering & analysis which follows the Planning, Risk analysis, development, and evaluation.

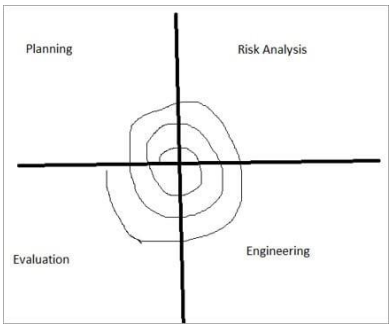


Figure 19. Flow of activities in Spiral model software development life cycle

The next loop is Designing followed by Implementation & then testing.

<u>Spiral Model has four phases:</u>

- Planning
- Risk Analysis
- Engineering
- Evaluation

(ii) **Planning:** The planning phase includes requirement gathering wherein all the required information is gathered from the customer and is documented. A software requirement specification document is created for the next phase.

(iii) **Risk Analysis:** In this phase, the best solution is selected for the risks involved and analysis is done by building the prototype. *For Example, the risk involved in accessing the data from a remote database can be that the data access rate might be too slow. The risk can be resolved by building a prototype of the data access subsystem.*

(iv) **Engineering:** Once the risk analysis is done, coding and testing are done.

(v) **Evaluation:** Customer evaluates the developed system and plans for the next iteration.

<u>Advantages of Spiral Model:</u>

- Risk Analysis is done extensively using the prototype models.

- Any enhancement or change in the functionality can be done in the next iteration.

Disadvantages of Spiral Model:

- The spiral model is best suited for large projects only.
- The cost can be high as it might take a large number of iterations which can lead to high time to reach the final product.

3.7.3.5. Iterative Incremental Model

The iterative incremental model divides the product into small chunks.

For Example, Feature to be developed in the iteration is decided and implemented. Each iteration goes through the phases namely Requirement Analysis, Designing, Coding, and Testing. Detailed planning is not required in iterations.

Once the iteration is completed, a product is verified and is delivered to the customer for their evaluation and feedback. Customer's feedback is implemented in the next iteration along with the newly added feature.

Hence, the product increments in terms of features, and once the iterations are completed the final build holds all the features of the product.

Phases of Iterative & Incremental Development Model:

a) Inception phase
b) Elaboration Phase
c) Construction Phase
d) Transition Phase

(i) **Inception Phase:** The inception phase includes the requirement and scope of the Project.

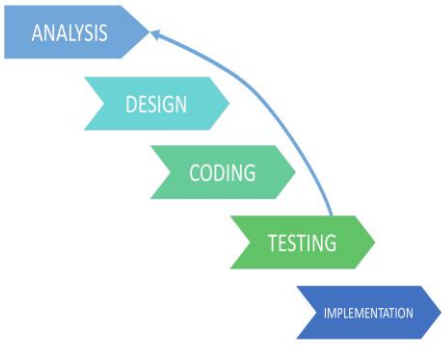


Figure 20. Flow of activities in Iterative Incremental model software development life cycle

(ii) **Elaboration Phase:** In the elaboration phase, the working architecture of a product is delivered which covers the risk identified in the inception phase and also fulfills the non-functional requirements.

(iii) **Construction Phase:** In the Construction phase, the architecture is filled in with the code which is ready to be deployed and is created through analysis, designing, implementation, and testing of the functional requirement.

(iv) **Transition Phase:** In the Transition Phase, the product is deployed in the Production environment.

Advantages of Iterative & Incremental Model:

- Any change in the requirement can be easily done and would not cost as there is a scope of incorporating the new requirement in the next iteration.
- Risk is analyzed & identified in the iterations.
- Defects are detected at an early stage.
- As the product is divided into smaller chunks it is easy to manage the product.

Disadvantages of Iterative & Incremental Model:

- Complete requirements and understanding of a product are required to break down and build incrementally.

3.7.3.6. Agile Model

The agile model is a combination of the Iterative and incremental model. This model focuses more on flexibility while developing a product rather than on the requirement.

In Agile, a product is broken into small incremental builds. It is not developed as a complete product in one go. Each build increments in terms of features. The next build is built on previous functionality.

In agile iterations are termed as sprints. Each sprint lasts for2-4 weeks. At the end of each sprint, the product owner verifies the product and after his approval, it is delivered to the customer.

Customer feedback is taken for improvement and his suggestions and enhancement are worked on in the next sprint. Testing is done in each sprint to minimize the risk of any failures.

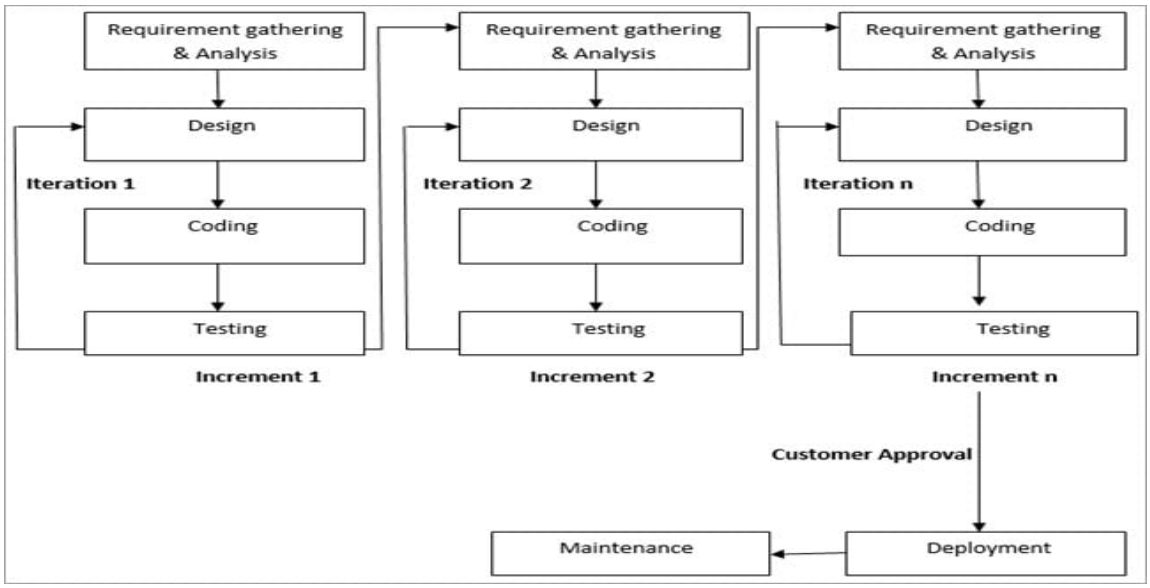


Figure 21. The flow of activities in an Agile model software development life cycle

Advantages of Agile Model:

- It allows more flexibility to adapt to the changes.
- The new feature can be added easily.
- Customer satisfaction as the feedback and suggestions are taken at every stage.

Disadvantages:

- Lack of documentation.
- Agile needs experienced and highly skilled resources.
- If a customer is not clear about how exactly they want the product to be, then the project would fail.

***Note**: Different Software Development Life Cycle models have their Pros and Cons. The best model for any Project can be determined by the factors like Requirement (whether it is clear or unclear), System Complexity, Size of the Project, Cost, Skill limitation, etc.*

***For example**, in case of an unclear requirement, Spiral and Agile models are best to be used as the required change can be accommodated easily at any stage.*

3.7.4. Problem Solving Process

Problem-solving is the process of transforming the description of a problem into a solution by using our knowledge of the problem domain and by relying on our ability to select and use appropriate problem-solving strategies, techniques, and tools.

A *problem* is an undesirable situation that prevents the organization from fully achieving its purpose, goals, and objectives. Or the problem can also be defined as the gap between the existing and the desired situation where problem-solving will try to fill this gap [7].

There are two approaches to problem-solving:

i. **Top-down Design:** is a systematic approach based on the concept that the structure of the problem should determine the structure of the solution and what should be done at a lower level. This approach will try to disintegrate a larger problem into more small and manageable problems to narrow the problem domain.

for example: - This top-down approach is essential if the work has to be shared out between a team of programmers, each programmer ending up with a specification for a part of the system which is to be written as a function (or modules). While writing a single module the programmer can concentrate on the solution of this one problem only and is thus more likely to be able to solve the problem and make fewer errors

Advantages:

- The main advantage of the top-down approach is that its strong focus on requirements helps to make a design responsive according to its requirements.

Disadvantages:

- Project and system boundaries tend to be application specification oriented. Thus, it is more likely that the advantages of component reuse will be missed.
- The system is likely to miss, the benefits of a well-structured, simple architecture.

ii. Bottom-up Design: is the reverse process where the lowest level component is built first and the system builds up from the bottom until the whole process is finally completed. The design starts with the lowest level components and subsystems. By using these components, the next immediate higher-level components and subsystems are created or composed. The process is continued till all the components and subsystems are composed into a single component, which is considered as the complete system. The amount of abstraction grows high as the design moves to more high levels. By using the basic information existing system, when a new system needs to be created, the bottom-up strategy suits the purpose.

Advantages:

- Economics can result when general solutions can be reused.
- It can be used to hide the low-level details of implementation and be merged with the top-down technique.

Disadvantages:

- It is not so closely related to the structure of the problem.
- High-quality bottom-up solutions are very hard to construct.
- It leads to a proliferation of 'potentially useful' functions rather than the most appropriate ones.

Key Differences Between Top-down and Bottom-up Approach

i. The top-down approach decomposes the large task into smaller subtasks whereas the bottom-up approach first chooses to solve the different fundamental parts of the task directly then combine those parts into a whole program.

ii. Each submodule is separately processed in a top-down approach. As against, the bottom-up approach implements the concept of the information hiding by examining the data to be encapsulated.

iii. The different modules in a top-down approach don't require much communication. On the contrary, the bottom-up approach needs interaction between the separate fundamental modules to combine them later.

iv. The top-down approach can produce redundancy while the bottom-up approach does not include redundant information.

v. The procedural programming languages such as Fortran, COBOL, and C follow a top-down approach. In contrast, object-oriented programming languages like C++, Java, C#, Perl, Python abides the bottom-up approach.

vi. The bottom-up approach is best for testing. Conversely, the top-down approach is utilized in module documentation, test case creation, debugging, etcetera.

Exercises

1. Define the term programming, computer programing, and programming language
2. Write the categories of programming language
3. What are the futures under each various generation of programming language?
4. Explain and all phase of SDLC
5. Write the difference between problem-solving and algorithm development skills.
6. Write the difference between top-down and bottom-up approaches.
7. Write the advantage and disadvantages of the top-down and bottom-up approach.
8. Which type of programming is a better choice for large sequential programs?
9. What skills do we need to be a programmer?
10. Differentiate the following statements?
    a) Machine language
    b) Assembly language
    c) High-level language
11. Which type of software development method is a rigid linear model that consists of sequential phases (requirements, design, implementation, verification, maintenance) focusing on distinct goals
12. List and explain the steps for the Software Development Life Cycle (SDLC).
13. Compare and contrast the following Software Development Life Cycle (SDLC) models?
    a) Waterfall vs. V – Model
    b) Prototype Model vs. Spiral Model
    c) Iterative and Incremental Model vs. Agile Model

References

[1] Raphael Finkel, Advanced Programming Language Design, Addison Wesley, 2016.

[2] Daniel P. Friedman; Mitchell Wand; Christopher T. Haynes, Essentials of Programming Languages, The MIT Press, 2011.

[3] Maurizio Gabbrielli and Simone Martini, Programming Languages: Principles and Paradigms, Springer, 2019.

[4] Ellis Horowitz, Fundamentals of Programming Languages, 2009.

[5] David A. Watt. , Programming Language Concepts and Paradigms, Prentice Hall, 2012.

[6] Bruce J. MacLennan, Principles of Programming Languages: Design, Evaluation, and Implementation, Oxford University Press, 2009.

[7] Terrence W. Pratt and Marvin V. Zelkowitz, Programming Languages: Design and Implementation (4th ed.), Prentice-Hall, 2010.

Chapter Four

4. Algorithm Development and Problem-Solving Techniques

Objectives: Given a certain problem that can be solved with a computer, the solution to the problem follows step-by-step techniques.

At the end of this chapter, the student should be able to:

- *understand the concept of computational problems*
- *understand what computer algorithms are*
- *use pseudo-code to prepare algorithms for problems*
- *use flow-chart to develop algorithms for problems*

4.1. Computational Problems

Computational problems are problems that can be solved step-by-step with a computer. These problems usually have well-defined input, constraints, and conditions that output must satisfy. Below is a brief explanation of the common computational problems.

A. Decision Problems

These are types of problems where the solution is a yes or a no answer. For example, given a natural number N, the problem is `N` even? is a decision problem. Given a natural number N, the following are typical examples of decision problem:

- Is N a prime number?
- Is N greater than 97?
- Is 5 a factor of N?

Some decision problems take more steps to solve than other decision problems. For instance, given a natural number n, the decision problem “is N a prime number?” takes more steps than the computational problem of “is Nan odd number?” To check if a natural number is odd or not, it is enough just to check if the modulus of the given

number and 2 (N%2) is 1 or not. If N%2 is 1, N is odd. Checking whether a given natural number is prime or not, on the other hand, needs additional steps. A natural number N is said to be a prime number if the number has only two factors, that is, its factors are 1 and N (the number itself). Thus, the primality test needs to check all the natural numbers in the range [2, N-1] if one of the numbers in the range is a factor of the number. If one natural number is found, then the natural number N is not a prime number.

B. Search Problem

These are types of computational problems where the solution of the problem consists of one or more values that satisfy a given condition. For example, given a list of natural numbers, identifying the maximum of the list could be a search problem.

C. Counting Problems

Counting problems are those where the answer is the number of solutions to a search problem

D. Optimization problem's

The optimization problem is one where the solution is the "best" possible solution, where the "best" can be defined differently. For instance, we may want to compute the fastest route from one location to another.

Questions such as "what is the meaning of life?" "Do I look good in this outfit?" are not computational problems because they do not have well-defined input, constraints, and conditions that the output must satisfy.

4.2. Introduction about Algorithm

The term algorithm originally referred to any computation performed via a set of rules applied to numbers written in decimal form. The term can include a different set of rules for performing the four basic arithmetic operations (addition, subtraction, multiplication, and division) on decimal numbers.

An algorithm is a representation of a solution to a problem. If a problem can be defined as a difference between the desired situation and the current situation in which one is, then a problem solution is a procedure, or method, for transforming the current situation to the desired one. We solve many such trivial problems every day without even thinking about it, for example making breakfast, traveling to the workplace, etc. But the solution to such problems requires little intellectual effort and is relatively unimportant. However, the solution to a more interesting problem of more importance usually involves stating the problem in an understandable form and communicating the solution to others. In the case where a computer is part of the means of solving the problem, a procedure, explicitly stating the steps leading to the solution, must be transmitted to the computer. This concept of problem solution and communication makes the study of algorithms important to computer science.

Throughout history, man has thought of ever more elegant ways of reducing the amount of labor needed to do things. A computer has immense potential for saving time/energy as most (computational) tasks that are repetitive or can be generalized can be done by a computer. For a computer to perform the desired task, a method for carrying out some sequence of events, resulting in accomplishing the task, must somehow be described to the computer. The algorithm can be described on many levels because the algorithm is just the procedure of steps to take and get the result. The language used to describe an algorithm to other people will be quite different from that which is used by the computer; however, the actual algorithm will in essence be the same. An example of an algorithm people use would be a recipe to make a cake.

The computer solves a variety of problems that can be expressed in a finite number of steps leading to a precisely defined goal by writing different programs. A program is not needed only to solve a problem but also it should be reliable, (maintainable) portable, and efficient. In computer programming, two facts are given more weight:

- The first part focuses on defining the problem and logical procedures to follow in solving it.

- The second introduces how programmers communicate those procedures to the computer system so that it can be executed.

An algorithm is defined as a step-by-step sequence of instructions that must terminate and describe how the data is to be processed to produce the desired outputs. Simply, the algorithm is a sequence of instructions. Algorithms are a fundamental Problem-solving Computer Program, which is a logical sequence of steps for solving a problem often written out as a flowchart, and Pseudocode. We will use those methods here. Generally, flowcharts work well for small problems but Pseudocode is used for larger ones.

Problem Solving consists of the following technical Activities:

a) Algorithm development,
b) Flowcharting or Pseudo-coding,
c) Coding in a particular programming language.

Algorithm development is the act of designing the steps that solve a particular problem for a computer or any other device to follow not excluding human beings but in this case computers only and computer-like devices.

1. Problem definition
2. Development of a model
3. Specification of Algorithm
4. Designing an Algorithm
5. Checking the correctness of Algorithm
6. Analysis of Algorithm
7. Implementation of Algorithm
8. Program testing
9. Document preparation

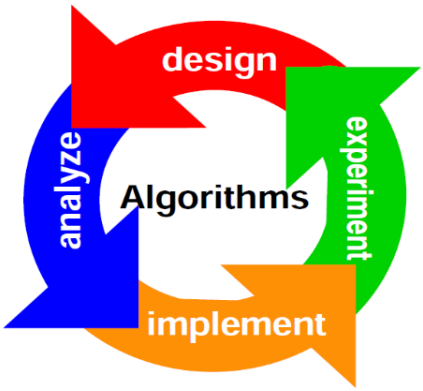


Figure 22: Algorithm development steps

4.3. Pseudo-code

Pseudo-code (derived from pseudo and code) is a compact and informal high-level description of a computer algorithm that uses the structural conventions of programming languages, but typically omits details such as subroutines, variables declarations, and system-specific syntax. The programming language is augmented with natural language descriptions of the details, where convenient, or with compact mathematical notation. The purpose of using pseudo-code is that it may be easier for humans to read pseudo-code than conventional programming languages. Furthermore, pseudo-code may be a compact and environment-independent generic description of the key principles of an algorithm.

Pseudo-code Structure

Input: READ, OBTAIN, GET and PROMPT

Output: PRINT, DISPLAY, and SHOW

Compute: COMPUTE, CALCULATE, and DETERMINE

Initialize: SET and INTIALIZE

Add one: INCREMENT

Example 1:

> *Original Program Specification: Write a program that obtains two integer numbers from the user and displays the sum of the two numbers.*

Pseudocode 1:

1. *Prompt the user to enter the first integer*
2. *Prompt the user to enter a second integer*
3. *Compute the sum of the two user inputs*
4. *Display an output prompt that shows the answer as the sum*

Pseudocode 2:

1. *Read the values of the two numbers (A and B)*

2. *Compute the sum of A and B*
3. *Set the sum to C*
4. *Display the result (C)*

Example 2:

Original program specification: Write a program that obtains a natural number from the user and display the sum of all the digits of the number.

Pseudocode:

1. *Start*
2. *Prompt the user to enter a natural number, N*
3. *Set sum to 0*
4. *Check if N is greater than 0*

   4.1. If N is greater than 0

   4.1.1. Increment the sum by N modulo 10

   4.1.2. Set N to N/10

   4.1.3. Repeat step 4

   4.2. If N is not greater than 0

   4.2.1. Display the sum result

   4.2.2. Terminate

Example 3:

Original Program Specification: Write a program that takes a natural number say N and displays the first N natural numbers.

Pseudo-code:

1. *Start*
2. *Read the number of natural numbers, N to display*
3. *Set the first natural number, I to 1*
4. *If I < N*

   4.1. Print I

   4.2. Increment I by 1

4.3.Repeat step 4

5. *Else*

   5.1.Stop

Example 4:

Original Program Specification: Write a pseudo program that checks whether a given natural number is negative or not.

Pseudo-code:

1. *Read a number x*
2. *If x is less than zero write a message negative*
3. *End If*
4. *else write a message not negative*

Example 5:

Original Program Specification: Write a program that computes the following sum:

Sum = 1+2+3+…. + 50

Pseudo-code:

1. Initialize sum to 0 and counter to 1

2. If the counter is less than or equal to 50

2.1. Add counter to sum

2.2. Increase counter by 1

2.3. Repeat step 2

3. Else

3.1. Display sum

3.2. Exit

Example 6:

Original program specification: write a program that takes two natural numbers from the user and computes GCD (Greatest Common Divisor) of the two numbers.

Pseudocode:

1. Start
2. Read the two numbers M and N
3. Set the larger of M and N to M; and set the smaller of M and N to N
4. *Compute the value of M mod N and set the value to R*
5. *Check if R is equal to 0*
6. *If R is 0*
    6.1.print N
    6.2.Stop
7. *If R is not 0*
    7.1.Set the value of M to N
    7.2.Set the value of N to R

Repeat step 3

4.4. Flow Chart

A flowchart (also spelled as a flow-chart or flow chart) is a schematic representation of an algorithm or a process. The advantage of the flowchart is that it doesn't depend on any particular programming language. Thus, it can be used to translate an algorithm to more than one programming language. A flowchart uses different symbols (geometrical shapes) to represent different processes. Flowcharting is a tool developed in the computer industry, for showing the steps involved in a process. A flowchart is a diagram made up of boxes, diamonds, and other shapes, connected by arrows - each shape represents a step in the process, and the arrows show the order in which they occur. Flowcharting combines symbols and flowlines, to show figuratively the operation of an algorithm.

In computing, there are dozens of different symbols used in flowcharting (there are even national and international flowcharting symbol standards). In the business process analysis, a couple of symbols are sufficient. A box with text inside indicates a step in the process, while a diamond with text represents a decision point.

If the flowchart is too messy to draw, try starting again, but leaving out all of the decision points and concentrating on the simplest possible course. Then the session can go back and add the decision points later. It may also be useful to start by drawing a high-level flowchart

for the whole organization, with each box being a complete process that has to be filled out later.

From this common understanding can come several things - process improvement ideas will often arise spontaneously during a flowcharting session. And after the session, the facilitator can also draw up a written procedure - a flowcharting session is a good way of documenting a process.

Process improvement starts with an understanding of the process, and flowcharting is the first step towards process understanding. The following Table 2 shows some of the common symbols.

Table 2: Common symbols in flowchart design

Symbol	Name	Function
	Terminal	used to represent the start or the end of a program
	Input/ output	used to represent data input or data output from a computer
	Processing	usually encloses operations or (command black) a group of operations (a process)
	Decision block	it usually contains a question within it. There are typically two output paths: one for the answer to the question is yes (true), and the other for the answer is no (false)
	Flowline	is used to indicate the direction of logical flow (a path from one operation to another
	On-page connecter	is used for connecting two points in connector a flow chart without drawing flow lines in one page.

	Off-page connector	It is used as an exit to or any entry from another part of the flowchart on another page

4.4.1. General Rules for Flowcharting

1. All boxes of the flowchart are connected with Arrows. (Not lines)
2. Flowchart symbols have an entry point on the top of the symbol with no other entry points. The exit point for all flowchart symbols is on the bottom except for the Decision symbol.
3. The Decision symbol has two exit points; these can be on the sides of the bottom and one side.
4. Generally, a flowchart will flow from top to bottom. However, an upward flow can be shown as long as it does not exceed 3 symbols.
5. Connectors are used to connect breaks in the flowchart. Examples are:
    a. From one page to another page.
    b. From the bottom of the page to the top of the same page.
    c. An upward flow of more than 3 symbols
6. Subroutines and Interrupt programs have their own and independent flowcharts.
7. All flow charts start with a Terminal or Predefined Process (for interrupt programs or subroutines) symbol.
8. All flowcharts end with a terminal or a contentious loop.

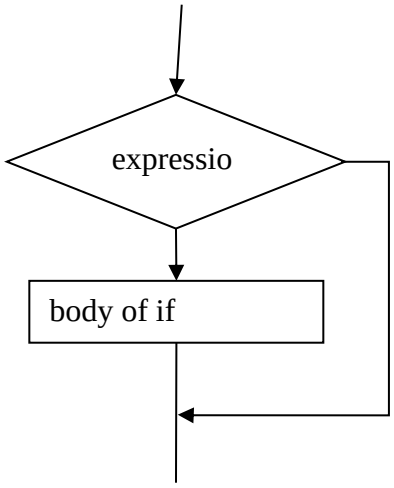
Flowcharting uses symbols that have been in use for several years to represent the type of operations and/or processes being performed. The standardized format provides a common method for people to visualize problems together in the same manner. The use of standardized symbols makes the flow charts easier to interpret; however, standardizing symbols is not as important as the sequence of activities that make up the process.

4.4.2. Flowcharting Tips

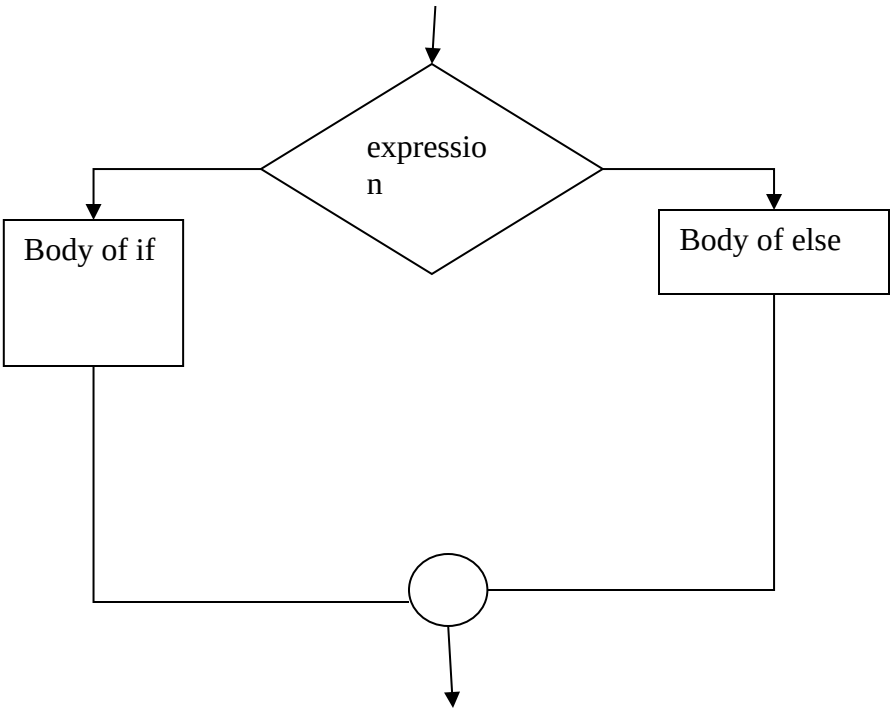
Chart the process the way it is occurring. Do not document the way a written process or a manager thinks the process happens. People typically modify existing processes to enable a more efficient process. If the desired or theoretical process is charted, problems with the existing process will not be recognized and no improvements can be made.

4.4.3. Different Basic Flowcharts

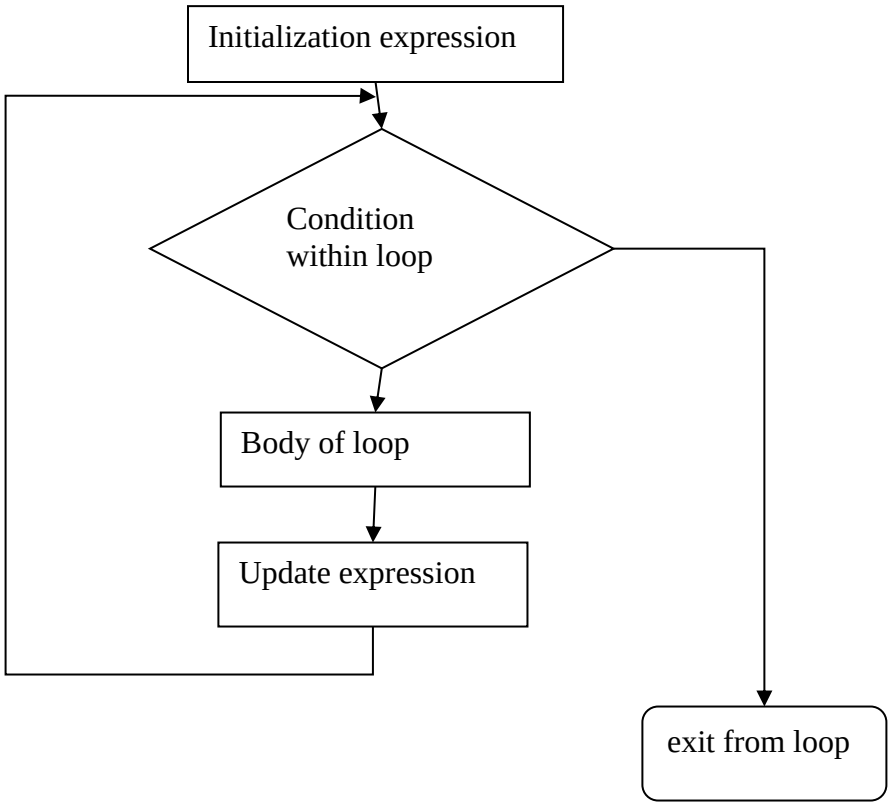
a. Flow chart for if



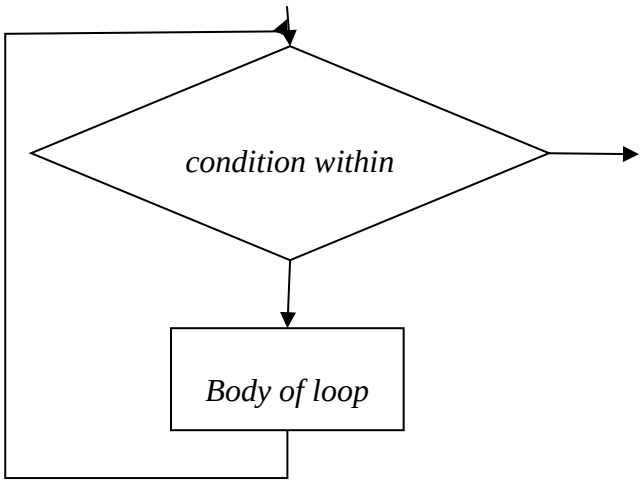
b. Flowchart for if-else



c. Flowchart for ‘for’ l*oop*

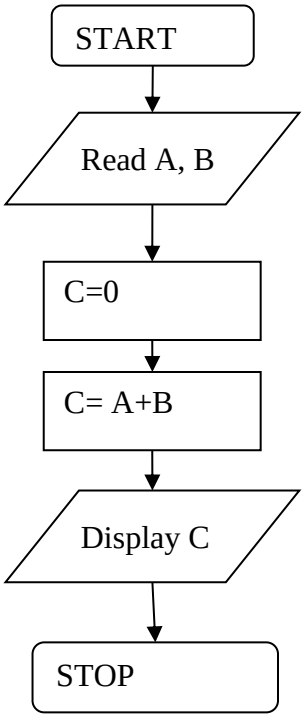


d. Flowchart for the while loop



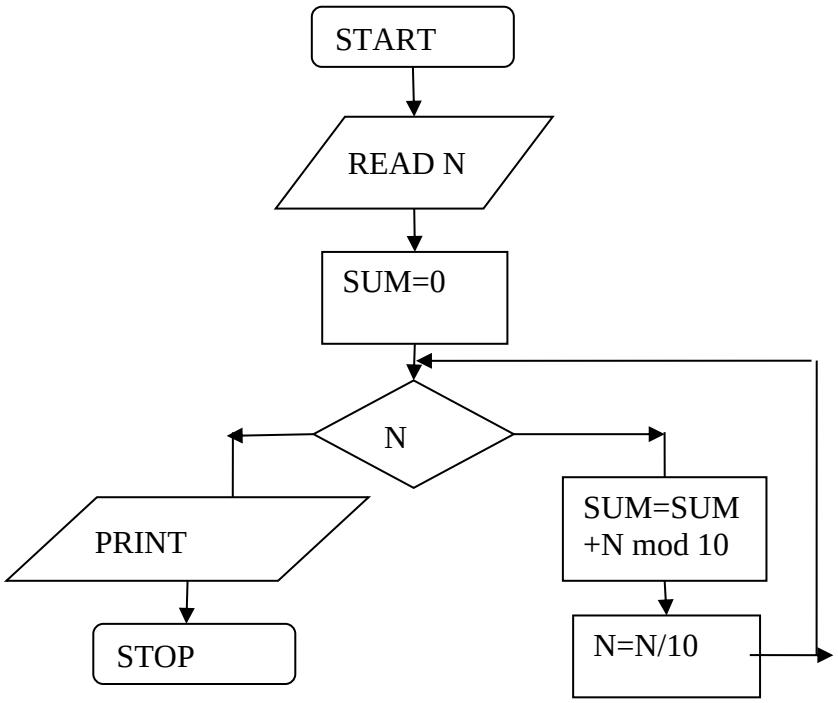
Example 1: Draw a flowchart for a program that obtains two integer numbers from the user and displays the sum of the two numbers.

Flowchart:



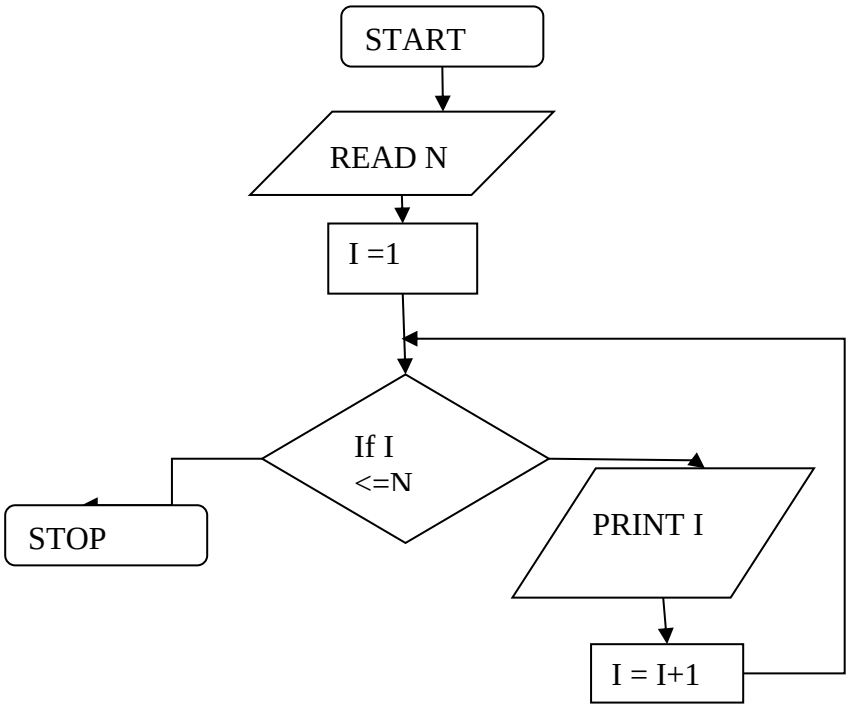
Example 2: *Draw a flowchart of a program that prompts for a natural number from the user and displays the sum of all the digits of the number.*

Flowchart:

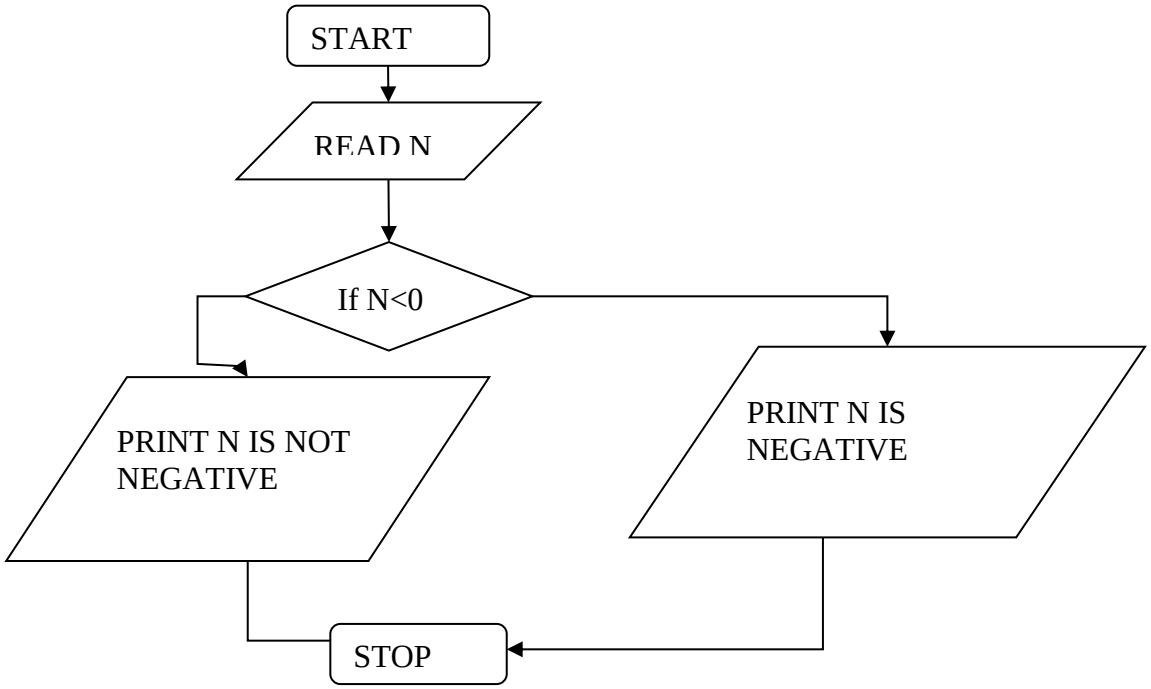


Example 3: Draw a flowchart for a program that takes a natural number say N and displays the first N natural numbers.

Flowchart:



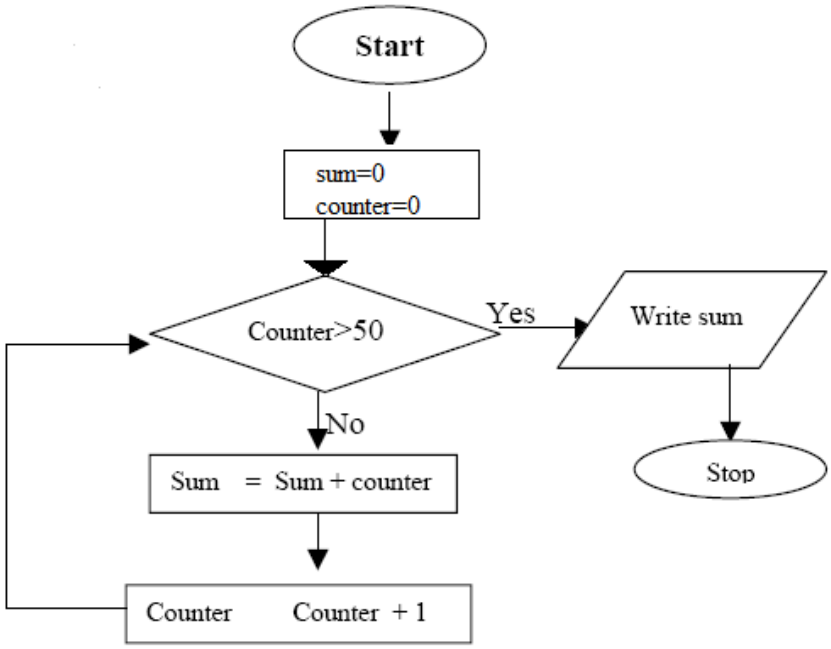
Example 4: Write an algorithm description and draw a flowchart for a program that checks a given natural number is negative or not.



Example 5: Draw a flow chart to find the following sum.

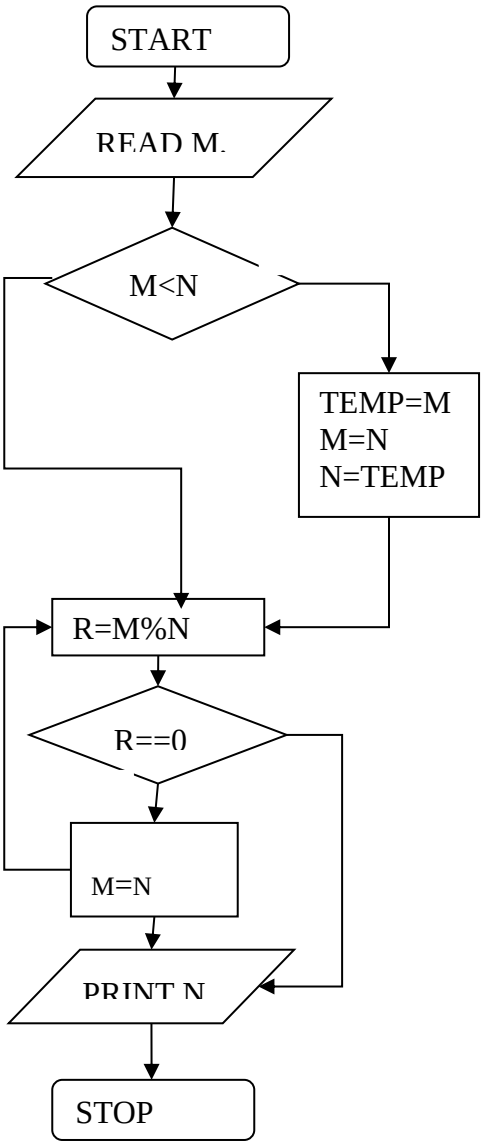
Sum = 1+2+3+…. + 50

Flowchart:



Example 6: Draw a flowchart for a program that takes two natural numbers from the user and computes GCD (Greatest Common Divisor) of the two numbers.

Flowchart:



4.5. Qualities/Features/Properties of a Good Algorithm

For a particular algorithm to be regarded as a good algorithm that solves a particular problem it must meet the following criteria:

1. ***Simplicity/Effectiveness***: The algorithm must be simple and not complex to follow in solving a particular problem. The operations performed by an algorithm should be simple so that they can be done exactly and in a finite length of time by some using pencil and paper.
2. ***Clarity/Definiteness***: The algorithm should be clear and easy to read and understand. In other words, the algorithm must be precisely defined and unambiguous. If Mr. X performs the actions in an algorithm, Mr. Y should be able to perform the same action in the same condition and obtain the same result.
3. ***Thoroughness:*** A thorough algorithm is one that considers all the possible cases and situations that may arise in that particular problem. This can be achieved by making use of variables in storing input. e.g., If you are developing an algorithm that a banker will be using in their daily transactions; deposit, and withdraw, and your algorithm did not consider alerting/sending the account owner, the balance that remains in the account at each time any of the transaction is made, then we can say that your algorithm is not thorough in whatever transaction made in the account, except if the alert is not required.
4. ***Correctness:*** The algorithm being developed must be correct. This means that the algorithm must produce the correct result/solution to the problem. However, if the result is not correct, then the algorithm should be modified. If, however, the algorithm could not produce the correct result/solution then we will say that the algorithm did not solve the problem or that the algorithm is wrong.
5. ***Efficiency:*** This is the most important of all. This means that the algorithm should be the best in the time consumed to solve the particular problem. The number of computer resources that require solving the problem should be considered when developing an algorithm. An algorithm that requires small memory space and less time to solve a particular problem are better considered/Taking. This now follows that if the total numbers of your class members are 1000 and were given the same problem to develop an algorithm that will solve the problem. The uniqueness starts with the use of variables. How? I may use x to represent 2 (i.e., x = 2), another person may use c to represent the same 2 (i.e., c = 2), still, another may like to phrase or even his name to represent the same value 2.

6. ***Finiteness:*** The actual steps executed when an algorithm is run should be of a reasonable size. This means that the execution time of the algorithm should be of reasonable length. The algorithm must be finite. If an old computer could not boot before the user dies, then the computer is useless.
7. ***Input:*** An algorithm may have some input(s), from a specified device. This input(s) may be from an external device (like a Hard disc, Flash drive) or within the computer (e.g., keyboard, RAM). The most common input in an algorithm includes Numbers – the integer's numbers, real numbers, character, etc.
8. ***Output:*** An algorithm must have an output(s). The output is always but not limited to the result of the algorithm. The output may be displayed in the visual displaying unit (VDU) or stored in the computer memory for another program/algorithm to use. The algorithm that possesses all these qualities should be chosen from millions of equivalent algorithms that solve the same problem.

Case study:

Write an algorithm that will request the length and width of a room/house, the algorithm will calculate and display the area of the room.

Premise: The question required us to develop this algorithm in such a way that it will request length & width before calculation of the area of the room will take place. It means that the algorithm/designer of the algorithm don't know the area and width of the said room abolition, in other word it can be an area of any room. So, the algorithm we are about to develop should be able to calculate the area of any room.

<u>Solution:</u>

Major Steps

1. Request Data
2. Calculate Area
3. Display Area

Stepwise Refinement

Request Data

a. Request length
b. Request width

Calculate Area

Area = length * width

Display Area

Explanation/Discussion:

The major steps list out the steps to be followed and the stepwise refinement breaks/refines the steps further into smaller steps that finally solve the problem.

Request Data: The algorithm will need the value of the length and width of the room to calculate its area.

Step 1.1: Request length, gets the value of the length, and

Step 1.2: Request width, gets the value of the width

Calculate Area: This step makes/carries out the calculation.

Step 2.1: Area = length * width:

Immediately the algorithm gets the value of the length and the width of the said room using step 1.1 and 1.2 above, the area will be assigned to the value of the length and the width so obtained from the request data steps.

Display Area: This step displays the area of the room so calculated in step 2.

Test the Algorithm:

After the development of your algorithm, the next thing is to test it. If the algorithm produces the wrong result, then you should study and adjust the algorithm. The above

algorithm is straight forward so testing it will not be a problem. However, testing some algorithms will be maybe so complex, design the algorithm chart/table/list to suit the algorithm being developed. The Table 3 shows the test for the example one algorithm.

Table 3: Test for the example one algorithm.

Variable	**Data/Information/Result**
Request length	10
Request width	3
Area = length * width	10 * 3
Display Area	30

Lab

1. Obtain two numbers from the keyboard, and determine and display which (if either) is the larger of the two numbers.
2. Design an algorithm and the corresponding flowchart for finding the sum of the numbers 2, 4, 6, 8, …, n
3. Take an integer from the user and display the factorial of that number (hint: n! = n*(n-1) *(n-2) …. *1)
4. Read an integer value from the keyboard and display a message indicating if this number is odd or even.
5. Find the minimum value in a list of a number

Review questions

1. Answer the following statements as either true or false:
2. Coding the program in a language like C++ is the first task of planning. You plan as you code.
3. Pseudo-code is the only commonly used planning tool.
4. Test data is developed for testing the program once it is code into a language like C++.
5. The word pseudo means false and includes the concepts of fake or imitation.

6. Many programmers pick up the bad habit of not completing the planning step before starting to code the program.
7. Compare and contrast between Top-down and Bottom-up approaches in computer programming?

Exercise

1. Develop an algorithm for the following problems using Pseudo-code?
   a. Obtain two numbers from the keyboard, and determine and display which (if either) is the larger of the two numbers.
   b. Take an integer from the user and display the factorial of that number (hint: n! = n*(n-1) *(n-2) …. *1)
   c. Read an integer value from the keyboard and display a message indicating if this number is odd or even.
   d. Find the minimum value in a list of a number
   e. Swap the content of two variables without using a third variable.
   f. Read an integer value from the keyboard and display a message indicating if this number is odd or even.
   g. Find the average of two numbers given by the user.
   h. Find the average, maximum, minimum, and sum of three numbers given by the user.
2. Find the area of a circle where the radius is provided by the user.
3. Find the perimeter of a rectangle where the length and width are provided by the user
4. Receive 3 numbers and display them in ascending order from smallest to largest
5. Read 10 integers from the keyboard in the range 0 - 100, and count how many of them are larger than 50, and display this result.
6. Take an integer from the user and display the factorial of that number.
7. For each of the problems below, develop an algorithm and a flow chart
8. Add and multiply the numbers from 1 to 100 and display the sum and multiplication.
9. Add the even numbers between 0 and any positive integer number given by the user.

10. Find the average of two numbers given by the user.
11. Find the average, maximum, minimum, and sum of three numbers given by the user.
12. Find the area of a circle where the radius is provided by the user.
13. Find the perimeter of a rectangle where the length and width are provided by the user
14. Receive 3 numbers and display them in ascending order from smallest to largest
15. Read 10 integers from the keyboard in the range 0 - 100, and count how many of them are larger than 50, and display this result.
16. Read the input from the user to compute the quadratic equation ($y=ax^2+ bx+ c$) and display the value of x.
17. Find sum value. Sum = $1+1/2+1/2^2+ \ldots+1/2^n$
    a. Swap the contents of two variables using a third variable.
    b. A program to generate Fibonacci series. The first two digits in the series are 0 and 1 and the rest is computed by adding the preceding two digits. (i.e., 0 1 2 3 5 8 13.)
    c. A problem to find the sum of digits of a given number. If for example the given number is 536, the sum of digits in the number = 5 + 3 + 6 =14
    d. A problem that Checks a given number is palindrome or not. A palindrome numbers

18. For each of the problems below develop top-down and bottom-up analysis

    a. Receive a number and determine whether it is odd or even.
    b. Receive 3 numbers and display them in ascending order from smallest to largest
    c. Add the even numbers between 0 and any positive integer number given by the user.
    d. Find the average of two numbers given by the user.
    e. Find the average, maximum, minimum, and sum of three numbers given by the user.

References

Horowitz, Ellis. *Fundamentals of programming languages*. Springer Science & Business Media, 2012.

Chapter Five

5. Fundamentals of C++ Programming Language

Objectives: C++ programming language has its structure in which variables are declared and various statements are written. Furthermore, Integrated Development Environments could be used for ease of C++ programming.

At the end of this chapter, students should be able to:

- *list different IDE tools for programming in C++*
- *work with Code blocks IDE for C++ programming*
- *know the basic structure of a C++ program*
- *know how to include comments in C++ programs*
- *identify the reserved keywords in C++ programming*
- *know the common data types in C++ programming*
- *know how to declare constants in C++ programming*
- *know how to declare and use a variable in C++ programming*
- *work with basic input and output in C++ programming*
- *know the various operators in C++ programming*

5.1. Mechanics of Creating a Program (C++)

C++ programs typically go through five-phase edits, preprocess, compile, linking, load:

Editor(edit): accepts the typing of the source code (or header file).

Source file: the file that contains the program you prepared in the editor after you save it. (.cpp)

Header file: header files such as iostream

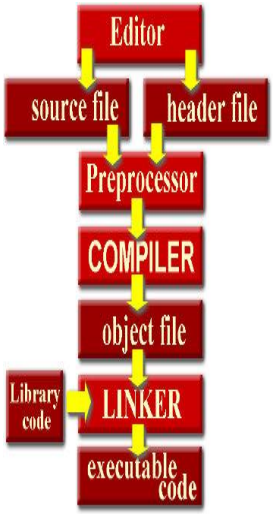


Figure 23: Phase of C++ program

Preprocessor: performs preliminary operations on files before they are passed to the compiler. The preprocessor is a program that scans the source code for preprocessor directives such as include directives. The preprocessor inserts into the source code all files included by the include directives. For example, the iostream standard library file is included by an include directive. Therefore, the preprocessor directive inserts the contents of that standard library file, including its definition of the cout object, into the source code file.

Compiler(compile): translates the source code to machine language. The compiler is another program that translates the preprocessed source code (the source code after the insertions made by the preprocessor) into corresponding machine language instructions, which are stored in a separate file, called an object file, having a .obj extension. There are different compilers for different programming languages, but the purpose of the compiler is essentially the same, the translation of a programming language into machine language, no matter which programming language is involved.

The compiler can understand your code and translate it into machine language only if your code is in the proper syntax for that programming language. C++, like other programming languages, and indeed most human languages, has rules for the spelling of words and the grammar of statements. If there is a syntax error, then the compiler cannot translate your code into machine language instructions, and instead will call your attention to the syntax errors. Thus, in a sense, the compiler acts like a spell checker and grammar checker.

Object code: the file containing the translated source code. (.obj) The archiver takes a collection of object files as input and produces a static library, or archive, which is simply a collection of object files grouped for convenient use.

Linker: links the object file with additional code, such as the library codes. While the object file has machine language instructions, the computer cannot run the object file as a program. The reason is that C++ also needs to use another code library, called the run-time

library, for common operations, such as the translation of keyboard input or the ability to interact with external hardware such as the monitor to display a message. Here, the linker takes a collection of object files and libraries and resolves their symbolic references to produce either an executable-is any program that can be executed by the operating system. or a dynamic library-also called a shared library, which is like an executable except that it can't be run on its own. On Windows, dynamic libraries are also called dynamic link libraries (DLLs).

Executable code(load): the loader takes the executable file from disk and transfers it to memory. the file containing the final product file (.exe).

5.2. Compiling Programs

Any program written in a language other than machine language needs to be translated into machine language. The set of instructions that do this task are known as translators. There are different kinds of translator software among that compiler and interpreters are of interest for most programmers.

- ***Compilers***: a compiler is a computer program that translates a series of statements written in source code (a collection of statements in a specific programming language) into a resulting object code (translated instructions of the statements in a programming language). A compiler changes or translates the whole source code into executable machine code (also called object code) which is output to a file for later execution. E.g., C++, Pascal, FORTRAN, etc.
- ***Interpreters***: is a computer program that translates a single high-level statement and executes it and then goes to the next high-level language line etc. E.g., QBASIC, Lisp, etc.

Different errors occur during writing C++ programs. The most common errors can be broadly classified as follows

1. ***Programming (syntax) errors:*** These are generated when typographical errors are made by programmers.

2. ***Compiler errors:*** These are errors detected by the compiler that makes the program un-compliable.
3. ***Linker error:*** These are errors generated when the executable of the program cannot be generated. This may be due to wrong function prototyping, incorrect header files.
4. ***Execution error:*** These errors occur at the time of execution. Looping and arithmetic errors fall under this category.
5. ***Logical errors:*** these errors solely depend on the logical thinking of the programmer and are easy to detect if we follow the line of execution and determine why the program takes that path of execution.

5.3. IDE Tools

You can use any plain-text editor such as Notepad to write the source code. You also can download a free compiler, which usually includes a preprocessor and linker. You then can compile and run your code from the command line. The command line may be, for example, a DOS prompt at which you type a command that specifies the action you want, such as compiling, followed by the name of the file you want to compile.

While there is nothing wrong with using a plain-text editor and command-line tools, many programmers prefer to create, compile, and run their programs in a C++ Integrated Development Environment, known by the acronym IDE. The term “integrated” in IDE means that the text editor, preprocessor, compiler, and linker are all together under one (software) roof. Thus, the IDE enables you to create, compile, and run your code using one program rather than separate programs. Additionally, most IDEs have a graphical user interface (GUI) that makes them easier for many to use than a command line. Finally, many IDEs have added features that ease your task of finding and fixing errors in your code.

There are several good IDEs on the market. Microsoft’s, called Visual C++, can be obtained separately or as part of Microsoft’s Visual Studio product. Borland offers C++ Builder, both in a free and commercial version. IBM has a VisualAge C++ IDE. There are several others as well such as Code Blocks, Code Warrior, Dev C++… etc.

In this module, we will use Code Blocks 13.12 IDE since we happen to have it. However, most IDEs work essentially the same way, and your code will compile and run the same no matter which IDE you use as long as you don't use any library files custom to a particular IDE. The standard library files we will be using, such as iostream, are the same in all C++ IDEs.

Additionally, we are running the code on a macOS operating system. The results should be similar on other operating systems, not just Windows operating systems, but additional types of operating systems as well, such as UNIX.

After you installed Code Blocks 13.12 IDE software on macOS operating system, you will get the interface as shown in Figure 24

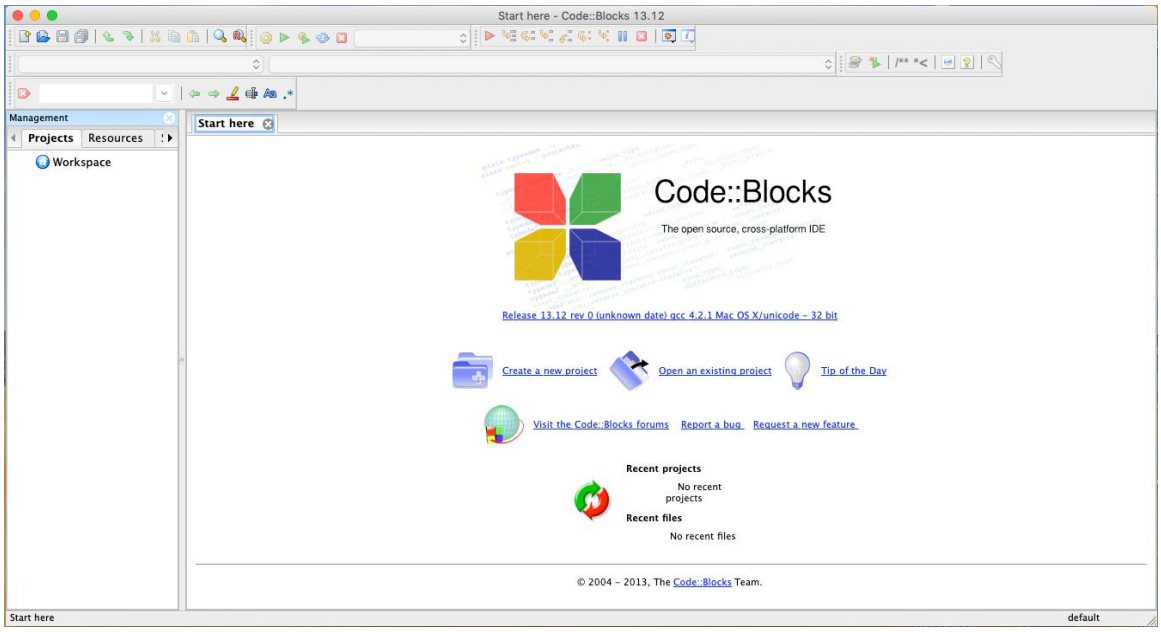


Figure 24: Code Blocks interface

Let's now use the IDE to write the source code for the "Hello World!" project, and then compile and run it.

5.3.1. Setting Up the "Hello World!" Project

Once you have installed Code Blocks 13.12 IDE successfully, you are now ready to start your first project, which is to create and run the "Hello World!" application.

1) Start Code Blocks
2) Open the New Project dialog box shown in Figure 25 using the File | New | Project menu command.

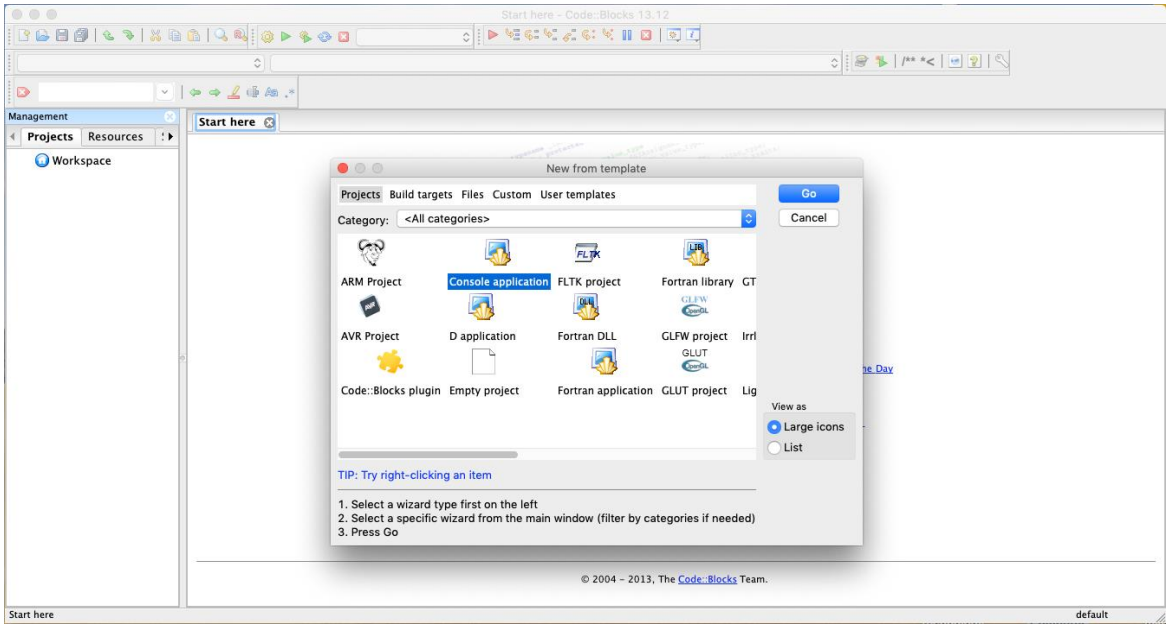


Figure 25: Creating a New Project

3) From the popup category projects, choose console application as shown in Figure 25 then click on the "Go" button.
4) After you click on the Go button, the console application dialog box popup as shown in Figure 26.

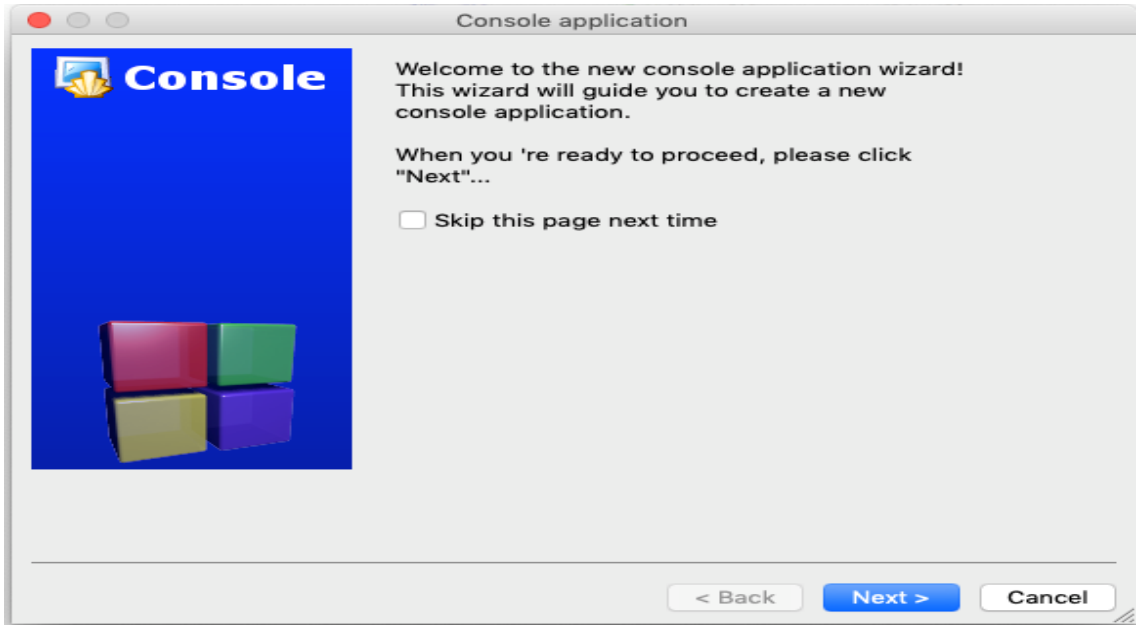


Figure 26: Console application interface (welcome screen)

5) Then click on the Next button and select C++ language from C and C++ listed as shown in Figure 27. The reason you choose the C++ language is that only we are focusing on C++ programming in this Module.

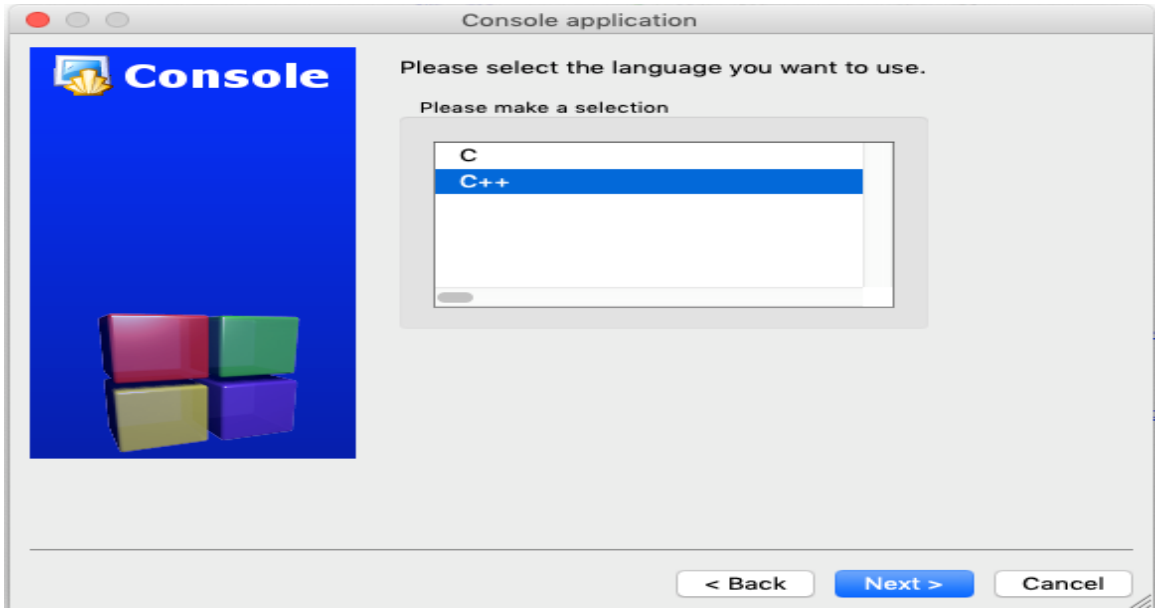


Figure 27: Console application interface (selected language)

6) Click on the Next button and in the Project Title field, type the name you've chosen for your project as shown in Figure 28. This will also be the name of the subfolder created to store your project files. We suggest you use a name that describes your project so you can locate it more easily later.

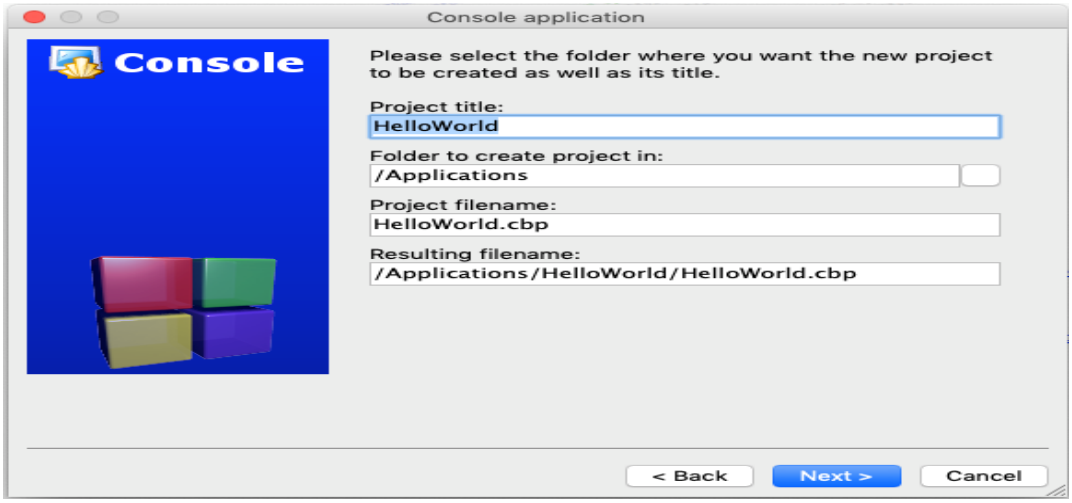


Figure 28: Console application interface project title selection

7) Click the Next >Finish button. This will display the application folder HelloWorld and its parent folder shown in Figure 29.

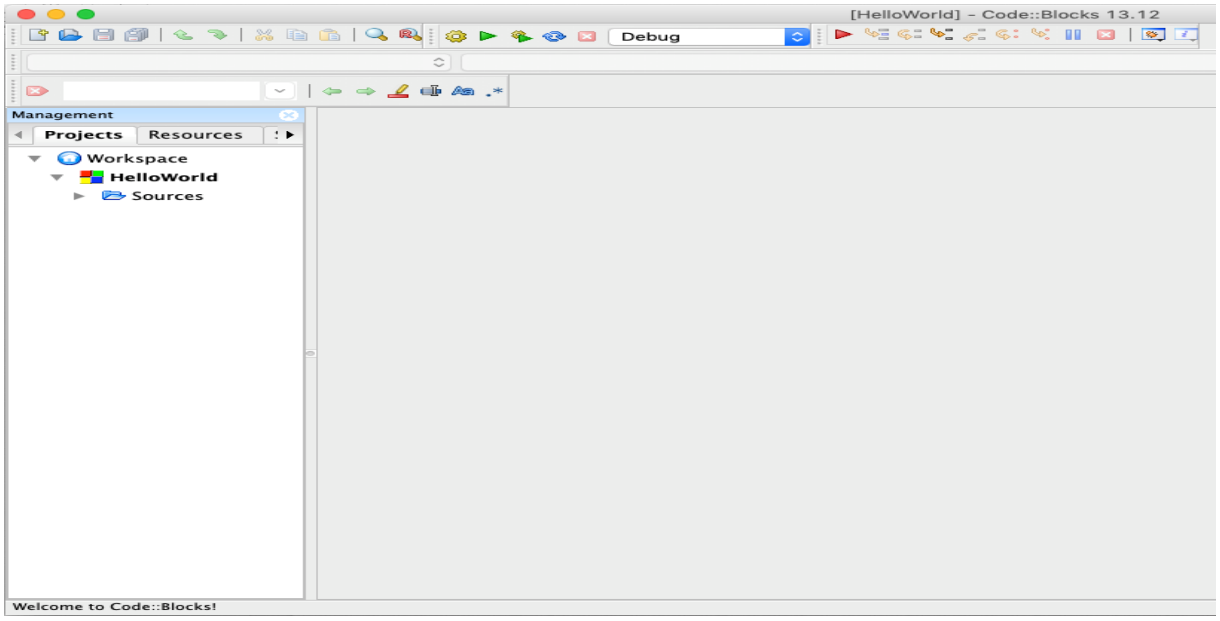


Figure 29: Starting the HelloWorld Application

The file in which the code for the "Hello World!" application will be written is a source file. Source files have a .cpp extension, CPP standing for C++. By contrast, the iostream file that is included by the include directive is a header file. Header files have a .h extension—the h standing for the header.

5.3.2. To add a new source file to the project

1. You can use the following steps to add a new source file to the project: Click on File menu>New menu for Code Blocks. Then choose the "File" menu Item.
2. Choose *C++ empty file* from template category as shown in as shown

Figure 30: C++ new source file creation

3. Click on the Go button and it prompts you to type the file name and location where you want to put it.
4. Generally, you can change the Location field, which is the folder in which the project files are stored. Type the name of the new source file in the FileName field. You need to type the .cpp extension. By typing hello.cpp, as shown in Figure 31, the new file will be called hello.cpp.

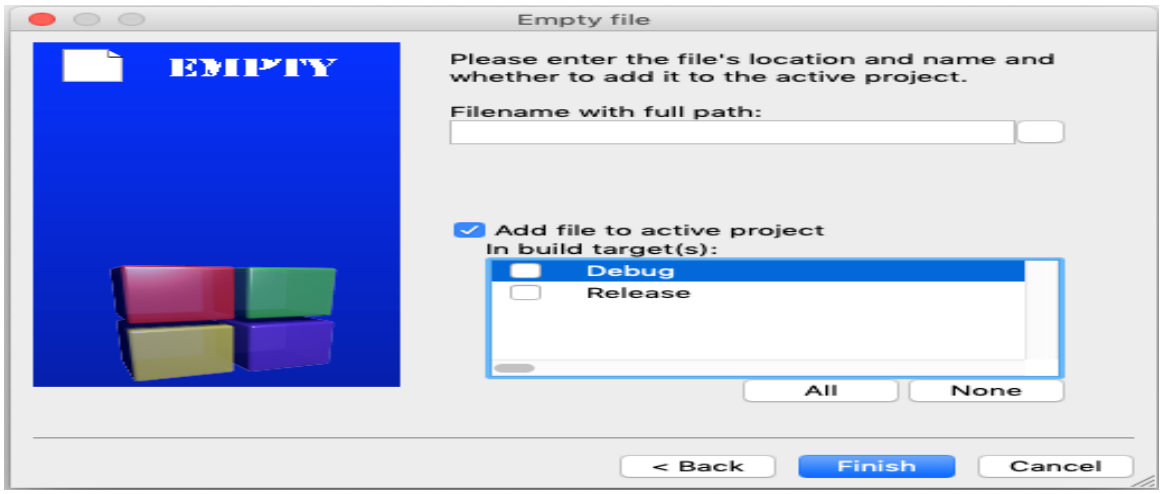


Figure 31: Adding a New Source File to a project

5. When you are done, click the Open button. Figure 32 shows the new hello.cpp file projects.

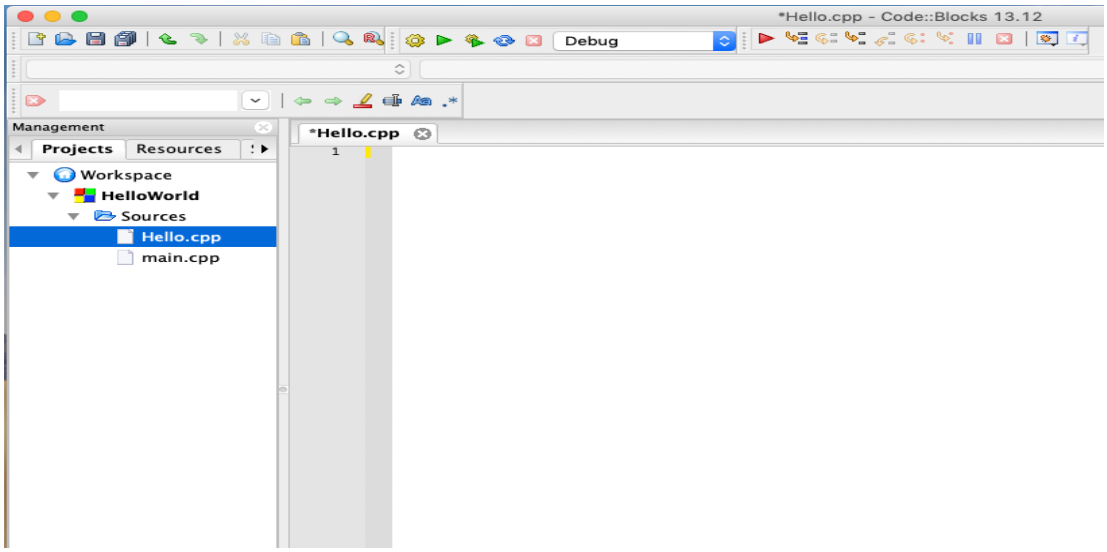


Figure 32: The source file before typing code

5.3.3. Running the Code

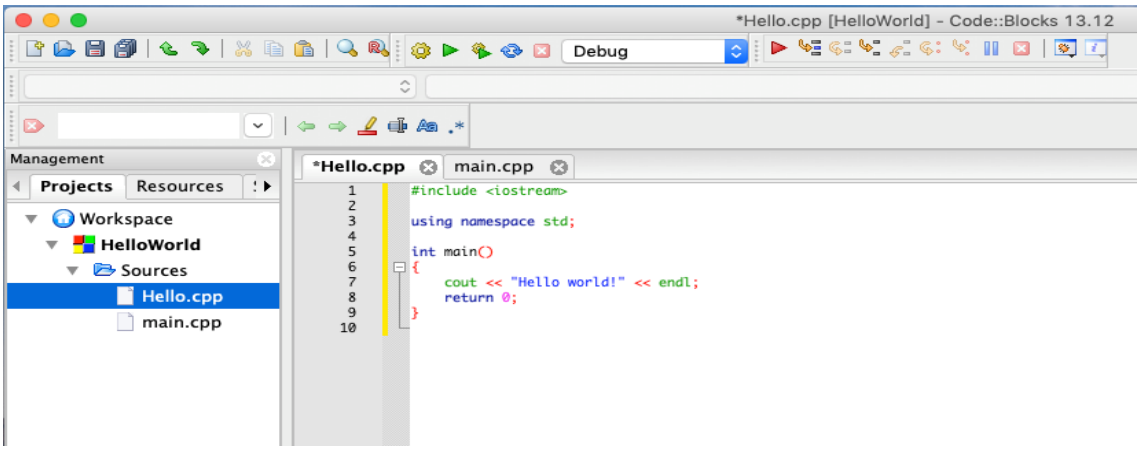


Figure 33: The source file after typing code

The final step is to run the code. You do so from the Build menu. You may choose either Build then Run | Build and Run. The result is the console window displaying "Hello World!" (in Figure 34).

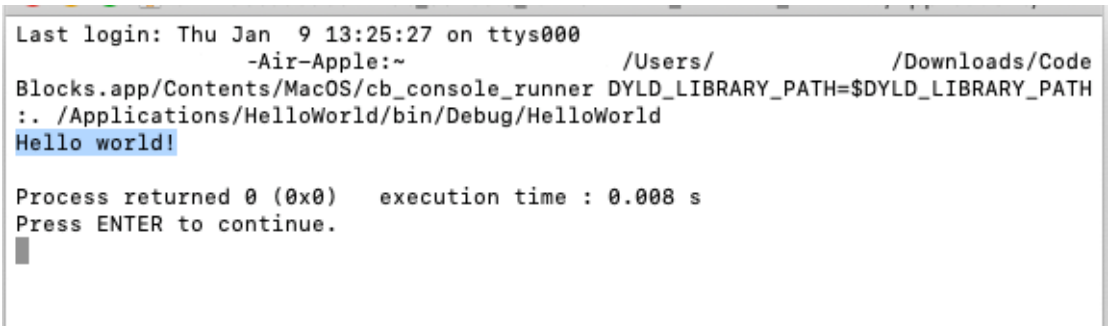


Figure 34: C++ program outputting "Hello World!" to the screen

5.4. The Structure of C++ Programs

Probably the best way to start learning a programming language is with a program. So, here is our first program:

```
// my first program in C++
#include <iostream>
using namespace std;
int main ()// main () is where program execution begins.
```

```
{
   cout << "Hello World!";
   return 0;
}
```

Output

```
Hello World!
```

The above box shows the source code for our first program, which we can name, for example, world.cpp, and showing the result of the program once compiled and executed. The way to edit and compile a program depends on the compiler you are using. The previous program is the first program that most programming apprentices write, and its result is the printing on the screen of the "Hello World!" sentence. It is one of the simpler programs that can be written in C++, but it already includes the basic components that every C++ program has. We are going to take a look at them one by one:

```
// my first program in C++
```

This is a comment line. All the lines beginning with two slash signs (//) are considered comments and do not have any effect on the behavior of the program. They can be used by the programmer to include short explanations or observations within the source itself. In this case, the line is a brief description of what our program does.

```
#include <iostream>
```

Sentences that begin with a pound sign (#) are directives for the preprocessor. They are not executable code lines but indications for the compiler. In this case, the sentence #include <iostream> tells the compiler's preprocessor to include the iostream standard header file. This specific file includes the declarations of the basic standard input-output library in C ++, and it is included because its functionality is used later in the program.

```
Namespace
using namespace std;
```

C++ uses namespaces to organize different names used in programs. Every name used in the iostream standard library file is part of a namespace called std. Consequently, the *cout*

object is called *std::cout*. The *using namespace std;* statement avoids the need for putting *std::* before every reference to *cout*, so we can just use *cout* in our code.

```
int main ()
```

This line corresponds to the beginning of the main function declaration. The main function is the point by where all C++ programs begin their execution. It is independent of whether it is at the beginning, at the end, or in the middle of the code - its content is always the first to be executed when a program starts. Also, for that same reason, all C++ programs must have a main function.

'main' is followed by a pair of parentheses () because it is a function. In C++ all functions are followed by a pair of parentheses () that, optionally, can include arguments within them. The content of the main function immediately follows its formal declaration and it is enclosed between curly brackets ({}), as in our example.

```
cout << "Hello World";
```

This instruction does the most important thing in this program. cout is the standard output stream in C++ (usually the screen), and the full sentence inserts a sequence of characters (in this case "Hello World") into this output stream (the screen). cout is declared in the iostream header file, so to be able to use it, that file must be included.

Notice that the sentence ends with a semicolon character (;). This character signifies the end of the instruction and must be included after every instruction in any C++ program (one of the most common errors of C++ programmers is indeed to forget to include a semicolon; at the end of each instruction).

```
return 0;
```

The return instruction causes the main () function to finishing and returns the code that the instruction is followed by, in this case, 0. This is the most usual way to terminate a program that has not found any errors during its execution. As you will see in the coming examples, all C++ programs end with a sentence similar to this.

Therefore, you may have noticed that not all the lines of this program did an action. Lines were containing only comments (those beginning by //), lines with instructions for the

compiler's preprocessor (those beginning by #), then some lines initiated the declaration of a function (in this case, the main function) and, finally lines with instructions (like the call to cout <<), these last ones were all included within the block delimited by the curly brackets ({}) of the main function.

The program has been structured in different lines to be more readable, but it is not compulsory to do so. For example, instead of

```
int main ()
{
cout << " Hello World ";
return 0;
}
```

we could have written:

```
int main () {cout << " Hello World "; return 0;}
```

In just one line and this would have had the same meaning.

5.5. Comments

Comments are pieces of source code discarded from the code by the compiler. They do nothing.

Their purpose is only to allow the programmer to insert notes or descriptions embedded within the source code.

C++ supports two ways to insert comments:

```
// line comment
/* block comment */
```

The first of them, the line comment, discards everything from where the pair of slash signs (//) is found up to the end of that same line. The second one, the block comment, discards everything between the /* characters and the next appearance of the */ characters, with the possibility of including several lines.

We are going to add comments to our second program:

```
/* my second program in C++
with more comments */
#include <iostream>
using namespace std;
int main ()
{
cout << "Hello World! "; // says Hello World!
cout << "I'm a C++ program"; // says I'm a C++ program
return 0;
}
```

Output

```
Hello World! I'm a C++ program
```

Semicolons & Blocks in C++:

- In C++, the semicolon is a statement terminator. That is, each statement must be ended with a semicolon. It indicates the end of one logical entity. For example, the following are three different statements:

```
x = y;
y = y+1;
y+=1;
```

- A block is a set of logically connected statements that are surrounded by opening and closing braces. For example:

```
{
cout << "Hello World"; // prints Hello World
return 0;
}
```

C++ does not recognize the end of the line as a terminator. For this reason, it does not matter where on a line you put a statement. For example:

```
x = y;
y = y+1;
```

is the same as

```
x = y; y = y+1;
```

5.6. C++ Basic Input/Output

The C++ standard libraries provide an extensive set of input/output capabilities which we will see in subsequent chapters. This chapter will discuss the very basic and most common I/O operations required for C++ programming. C++ I/O occurs in streams, which are sequences of bytes. If bytes flow from a device likes a keyboard, a disk drive, or a network connection, etc. to main memory, this is called input operation and if bytes flow from main memory to a device like a display screen, a printer, a disk drive, or a network connection, etc., this is called output operation.

5.6.1. I/O Library Header Files

There are the following header files important to C++ programs:

Table 4: I/O Library Header Files

Header File	Function and Description
<iostream>	This file defines the cin, cout, cerr, and clog objects, which correspond to the standard input stream, the standard output stream, the un-buffered standard error stream, and the buffered standard error stream, respectively.
<iomanip>	This file declares services useful for performing formatted I/O with so-called parameterized stream manipulators, such as setw and setprecision.
<fstream>	This file declares services for user-controlled file processing.

5.6.2. The Standard Output Stream (cout)

The predefined object *cout* is an instance of an *ostream* class. The *cout* object is said to be "connected to" the standard output device, which usually is the display screen. The *cout* is used in conjunction with the stream insertion operator, which is written as << which are two less than signs as shown in the following example.

```
#include <iostream>
using namespace std;
int main ()
{
int year=2013;
cout << "Value of year is: " << year << endl;
}
```

When the above code is compiled and executed, it produces the following result:

```
Value of year is:2013
```

The C++ compiler also determines the data type of variable to be output and selects the appropriate stream insertion operator to display the value. The << operator is overloaded to output data items of built-in types integer, float, double, strings, and pointer values.

The insertion operator << may be used more than once in a single statement as shown above and endl is used to add a new-line at the end of the line.

5.6.3. The Standard Input Stream (cin)

The predefined object *cin* is an instance of *istream* class. The *cin* object is said to be attached to the standard input device, which usually is the keyboard. The *cin* is used in conjunction with the stream extraction operator, which is written as >> which are two greater-than signs as shown in the following example.

```
#include <iostream>
using namespace std;
int main () {
int phone;
```

```
cout << "Please enter your phone number: ";
cin >> phone;
cout << "Your phone is: " << phone << endl;
}
```

When the above code is compiled and executed, it will prompt you to enter a name. You enter a value and then hit enter to see the result something as follows:

```
Please enter your phone: 910203040
Your phone is: 910203040
```

The C++ compiler also determines the data type of the entered value and selects the appropriate stream extraction operator to extract the value and store it in the given variables. The stream extraction operator >> may be used more than once in a single statement. To request more than one datum you can use the following:

```
cin>> name >> age;
```

This will be equivalent to the following two statements:

```
cin>>name;
cin>>age;
```

5.7. Constants, Keywords, Variables, Data Types and Operators

5.7.1. Keywords (reserved words)

Reserved/Keywords have a unique meaning within a C++ program. These symbols, the reserved words, must not be used for any other purposes. All reserved words are in lower-case letters. The following are some of the reserved words of C++.

Table 5: Reserved words in C++

asm	auto	bool	break	case	catch
const_cast	class	const	char	continue	default
dynamic_cast	do	double	delete	else	enum
explicit	extern	false	float	for	friend
goto	if	inline	int	long	mutable

namespace	new	operator	private	protected	public
reinterpret_cast	register	return	short	signed	sizeof
static_cast	static	struct	switch	template	this
throw	true	try	typedef	typeid	typename
union	unsigned	using	virtual	void	volatile
wchar_t					

5.7.2. Variable

The usefulness of the "Hello World" programs shown in the previous section are something more than questionable. We had to write several lines of code, compile them, and then execute the resulting program just to obtain a sentence on the screen as result. It would indeed have been much faster to simply write the output sentence by ourselves, but programming is not limited only to printing texts on screen. To go a little further on and to become able to write programs that perform useful tasks that save us work we need to introduce the concept of the variable. All this process that you have made is a simile of what a computer can do with two variables. This same process can be expressed in C++ with the following instruction set:

```
a = 5;
b = 2;
a = a + 1;
result = a - b;
```

This is a very simple example since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them. Therefore, we can define a variable as a portion of memory to store a determined value. A variable is a symbolic name for a memory location in which data can be stored and subsequently recalled. Variables are used for holding data values so that they can be utilized in various computations in a program. Each variable needs an identifier that distinguishes it from the others, for example, in the previous code the variable identifiers were a, b and result, but we could have called the variables any names we wanted to invent, as long as they were valid identifiers.

5.7.3. Identifiers

An identifier is the name associated with a function or data object and used to refer to that function or data object. An identifier must:

- Start with a letter or underscore
- Consist only of letters, the digits 0-9, or the underscore symbol _
- Not be a reserved word

Syntax of an identifier

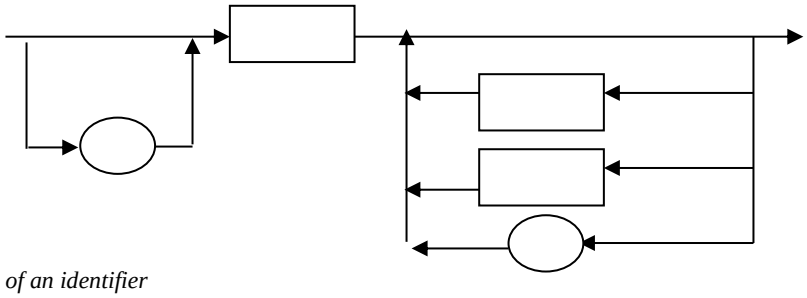


Figure 35. Syntax of an identifier

In C++ identifiers, the underscore symbol, _, is considered to be a letter. Its use as the first character in an identifier is not recommended though, because many library functions in C++ use such identifiers. Similarly, the use of two consecutive underscore symbols, _ _, is forbidden.

The following are valid identifiers

Length	days_in_year	DataSet1	Profit95
Int	_Pressure	first_one	first_1

The following are invalid:

days-in-year	1data	int	first.val
throw	My+best	no##	bestWish!

Although it may be easier to type a program consisting of single-character identifiers, modifying or correcting the program becomes more and more difficult. The minor typing effort of using meaningful identifiers will repay itself many folds in the avoidance of simple programming errors when the program is modified.

Very important: The C++ language is "case sensitive", which means that an identifier written in capital letters is not equivalent to another one with the same name but written in small letters. Thus, for example, the variable *RESULT* is not the same as the variable result nor the variable Result.

5.7.4. Data Type

When programming, we store the variables in our computer's memory, but the computer must know what we want to store in them since storing a simple number, a letter or a large number is not going to occupy the same space in memory.

Our computer's memory is organized in bytes. A byte is the minimum amount of memory that we can manage. A byte can store a relatively small amount of data, usually an integer between 0 and 255 or one single character. But also, the computer can manipulate more complex data types that come from grouping several bytes, such as long numbers or numbers with decimals. Next, you have a list of the existing fundamental data types in C++, as well as the range of values that can be represented with each one of them: there are two types of data types, built-in data type, and user define the data type.

Built-in data type: data types that come within the IDE

Table 6: Data Types, Sizes, and Ranges

Name	Bytes*	Description	Range*
char	1	character or integer 8bits length.	**signed:** -128 to 127 unsigned: 0 to 255
short	2	integer 16 bits length.	**signed:** -32768 to 32767 **unsigned:** 0 to 65535
long	4	integer 32 bits length.	**signed:** -2147483648 to 2147483647 **unsigned:** 0 to 4294967295
int	*		See short, long
float	4	floating-point number.	3.4e + / - 38 (7 digits)
double	8	double precision floating-point number.	1.7e + / - 308 (15 digits)
long double	10	a long double-precision floating-point number.	1.2e + / - 4932 (19 digits)
bool	1	Boolean value. It can take one of two values:	true or false
wchar_t	2	Wide character. It is designed as a type ton store international characters of a two-byte character set	

NOTE:

Integer. Its length traditionally depends on the length of the system's Word **type**, thus in MSDOS, it is 16 bits long, whereas in 32-bit systems (like Windows 9x/2000/NT and systems that work under protected mode in x86 systems) it is 32 bits long (4 bytes).

The bool data type is a type recently added by the ANSI-C++ standard. Not all compilers support it. Consult section bool type for compatibility information. **true** or **false**

Unsigned vs. Signed Data Type: Table 6 lists three data types: short, int, and long. Each of these three data types has either the word unsigned in front of it or nothing at all—as in unsigned short and short.

Unsigned means the number is always zero or positive, never negative. ***Signed*** means the number may be negative or positive (or zero). If you don't specify signed or unsigned, the data type is presumed to be signed. Thus, signed short and short are the same.

Since an unsigned data type means its value is always 0 or positive, never negative, in Table 6 the smallest value of an unsigned short is, therefore, zero; an unsigned short cannot be negative. By contrast, the smallest value of a short is –32767, since a signed data type may be negative, positive, or zero.

User-defined data type: data types that the programmer defines itself

5.7.5. typedef Declarations

You can create a new name for an existing type using typedef. Following is the simple syntax to define a new data type using typedef:

```
typedef type newName;
```

For example, the following tells the compiler that feet are another name for int:

```
typedef int feet;
```

Now, the following declaration is perfectly legal and creates an integer variable called distance:

```
feet distance;
```

5.7.6. Variable Declaration

To use a variable in C++, we must first declare it specifying which of the data types above we want it to be. The syntax to declare a new variable is to write the data type specifier that we want (like int, short, float...) followed by a valid variable identifier.

Syntax rule for declaring a variable is:

```
Datatype nameIdentifier;
```

For example:

```
int a;
float myNumber;
```

IMPORTANT- Variables must be declared before used!

Creating More Than One Variable at a Time

You can create more than one variable of the same type in one statement by writing the type and then the variable names, separated by commas. For example:

```
int myAge, myWeight; // two int variables
long area, width, length; // three longs
```

As you can see, myAge and myWeight are each declared as integer variables. The second line declares three individual long variables named area, width, and length. However, keep in mind that you cannot mix types in one definition statement.

Assigning Values to Your Variables

You assign a value to a variable by using the assignment operator (=). Thus, you would assign 5 to Width by writing

```
int Width;
Width = 5;
```

You can combine these steps and initialize Width when you define it by writing

```
int Width = 5;
```

Initialization looks very much like an assignment, and with integer variables, the difference is minor. The essential difference is that initialization takes place at the moment you create the variable.

Just as you can define more than one variable at a time, you can initialize more than one variable at creation. For example:

// create two int variables and initialize them

```
int width = 5, length = 7;
```

This example initializes the integer variable width to the value 5 and the length variable to the value 7. It is possible to even mix definitions and initializations:

```
int myAge = 39, yourAge, hisAge = 40;
```

This example creates three types of int variables, and it initializes the first and third.

Example…

```
1.
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6. unsigned short int Width = 5, Length;
7. Length = 10;
8. // create an unsigned short and initialize with result
9. // of multiplying Width by Length
10.
11.     unsigned short int Area = Width * Length;
12.
13.     cout << "Width:" << Width << "\n";
14.     cout << "Length: "<< Length << endl;
15.     cout << "Area: " << Area << endl;
16.
17.     return 0;
18.     }
```

Output:

```
Width:5
Length: 10
Area: 50
```

Line 2 includes the required include statement for the iostream library so, that cout will work. Line 4 begins the program.

On line 6, Width is defined as an unsigned short integer, and its value is initialized to 5. Another unsigned short integer, Length, is also defined, but it is not initialized. On line 7, the value 10 is assigned to Length.

On line 11, an unsigned short integer, Area, is defined, and it is initialized with the value obtained by multiplying Width times Length. On lines 13-15, the values of the variables are printed to the screen. Note that the special word endl creates a new line.

5.7.7. C++ CONSTANTS/LITERALS

Constants refer to fixed values that the program may not alter and they are called literals.

Constants can be of any of the basic data types and can be divided into Integer Numerals, Floating-Point Numerals, Characters, String s, and Boolean Values. Again, constants are treated just like regular variables except that their values cannot be modified after their definition.

5.7.7.1. Integer Literals

An integer literal can be a decimal, octal, or hexadecimal constant. A prefix specifies the base or radix: 0x or 0X for hexadecimal, 0 for octal, and nothing for decimal. An integer literal can also have a suffix that is a combination of U and L, for unsigned and long, respectively. The suffix can be uppercase or lowercase and can be in any order.

Here are some examples of integer literals:

```
212 // Legal
215u // Legal
```

```
0xFeeL // Legal
078 // Illegal: 8 is not an octal digit
032UU // Illegal: cannot repeat a suffix
```

Following are other examples of various types of Integer literals:

```
85 // decimal
0213 // octal
0x4b // hexadecimal
30 // int
30u // unsigned int
30l // long
30ul // unsigned long
```

5.7.7.2. Floating-point Literals

A floating-point literal has an integer part, a decimal point, a fractional part, and an exponent part. You can represent floating point literals either in decimal form or exponential form.

While representing using the decimal form, you must include the decimal point, the exponent, or both and while representing using an exponential form; you must include the integer part, the fractional part, or both. The signed exponent is introduced by e or E.

Here are some examples of floating-point literals:

```
3.14159 // Legal
314159E-5L // Legal
510E // Illegal: incomplete exponent
210f // Illegal: no decimal or exponent
.e55 // Illegal: missing integer or fraction
```

5.7.7.3. Boolean Literals

There are two Boolean literals and they are part of standard C++ keywords: A value of true representing true. The value of false represents false. You should not consider the value of true equal to 1 and the value of false equal to 0.

5.7.7.4. Character Literals

Character literals are enclosed in single quotes. If the literal begins with L (uppercase only), it is a wide-character literal (e.g., L'x') and should be stored in wchar_type of a variable. Otherwise, it is a narrow character literal (e.g., 'x') and can be stored in a simple variable of char type. A character literal can be a plain character (e.g., 'x'), an escape sequence (e.g., '\t'), or a universal character (e.g., '\u02C0'). There are certain characters in C++ when they are preceded by a backslash, they will have special meaning and they are used to represent like newline (\n) or tab (\t). Here, you have a list of some of such escape sequence codes:

Table 7: Escape sequence codes for C++

Esc ape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\b	Backspace
\n	Newline
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three dig its
\xhh . . .	Hexadecimal number of one or more digits

Following is the example to show few escape sequences characters:

```
#include <iostream>
using namespace std;
int main()
{
cout << "Hello\tWorld\n\n";
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Hello     World
```

5.7.7.5. String Literals

String literals are enclosed in double-quotes. A string contains characters that are similar to character literals: plain characters, escape sequences, and universal characters. You can break a long line into multiple lines using string literals and separate them using whitespaces.

Here are some examples of string literals. All three forms are identical string s.

```
"Hello, dear"
"Hello, \
dear"
"Hello, " "d" "ear"
```

5.7.7.6. Defining Constants

There are two simple ways in C++ to define constants:

- Using #define preprocessor.
- Using the const keyword.

5.7.7.6.1. The #define Preprocessor

Following is the form to use #define preprocessor to define a constant:

```
#define identifier value
```

Following example explains it in detail:

```
#include <iostream>
#define LENGTH 10
#define WIDTH 5
#define NEWLINE '\n'
using namespace std;
int main()
{
int area;
area = LENGTH * WIDTH;
cout << area;
```

```
cout << NEWLINE;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
50
```

5.7.7.6.2. The const Keyword

You can use const prefix to declare constants with a specific type as follows:

```
const type variable = value;
```

Following example explains it in detail:

```
#include <iostream>
using namespace std;
int main(){
const int LENGTH = 10;
const int WIDTH = 5;
const char NEWLINE = '\n';
int area;area = LENGTH * WIDTH;
cout << area;
cout << NEWLINE;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
50
```

Note that it is a good programming practice to define constants in CAPITALS.

5.7.8. C++ Operator

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provides the following types of operators:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Misc Operators

This chapter will examine the arithmetic, relational, logical, bitwise, assignment, and other operators one by one.

5.7.8.1. Arithmetic Operators

There are following arithmetic operators supported by C++ language: Assume variable A holds 10 and variable B holds 20, then:

Table 8: The common arithmetic operators supported by C++ language

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus Operator and the remainder of after an integer division	B % A will give 0
++	Increment operator increases integer value by one	A++ will give 11
--	Decrement operator decreases integer value by one	A-- will give 9

Add some examples for individual operators

5.7.8.2. Relational Operators

There are following relational operators supported by C++ language Assume variable A holds 10 and variable B holds 20, then:

Table 9: The common relational operators supported by C++ language

Operator	Description	Example
==	if the values of two operands are equal or not, if yes then the condition becomes true.	(A==B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then the condition becomes true.	(A!=B) is true.
>	Checks if the value of the left operand is greater than the value of the right operand, if yes then the condition becomes true.	(A>B) is not true.
<	Checks if the value of the left operand is less than the value of the right operand, if yes then the condition becomes true.	(A<B) is true.
>=	Checks if the value of the left operand is greater than or equal to the value of the right operand, if yes then the condition becomes true.	(A>=B) is not true
<=	Checks if the value of the left operand is less than or equal to the value of the right operand, if yes then the condition becomes true.	(A<=B) is true.

5.7.8.3. Logical Operators

There are following logical operators supported by C++ language Assume variable A holds 1 and variable B holds 0, then:

Table 10: The common logical operators supported by C++ language.

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make false.	!(A && B) is true.

Example: What you are expecting from the given sample code?

```
#include <iostream>
using namespace std ;
int main()
{
int a = 1 , b = 0 ;
```

```
cout << "AND logic:" << endl ;
cout << "(a && a) " << ( a && a ) << endl;
cout << "(a && b) " << ( a && b ) << endl ;
cout << "(b && b) " << ( b && b ) << << endl;

cout << endl << "OR logic:" << endl ;
cout << "(a || a) " << ( a || a ) << endl ;
cout << "(a || b) " << ( a || b ) << endl ;
cout << "(b || b) " << ( b || b ) << endl ;
cout << endl << "NOT logic:" << endl ;

cout<<"a="<<a<<" !a="<<!a<<"  ";
cout<<"b="<<b<<" !b="<<!b<<endl;
return 0 ;
}
```

5.7.8.4. Bitwise Operators

Bitwise operation means to convert the number into binary and carry out the operation on each bit individually. For example, let us take the operation complement represented by the symbol (~). Let us take a number

short A = 42;

A short number is stored in two-byte or 16 bits. Therefore, A when expressed in binary = 00000000 00101010 Complement of A is ~A = 11111111 11010101

Bitwise operators work on bits and perform the bit-by-bit operations. The truth tables for &, |, and ^ are as follows:

Table 11: The common bitwise operators supported by C++ language

p	q	p & q	p | q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; now in the binary format, they will be as follows:

```
A = 0011 1100
B = 0000 1101
-----------------
A&B = 0000 1100
A|B = 0011 1101
A^B = 0011 0001
~A = 1100 0011
```

The Bitwise operators supported by the C++ language are listed in the following Table 12. Assume variable A holds 60 and variable B holds 13, then:

Table 12: The common assignment operators supported by C++ language

Operator	Description	Example of code
&	Binary AND Operator copies a bit to the result if it exists in both operands.	A&B ;
|	Binary OR Operator copies a bit if it exists in either operand.	A|B;
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	A^B ;
~	Binary One's Complement Operator is unary and has the effect of 'flipping ' bits.	~A;
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A<<2;
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >>2; //shift right by 2 places
&=	Bitwise AND assign	A &= B;

Example

```
(A & B) will give 12 which is 0000 1100
(A | B) will give 61 which is 00111101
(A ^ B) will give 49 which is 0011 0001
(~A) will give -61 which is 1100 0011 in 2's complement
form due to a signed binary number.
```

```
A << 2 will give 240 which is 1111 0000
A >> 2 will give 15 which is 0000 1111
```

PROGRAM: Operator complement (~) is used to determine the maximum value of an integer that can be stored on my PC.

```
#include<iostream>
   using namespace std;
int main()
{
unsigned A = 0;
A = ~ A;
cout << "A = "<<A <<endl;
 }
```

The expected output is given below.

```
A = 4294967295
```

PROGRAM: – Illustrates bitwise AND operator.

```
#include<iostream>
using namespace std;
   int main ()
   {
short A =24;
short B = 8;
int C = A&B;
cout << "C = "<<C <<endl;
}
```

The expected output is given below.

```
C=8
```

The output is explained below. A short number is stored in two bytes, therefore,

```
24 in binary is equal to 00000000 00011000
8 in binary is equal to  00000000 00001000
Bitwise AND is equal to 00000000 00001000
The result 0000000000001000 in binary is equal to 8.
```

PROGRAM: – Illustrates the operators OR and left shift (<<).

```
#include<iostream>
   using namespace std;
   int main ()
   {
  short A =42;
   short B = 12;
int C = A|B; int D = A<<1;
cout << "C = "<<C <<endl; cout<< "D = "<<D <<endl;
}
```

The expected output is as under.

```
C = 46 D = 84
```

An explanation of output is given below. A short number is stored in two bytes or 16 bits. Therefore,

```
42 in binary is equal to 00000000 00101010
12 in binary is equal to 00000000 00001100
Bitwise OR operation results in 00000000 00101110
This is equal to 46 in decimal
```

For the variable D, the digits of A are shifted left by 1 place and assigned to D. So the original binary number 00000000 00101010 now becomes 00000000 01010100 which is equal to 84, i.e. the number gets multiplied by 2. Thus shifting a digit to the left means multiplication by two.

PROGRAM: – Illustrates the operators ^ , >>= and <<= .

```
#include<iostream>
   using namespace std;
   int main ()
   {
short A =42;
   short B = 12;
    short C = 24;
short D = A^B; // XOR operator C <<= 1;
A <<=2; // Shift to left by 2 places and assign B>>=2; //
shift right by 2 places and assign
cout<< "A = "<<A<< " \tB = "<< B <<endl; cout << "C = "<<C
<<endl;
cout << "D = "<< D <<endl;
}
```

The expected output is given below.

```
A = 168 B = 3 C = 48
D = 38
```

The binary equivalent of A = 42 has been shifted two places to the left. So, the number gets multiplied by 4. So, A = 168. Similarly, the right shift by two places in the case of B divides it by 4. The binary equivalent of 24 has been shifted to left by one place and assigned to C. Therefore, C = 24×2 = 48. For A ^ B see the following explanation

```
Binary equivalent of 42   is 00000000 00101010
Binary equivalent of 12 00000000 00001100
Bitwise XOR operation gives 00000000 00100110
This is equivalent to 38.
```

Exercise: Write a code that uses bitwise operations to assess whether a given integer is a power of two.

Hint: The binary string of the even integer 2^p consists of zeros led by one, whereas the binary string of the odd integer $2^p - 1$ consists of ones.

5.7.8.5. Assignment Operators

There are the following assignment operators supported by C++ language:

Table 13: The common assignment operators supported by C++ language

Operator	Description
=	A simple assignment operator assigns values from right side operands to left side operand
+=	Add AND assignment operator, it adds right operand to the left operand and assigns the result to left operand C
-=	Subtract AND assignment operator, it subtracts right operand from the left operand and assigns the result to left operand
*=	Multiply AND assignment operator, it multiplies right operand with the left operand and assigns the result to left operand
/=	Divide AND assignment operator, it divides left operand with the right operand and assigns the result to left operand
%=	Modulus AND assignment operator, it takes modulus using two operands and assign the result to left operand
<<=	Left shift AND assignment operator
>>=	Rig ht shift AND assignment operator
&=	Bitwise AND assignment operator
^=	bitwise exclusive OR and assignment operator bitwise inclusive OR and
|=	assignment operator

Example

```
C = A + B will assig n value of A + B into C
+= A is equivalent to C = C + A
C -= A is equivalent to C = C - A
C *= A is equivalent to C = C * A
C /= A is equivalent to C = C / A
C %= A is equivalent to C = C % A
C <<= 2 is same as C = C << 2
C >>= 2 is same as C = C >> 2
C &= 2 is same as C = C & 2
```

```
C ^= 2 is same as C = C ^ 2
C |= 2 is same as C = C | 2
```

5.7.8.6. Misc Operators

There are few other operators supported by C++ Language.

Table 14: The miscellaneous operators supported by C++ language

Operator	Description
sizeof	sizeof operator returns the size of a variable. For example, sizeof(a), where a is an integer, will return 4.
Condition? X: Y	Conditional operator. If Condition is true? then it returns value X: Otherwise value Y
,	Comma operator causes a sequence of operations to be performed. The value of the entire comma expression is the value of the last expression of the comma-separated list.
. (dot) and -> (arrow)	Member operators are used to referencing individual members of classes, structures, and unions.
cast	Casting operators convert one data type to another. For example, int (2.2000) would return 2.
&	Pointer operator & returns the address of a variable. For example, &a; will give the actual address of the variable.
*	Pointer operator * is a pointer to a variable. For example, *var; will pointer to

Example on Condition operator: min = a<b? a: b; //Here if expression (a<b) is true then the value of "a" will be assigned to min otherwise the value of b will be assigned to min.

What you are expecting from the given code?

```
#include<iostream>
using namespace std;
int main ()
{
int a, b, max;
a = 1, b = 2 ;
cout << "Variable a value is: " ;
cout << ( ( a != 1 ) ? "not one, " : "one, " ) ;
cout << ( ( a % 2 != 0 ) ? "odd" : "even" ) ;
```

```
cout << endl << "Variable b value is: " ;
cout << ( ( b != 1 ) ? "not one, " : "one, " ) ;
cout << ( ( b % 2 != 0 ) ? "odd" : "even" ) ;
max = ( a > b ) ? a : b ;
cout << endl << "Greater value is: " << max << endl ;
return 0 ;
}
```

5.8. Operators Precedence in C++

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator: For example, x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has higher precedence than +, so it first gets multiplied with 3*2 and then adds into 7. Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Table 15: Precedence of operators in C++

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

For example, in

```
a == b + c * d
```

c * d is evaluated first because * has higher precedence than + and ==. The result is then added to b because + has higher precedence than ==, and then == is evaluated. Precedence rules can be overridden using brackets. For example, rewriting the above expression as

```
a == (b + c) * d
```

causes + to be evaluated before *.

Operators with the same precedence level are evaluated in the order specified by the last column of the above Table 15. For example, in

```
a = b += c
```

the evaluation order is right to left, so first b += c is evaluated, followed by a = b.

5.9. Simple Type Conversion

A value in any of the built-in types we have seen so far can be converted (type-cast) to any of the other types. For example:

```
(int) 3.14       // converts 3.14 to an int to give 3
(long) 3.14            // converts 3.14 to along to give 3L
(double) 2       // converts 2 to a double to give 2.0
(char) 122       // converts 122 to a char whose code is 122
(unsigned short) 3.14 // gives 3 as an unsigned short
```

As shown by these examples, the built-in type identifiers can be used as type operators. Type operators are unary (i.e., take one operand) and appear inside brackets to the left of their operand. This is called explicit type conversion. When the type name is just one word, an alternate notation may be used in which the brackets appear around the operand:

```
int (3.14) // same as: (int) 3.14
```

In some cases, C++ also performs an implicit type conversion. This happens when values of different types are mixed in an expression. For example:

```
double d = 1;          // d receives 1.0
```

```
int i = 10.5;          // i receives 10
i = i + d;             // means: i = int(double(i) + d)
```

In the last example, i + d involves mismatching types, so i is first converted to double (promoted) and then added to d. The result is a double which does not match the type of i on the left side of the assignment, so it is converted to int (demoted) before being assigned to i.

The above rules represent some simple but common cases for type conversion

5.10. Statements

C++ provides different forms of statements for different purposes. Declaration statements are used for defining variables. Assignment-like statements are used for simple, algebraic computations. Branching statements are used for specifying alternate paths of execution, depending on the outcome of a logical condition. Loop statements are used for specifying computations that need to be repeated until a certain logical condition is satisfied. Flow control statements are used to divert the execution path to another part of the program.

5.10.1. Null Statement

```
Syntax:
            ;
```

Description: Do nothing

5.10.2. The Block Statements

```
Syntax:
{
    [<Declarations>].
  <List of statements/statement block>.
}
```

Any place you can put a single statement, you can put a compound statement, also called a block. A block begins with an opening brace ({) and ends with a closing brace (}). Although every statement in the block must end with a semicolon, the block itself does not end with a semicolon. ***For example***

```
{
      temp = a;
      a = b;
      b = temp;
}
```

This block of code acts as one statement and swaps the values in the variables a and b.

5.10.3. The Assignment Statements.

```
Syntax:
      <Variable Identifier> = < expression>;
```

Description:

The <expression> is evaluated and the resulting value is stored in the memory space reserved for <variable identifier>.

```
      Eg: - int x, y;
            x=5;
            y=x+3;
            x=y*y;
```

Lab

1. Add extra brackets to the following expressions to explicitly show the order in which the operators are evaluated:

```
(n <= p + q && n >= p - q || n == 0)
(++n * q-- / ++p - q)
(n | p & q ^ p << 2 + q)
(p < q ? n < p ? q * n - 2 : q / n + 1 : q - n)
```

2. What will be the value of each of the following variables after its initialization?

```
double      d = 2 * int(3.14);
long  k = 3.14 - 3;
char  c = 'a' + 2;
char  c = 'p' + 'A' - 'a';
```

3. Write a program which inputs a positive integer n and outputs 2 raised to the power of n.
4. Write a C++ program that receives two integer values from the user.The program then should print the sum (addition), difference (subtraction), product (multiplication), quotient (division), and remainder after division (modulus).
5. Why does this program not compile? Show what must be done to fix it?

```
     #include <iostream>
     #include <conio.h>
     int Main () {
       Cout << "Great day Students !!\n";
     return 0;
    }
```

6. Why does this program not compile? Show what must be done to fix it?

```
     #include <iostream>
     #include <iomanip>
     using namespace std
     int main ()
       cout << "Something is very wrong here" << endl;
  return 0;
  }
```

7. Convert each of these formulas into a proper C++ statement.
   a) F = m a (Force = mass times acceleration)
   b) A = PI R^2 (area of a circle)
   c) $total = \frac{n(n+1)}{2}$
   d) x = sin (2 PI y);

e) $x = \dfrac{-B + \sqrt{B^2 - 4AC}}{2A}$

f) $y = ae^x + b$

8. Write a program that converts a Fahrenheit temperature to Celsius. The formula is

$$C = \frac{5}{9}(F - 32)$$

9. Write a program that, prompts the user to input a set of values for the initial height, initial velocity, and the final height. Then calculate and display the number of seconds until the object reaches the final height. All data are in the metric system. Assume that the final height is less than the initial height. All program variables should be doubles. Use the quadratic equation to solve for t. However, you must determine whether to use the + or – root.

$$y = y_0 + v_0 t - \frac{g}{2} t^2$$

where

y_0 is the initial height of the object

v_0 is the initial velocity of the falling object y is the final height of the object

t is the time

g is the gravitational acceleration: 9.8 m/s/s

10. Write a program that reads in the radius and length of a cylinder and computes volume using the following formulas:

```
area = radius * radius * π
volume = area * length
```

11. Write a program that calculates the energy needed to heat water from an initial temperature to a final temperature. Your program should prompt the user to enter the amount of water in kilograms and the initial and final temperatures of the water. The formula to compute the energy is

    Q = M * (final temperature – initial temperature) * 4184

    *where **M** is the weight of water in kilograms, temperatures are in degrees Celsius, and energy **Q** is measured in joules.*

12. Write a program that prompts the user to enter the starting velocity v_0 in meters/second, the ending velocity v_1 in meters/second, and the period *t* in seconds, and displays the average acceleration. Average acceleration is defined as the change of velocity divided by the time taken to make the change, as shown in the following formula:

$$a = \frac{v_1 - v_0}{t}$$

13. In mathematics, the midpoint between the two points (x_1,y_1) and (x_2,y_2) is computed by the formula

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$$

    Write a C++ program that receives two mathematical points from the user and computes and prints their midpoint.

Exercise

1. Why is C++ a good programming language to learn?
2. How many main functions should a C++ program have?
3. What is a standard library file?

4. What is the purpose of an include directive?
5. What does a preprocessor do?
6. What does a compiler do?
7. What does a linker do?
8. Is the size of a data type always the same no matter which computer you may be working on?
9. What is the difference between an unsigned and signed data type?
10. What does the sizeof operator do?
11. What is a literal string?
12. What is the effect of declaring a variable?
13. Can you refer to a variable before declaring it as long as you declare it later?
14. Can you use one cin statement to assign values to several variables of different data types?
15. Which of the arithmetic operators can operate on the string as well as numeric operands?
16. What operator or function do you use to raise a number to a certain power?
17. State step by step the sequence of operations that a computer would follow to evaluate the following expressions. (Hint - use precedence of operators)

    a) Z=5y+3y(10y+5/2) for y=2.
    b) Z=7y%2+2(3+(y%3+2))fory=19

18. Answer the following statements as either true or false:

    a. A data type defines a set of values and the set of operations that can be applied to those values.
    b. Reserved or keywords can be used as identifier names.
    c. The concept of precedence says that some operators (like multiplication and division) are to be executed before other operators (like addition and subtraction).
    d. An operator that needs two operands will promote one of the operands as needed to make both operands be of the same data type.

e. Parentheses change the precedence of operators.

19. What values do the following arithmetic expressions have?

a. 3 / 10
b. 11 % 4
c. 15 / 2.0
d. 3 + 4 % 5
e. 3 * 7 % 4
f. 7 % 4 *

20. How are operands and operators in the following expression associated?

x = –4 * i++ – 6 % 4;

a. Insert parentheses to form equivalent expressions.
b. What value will be assigned in part to the variable x if the variable i has a value of –2?

21. The int variable x contains the number 7. Calculate the value of the following logical expressions:

```
a. x < 10 && x  >= -1
b. !x && x >=3
c. x++ == 8 || x == 7
```

22. Which of the following represent valid identifiers?

```
identifier
seven_11
_unique_
gross-income
gross$income
2by2
default
average_weight_of_a_large_pizza
```

```
variable
object.orient
```

23. Define variables to represent the following entities:

- Age of a person.
- The income of an employee.
- Several words in a dictionary.
- A letter of the alphabet.
- A greeting message.

24. Write a corrected code of the following program segment by underlining the error corrected.

```
void main()
{
int a,b;
cin<<a;
b=a;
cout <<"b=",a;
};
```

25. The following program contains several errors:

```
*/ Now you should not forget your glasses //
#include <stream>
int main
{
cout << "If this text",
cout >> " appears on your display, ";
cout << " endl;"
cout << 'you can pat yourself on '
<< " the back!" << endl.
return 0;
```

```
)
```

Resolve the errors and run the program to test your changes.

26. What will be the output of the following C++ code?

```
#include <iostream>
using namespace std;
int main ()
{
    int x, y;
    x = 5;
    y = ++x * ++x;
    cout << x << y;
    x = 5;
    y = x++ * ++x;
    cout << x << y;
    return 0;
}
```

27. What is the output when the following code fragment is executed?

```
int n, k = 5;
n = (100 % k ? k + 1 : k - 1);
cout << "n = " << n << " k = " << k << endl;
```

28. What is the output when the following code fragment is executed?

```
int n;
float x = 3.8;
n = int(x);
cout << "n = " << n << endl;
```

29. Write a C++ program that outputs the following text on screen:

```
Oh, what
```

```
a happy day!
Oh yes,
what a happy day!
```

30. Prompt the user for two numbers A & B, computes and displays C=A/B. If the number B is zero, displays a “division by Zero” message.
31. Write a C++ program that two defines variables for floating-point numbers and initializes them with the values 123.456 and 76.543
32. Then display the sum and the difference of these two numbers on a screen.
33. Write a program that reads two integers (in any order) and then print either “multiple” or “not “according to whether the 2nd number is multiple of the 1st one or not.
34. Write a program that takes two numbers from the user and determines that the first number is a factor of the second number.
35. The size of the operator can be used to determine the number of bytes occupied in memory by a variable of a certain type. For example, the size of(short) is equivalent to 2. Write a C++ program that displays the memory space required by each fundamental type on screen

References

1. Hubbard, J. R. Theory and problems of programming with C++/by John R. Hubbard. *Schaum's outline series.*
2. Milewski, B. (2001). *C++ In Action: Industrial-Strength Programming Techniques*. Addison-Wesley.
3. Langr, J. (2013). *Modern C++ Programming with Test-Driven Development: Code Better, Sleep Better*. Pragmatic Bookshelf.
4. Hughes, C. (2008). *Professional Multicore Programming Design and Implementation for C++ Developers*. John Wiley & Sons.

Chapter Six

6. Control Flow Statements

Objectives: Execution of the statements in a C++ program has its flow. There are various control flow statements in C++ that students are supposed to learn.

At the end of this chapter, students should be able to:

- *know what control flow statements are*
- *differentiate conditional selection statements, looping statements, and jump statements in C++ programming*
- *use if, if…else, if…else if…else conditional statements in a C++ program*
- *use the while, for, and do…while repetition statements to execute statements in a program repeatedly*
- *use the break, continue, and goto program control statements to alter the flow of control*
- *undergo multiple selections using the switch selection statement*
- *perform nested conditional selection statements to form complex conditional logic*

6.1. Introduction

A running program spends all of its time executing statements. The order in which statements are executed is called flow control (or control flow). This term reflects the fact that the currently executing statement has the control of the CPU, which when completed will be handed over (flow) to another statement. Flow control in a program is typically sequential, from one statement to the next, but maybe diverted to other paths by branch statements. Flow control is an important consideration because it determines what is executed during a run and what is not, therefore affecting the overall outcome of the program. Like many other procedural languages, C++ provides different forms of statements for different purposes. Declaration statements are used for defining variables. Assignment-like statements are used for simple, algebraic computations. Branching

statements are used for specifying alternate paths of execution, depending on the outcome of a logical condition. Loop statements are used for specifying computations, which need to be repeated until a certain logical condition is satisfied. Flow control statements are used to divert the execution path to another part of the program. We will discuss these in turn.

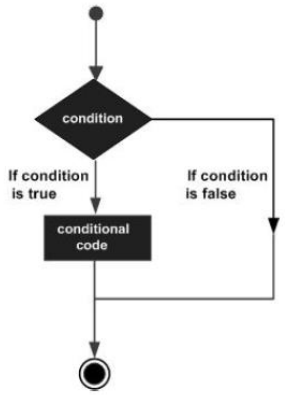


Figure 36. A general from of a typical decision-making structure

Control Statements sometimes known as decision-making statements. Decision making structures require that the programmer specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

Following is the general form of a typical decision-making structure found in most of the programming languages:

C++ programming language provides the following types of decision-making statements. These are conditional statements, Looping Statements, and others

6.2. Conditional Statements

6.2.1. The if Statement

It is sometimes desirable to make the execution of a statement dependent upon a condition being satisfied. The different forms of the ‘If’ statement will be used to decide whether to execute part of the program based on a condition that will be tested either for the TRUE or FALSE result.

A simple if statement consists of a Boolean expression followed by one or more statements. If the given condition is true then it will execute statements; otherwise, it will execute the optional statements.

The braces {and} are used to group declaration and statements into a compound statement or a block. These compound statements or blocks are always considered as a single statement. The basic simple structure of if the statement is shown below

Syntax:

```
if(boolean_expression)

{

                // statement(s) will execute if the boolean expression is
true

}
```

If the Boolean expression evaluates to true, then the block of code inside the if statement will be executed. If the Boolean expression evaluates to false, then the first set of code after the end of the if statement (after the closing curly brace) will be executed.

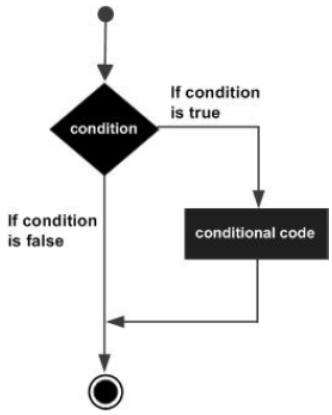


Figure 37. Control flow of a simple if statement block in C++

For example, when dividing two values, we may want to check that the denominator is nonzero:

```
if (count != 0)

average = sum / count;
```

To make multiple statements dependent on the same condition, we can use a compound statement:

```
if (balance > 0) {

              interest = balance * creditRate;

                          balance += interest;
```

```
}
```

Example2

```
#include <iostream.h>

int main ()

{

// local variable declaration:

int a = 10;

// check the boolean condition

if( a < 20 )

{

// if condition is true then print the following

cout << "a is less than 20;" << endl;



}

cout << "value of a is: " << a << endl;

return 0;

}
```

When the above code is compiled and executed, it produces the following result:

```
a is less than 20;

value of a is: 10
```

6.2.2. The if…. else Statement

A variant form of the if statement allows us to specify two alternative statements: one which is executed if a condition is satisfied and one which is executed if the condition is not satisfied. This is called the if-else statement and has the general form:

Syntax:

```
if(boolean_expression)
{
        // statement(s) will execute if the Boolean expression is true
} else
{
          // statement(s) will execute if the Boolean expression is false
}
```

If the Boolean expression evaluates to true, then if a block of code will be executed, otherwise else block of code will be executed.

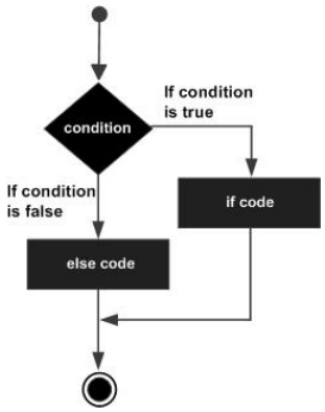


Figure 38. Control flow of an if else block in C++

Example 1

```
if(age>18)
   cout<<"You are aduld";
else
   cout<<"You are a kid";
```

Example 2

```
int x;
cout<<”Enter a number: “;
```

```
cin>>x;

if(x%2==0)

        cout<<" The Number is Even";

else

        cout<<"The Number is Odd";
```

Example 3

```
#include <iostream>

using namespace std;

int main ()

{ // local variable declaration:

int a = 100;

// check the boolean condition

if( a < 20 )

{ // if condition is true then print the following

cout << "a is less than 20;" << endl;

}

else

{ // if condition is false then print the following

cout << "a is not less than 20;" << endl;

}

cout << "value of a is : " << a << endl;

return 0; }
```

When the above code is compiled and executed, it produces the following result:

```
a is not less than 20;

value of a is: 100
```

Example 4: A program to read any two numbers from the keyboard and to display the largest value of them.

```
#include <iostream>
using namespace std;
int main ()
{ // local variable declaration:
float x,y;
cout << "Enter any two Numbers \n ";
cin>>x>>y;
if(x>y)
   cout<< "The largest value is"<<x<<endl;
else
   cout<<"Largest value is "<<y<<endl;
return 0; }
```

When the above code is compiled and executed, it produces the following result:

```
Enter any two numbers
20 40
The largest value is 40
```

6.2.3. The if...else if...else Statement:

An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using a single if...else if statement. When using if, else if, else statements there are few points to keep in mind. An if can have zero or one else's and it must come after any else if. An if can have zero to many else if's and they must come before the else. Once an else if succeeds, none of the remaining else if or else will be tested.

Syntax:

The syntax of an if...else if...else statement in C++ is:

```
if (boolean_expression 1)
{
// Executes when the boolean expression 1 is true
} else if (boolean_expression 2)
{
// Executes when the boolean expression 2 is true
} else if (boolean_expression 3)
{
// Executes when the boolean expression 3 is true
} else
{
// executes when none of the above conditions is true.
}
```

Example:

```
#include <iostream>
using namespace std;
int main ()
{
// local variable declaration:
int a = 100;
// check the boolean condition
if (a == 10)
{
// if condition is true then print the following
cout << "Value of a is 10" << endl;
}
else if (a == 20)
{
// if else if condition is true
cout << "Value of a is 20" << endl;
}
else if (a == 30)
```

```
{
// if else if condition is true
cout << "Value of a is 30" << endl;
}
else
{
// if none of the conditions is true
cout << "Value of a is not matching" << endl;
}
cout << "The exact value of a is: " << a << endl;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Value of a is not matching
The exact value of a is: 100
```

Example 2: A program to display the name of the day in a week, depending upon the number which is entered by the keyboard

```
#include <iostream>
using namespace std;
int main ()
{
int day;
cout << "Enter a number between 1 to 7 \n";
cin>>day;
if (day == 1)
   cout << "Monday \n";
else if (day == 2)
   cout << "Tuesday \n";
else if (day == 3)
   cout << "Wednesday \n";
else if (day == 4)
   cout << "Thursday \n";
else if (day == 5)
```

```
   cout << "Friday \n";
else if (day == 6)
   cout << "Saturday \n";
else if (day == 7)
   cout << "Wednesday \n";
else
   cout << "Enter a correct number \n";
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter a number between 1 to 7
4
Thursday
```

6.2.4. Nested if statement

It is always legal to nest if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax:

The syntax for a nested if the statement is as follows:

```
if (boolean_expression 1)
{// Executes when the boolean expression 1 is true
if (boolean_expression 2)
{// Executes when the boolean expression 2 is true
}
}
```

You can nest else if...else in the similar way as you have nested if statement.

Example:

```
#include <iostream>
using namespace std;
```

```
int main ()
{
// local variable declaration:
int a = 100;
int b = 200;
// check the boolean condition
if( a == 100 )
{
// if condition is true then check the following
if( b == 200 )
{
// if condition is true then print the following
cout << "The alue of a is 100 and b is 200" << endl;
}
}
cout << "The exact value of a is: " << a << endl;
cout << "The exact value of b is: " << b << endl;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
The value of a is 100 and b is 200
The exact value of a is: 100
The exact value of b is: 200
```

Example 2: A program to find the largest value among any three numbers.

```
#include <iostream>
using namespace std;
int main ()
{
float x,y,z;
cout << "Enter any three numbers \n";
cin>>x>>y>>z;
if (x>y){
   if (x>z)
```

```
      cout<< "Largest value: "<<x<<endl;
   else
      cout<< "Largest value: "<<z<<endl;
}
else{
   if (y>z)
      cout<< "Largest value: "<<y<<endl;
   else
      cout<< "Largest value: "<<z<<endl;
}
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter any three numbers
10 40 50
Largest value: 50
```

Example 3: A program to find the largest value among any four numbers.

```
#include <iostream>
using namespace std;
int main ()
{
float a,b,c,d;
cout << "Enter any four numbers \n";
cin>>a>>b>>c>>d;
if(a>b){
    if (a>c){
      if (a>d)
         cout<< "Largest value: "<<a<<endl;
      else
         cout<< "Largest value: "<<d<<endl;
    }
    else{
      if (c>d)
```

```
          cout<< "Largest value: "<<c<<endl;
      else
          cout<< "Largest value: "<<d<<endl;
    }
}
else{
        if (b>c){
          if (b>d)
            cout<< "Largest value: "<<b<<endl;
          else
            cout<< "Largest value: "<<d<<endl;
    }
       else{
          if (c>d)
            cout<< "Largest value: "<<c<<endl;
         else
            cout<< "Largest value: "<<d<<endl;
    }

}
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter any four numbers
10 60 40 50
Largest value: 60
```

6.2.5. C++ switch Statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax:

The syntax for a switch statement in C++ is as follows:

```
switch(expression){
case constant-expression:
statement(s);
break; //optional
case constant-expression:
statement(s);
break; //optional
// you can have any number of case statements.
default : //Optional
statement(s);
}
```

The following rules apply to a switch statement:

- ✓ The expression used in a switch statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type. You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- ✓ The constant-expression for a case must be the same data type as the variable in the switch, and it must be a constant or a literal. When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement. Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- ✓ A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

Flow Diagram:

```
#include <iostream>
using namespace std;
int main ()
{
// local variable declaration:
char grade = 'D';
switch(grade)
{
case 'A' :
cout << "Excellent!" << endl;
break;
case 'B' :
case 'C' :
cout << "Well done" << endl;
break;
case 'D' :
cout << "You passed" << endl;
break;
case 'F' :
cout << "Better try again" << endl;
break;
default :
cout << "Invalid grade" << endl;
}
cout << "Your grade is " << grade << endl;
return 0;
}
```

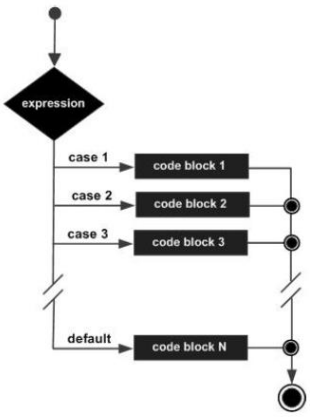


Figure 39. Control flow of a switch statement in C++

This would produce the following result:

```
You passed
Your grade is D
```

Example 2: A program to make arithmetic operations.

```
#include <iostream>
using namespace std;
```

```
int main ()
{
      float operand1, operand2, result;
      char op;
      cout<<"Enter two numbers: \n";
      cin>>operand1>>operand2;
      cout<<"Enter operator: \n";
      cin>>op;
      switch(op){
            case '+':
                  result=operand1 + operand2;
                  cout<<"Sum: "<<result;
                  break;
            case '-':
                  result=operand1 - operand2;
                  cout<<"Subtraction: "<<result;
                  break;
            case '*':
                  result=operand1 * operand2;
                  cout<<"Multiplication: "<<result;
                  break;
            case '/':
                  if(operand2 !=0){
                        result=operand1 / operand2;
                        cout<<"Division: "<<result;
                  }
                  else{
                        cout<<"Can't Divide by Zero";
                  }
                  break;
            default:
                  cout << "unknown operator: " << op << endl;
                  break;
      }
      return 0;
```

```
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter two numbers:
15  8
Enter operator:
-
Subtraction: 7
```

As illustrated by example 2, it is usually necessary to include a **break** statement at the end of each case. The break terminates the switch statement by jumping to the very end of it. There are, however, situations in which it makes sense to have a case without a break. For example, if we extend the above statement to also allow **X** to be used as a multiplication operator, we will have

```
switch (op) {
      case '+':
   result = operand1 + operand2;
   cout<<"Sum: "<<result;
         break;
      case '-':
          result = operand1 - operand2;
        cout<<"Subtraction: "<<result;
        break;
      case 'x':
      case '*':
  result = operand1 * operand2;
  cout<<"Multiplication: "<<result;
        break;
      case '/':
        if(operand2 !=0){
            result=operand1 / operand2;
            cout<<"Division: "<<result;
        }
        else{
```

```
            cout<<"Can't Divide by Zero";
        }
  break;
      default:
  cout << "unknown operator: " << op << '\n';
        break;
}
```

Invalid Switch-case expressions

The following program structures show the invalid usage of constructing the switch-case statements

1. The case labels must not be floating point numbers, they should always be integer constants or character constants

```
switch (value) { //error
      case 10.11:
            ________________________
            ________________________
             break;
      case 5.7:
        ________________________
            ________________________
             break;
      default:
        ________________________
            ________________________
             break;
}
```

2. The case label must not be a string expression

```
switch (value) { //error
      case "good":
            ________________________
```

```
            ________________________
            break;
      case "bad":
           ________________________
            ________________________
            break;
      default:
           ________________________
            ________________________
            break;
}
```

3. The case label cannot be an expression

```
switch (expression) { //error
      case a+b:
            ________________________
            ________________________
            break;
      case a-b:
           ________________________
            ________________________
            break;
      default:
           ________________________
            ________________________
            break;
}
```

Class Ex. Rewrite example 3 from nested if using a switch statement

6.2.6. C++ Nested switch Statements

It is possible to have a switch as part of the statement sequence of an outer switch. Even if the case constants of the inner and outer switch contain common values, no conflicts will arise.

C++ specifies that at least 256 levels of nesting be allowed for switch statements.

Syntax:

The syntax for a nested switch statement is as follows:

```
switch(ch1) {
case 'A':
cout << "This A is part of outer switch";
switch(ch2) {
case 'A':
cout << "This A is part of inner switch";
break;
case 'B': // ...
}
break;
case 'B': // ...
}
```

Example:

```
#include <iostream>
using namespace std;
int main ()
{
int a = 100;
int b = 200;
switch(a) {
case 100:
cout << "This is part of outer switch" << endl;
switch(b) {
```

```
case 200:
cout << "This is part of inner switch" << endl;
}
}
cout << "Exact value of a is : " << a << endl;
cout << "Exact value of b is : " << b << endl;
return 0;
}
```

6.3. Looping Statements

Looping statements control a block of code to be executed repeatedly for a fixed number of times or until a certain condition fails.

There are three C++ repetition statements:

1. The for Statement or loop
2. The while statement or loop
3. The do…while statement or loop

6.3.1. The 'for' Statement

The for statement (also called for loop) is similar to the while statement but has two additional components: an expression that is evaluated only once before everything else, and an expression that is evaluated once at the end of each iteration. The general form of the for statement is:

```
      for (expression1; expression2; expression3)
            statement;
```

First expression1 is evaluated. Each time round the loop, expression2 is evaluated. If the outcome is nonzero then statement is executed and expression3 is evaluated. Otherwise, the loop is terminated. The general for loop is equivalent to the following while loop:

```
            expression1;
      while (expression2) {
            statement;
            expression3;
      }
```

The most common use of for loops is for situations where a variable is incremented or decremented with every iteration of the loop. The following for loop, for example, calculates the sum of all integers from 1 to n.

```
sum = 0;
for (i = 1; i <= n; ++i)
      sum += i;
```

This is preferred to the while-loop version we saw earlier. In this example, i is usually called the loop variable.

C++ allows the first expression in a for loop to be a variable definition. In the above loop, for example, i can be defined inside the loop itself:

```
for (int i = 1; i <= n; ++i)
      sum += i;
```

Contrary to what may appear, the scope for i is not the body of the loop, but the loop itself. Scope-wise, the above is equivalent to:

```
int i;
for (i = 1; i <= n; ++i)
      sum += i;
```

Any of the three expressions in a for loop may be empty. For example, removing the first and the third expression gives us something identical to a while loop:

```
for (; i != 0;)        // is equivalent to:  while (i != 0)
      something;       //                    something;
```

Removing all the expressions gives us an infinite loop. This loop's condition is assumed to be always true:

```
for (;;)                    // infinite loop
      something;
```

For loops with multiple loop variables are not unusual. In such cases, the comma operator is used to separate their expressions:

```
for (i = 0, j = 0; i + j < n; ++i, ++j)
      something;
```

Because loops are statements, they can appear inside other loops. In other words, loops can be nested. For example,

```
for (int i = 1; i <= 3; ++i)
      for (int j = 1; j <= 3; ++j)
            cout << '(' << i << ',' << j << ")\n";
```

produces the product of the set {1,2,3} with itself, giving the output:

```
(1,1)
(1,2)
(1,3)
(2,1)
(2,2)
(2,3)
(3,1)
(3,2)
(3,3)
```

Example: A program to find the sum and average of the given numbers using for loop.

```
#include <iostream>
using namespace std;
int main ()
{
      int n;
      float avg, a, sum;
```

```
      cout<<"How many numbers? \n";
      cin>>n;
      sum=0;
      for (int i=0;i<=n-1;i++){
            cout<<"Enter a number: \n";
            cin>>a;
            sum=sum + a;
      }
      avg= sum/n;
      cout<<"Sum= "<<sum<<'\t';
      cout<<"Average= "<<avg;
      return 0;
}
```

Example 3: A C++ program to convert birr value in to equivalent US dollar using inserted conversion rate for loop.

```
#include <iostream>
using namespace std;
int main ()
{
      float birr, maxBirr, rate;
      cout << "\n* * * TABLE OF EXCHANGE  BIRR TO USD * *
*\n\n";
      cout << "Please give the rate of exchange:  one USD in
Birr: \n";
      cin >> rate;
      cout << "\nPlease enter the maximum # of birr: ";
      cin >> maxBirr;
      cout <<"Birr " << "\t\t" << "USD"<<"\t\tRate: " <<
rate << endl;
      for(birr=1;birr<=maxBirr; ++birr){
            cout<<birr<<"\t\t"<<birr * rate<<endl;
      }
      return 0;
```

```
}
```

6.3.2. The 'while' Statement

The while statement (also called while loop) provides a way of repeating a statement while a condition holds. It is one of the three flavors of iteration in C++. The general form of the while statement is:

```
while (expression)
      statement;
```

The first expression (called the loop condition) is evaluated. If the outcome is nonzero then the statement (called the loop body) is executed and the whole process is repeated. Otherwise, the loop is terminated.

For example, suppose we wish to calculate the sum of all numbers from 1 to some integer denoted by n. This can be expressed as:

```
i = 1;
sum = 0;
while (i <= n)
      sum += i;
```

For n set to 5, Table 16 provides a trace of the loop by listing the values of the variables involved and the loop condition.

Table 16: Loop condition example

Iteration	i	n	i <= n	sum += i++
First	1	5	1	1
Second	2	5	1	3
Third	3	5	1	6
Fourth	4	5	1	10
Fifth	5	5	1	15
Sixth	6	5	0	

It is not unusual for a while loop to have an empty body (i.e., a null statement). The following loop, for example, sets n to its greatest odd factor.

```
while (n % 2 == 0 && n /= 2);
```

Here the loop condition provides all the necessary computation, so there is no real need for a body. The loop condition not only tests that n is even but it also divides n by two and ensures that the loop will terminate should n be zero.

Example 2: A program to find the sum and average of the given numbers.

```
#include <iostream>
using namespace std;
int main ()
{
      int n, i;
      float avg, a, sum;
      cout<<"How many numbers? \n";
      cin>>n;
      i=0;
      sum=0;
      while (i<=n-1){
            cout<<"Enter a number: \n";
            cin>>a;
            sum=sum + a;
            ++i;
      }
      avg= sum/n;
      cout<<"Sum= "<<sum<<'\t';
      cout<<"Average= "<<avg;
      return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
How many numbers:
4
Enter a number:
15
Enter a number:
9
```

```
Enter a number:
23
Enter a number:
78
Sum=125          Average=31.25
```

Example 3: A C++ program to convert birr value into equivalent US dollar using n inserted conversion rate.

```
#include <iostream>
using namespace std;
int main ()
{
      float birr=1, maxBirr, rate;
      cout << "\n* * * TABLE OF EXCHANGE  BIRR TO USD * *
*\n\n";
      cout << "Please give the rate of exchange:  one USD in
Birr: \n";
      cin >> rate;
      cout << "\nPlease enter the maximum # of birr: ";
      cin >> maxBirr;
      cout <<"Birr " << "\t\t" << "USD"<<"\t\tRate: " <<
rate << endl;
      while(birr<=maxBirr){
            cout<<birr<<"\t\t"<<birr * rate<<endl;
            ++birr;
      }
      return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
*** TABLE OF EXCHANGE BIRR TO USD ***
Please give the rate of exchange: one USD in Birr:
30.25
Please enter the maximum # of birr:
5
```

```
Birr            USD             Rate:30.25
30.25
60.5
90.75
121
151.25
```

6.3.3. The 'do…while' Statement

The do statement (also called do loop) is similar to the while statement, except that its body is executed first and then the loop condition is examined. The general form of the do statement is:

```
      do
            statement;
      while (expression);
```

The first statement is executed and then the expression is evaluated. If the outcome of the latter is nonzero then the whole process is repeated. Otherwise, the loop is terminated.

The do loop is less frequently used than the while loop. It is useful for situations where we need the loop body to be executed at least once, regardless of the loop condition. For example, suppose we wish to repeatedly read a value and print its square, and stop when the value is zero. This can be expressed as the following loop:

```
do {
      cin >> n;
      cout << n * n << '\n';
} while (n != 0);
```

Unlike the while loop, the do loop is never used in situations where it would have a null body. Although a do loop with a null body would be equivalent to a similar while loop, the latter is always preferred for its superior readability.

6.4. Pitfalls in Writing Looping Statements

There are some pitfalls worth mentioning in repletion statements. These pitfalls are the most common programming errors committed by programmers

- **Infinite loop**: no matter what you do with the while loop (and other repetition statements), make sure that the loop will eventually terminate.

Example 1: Do you know why the following is an infinite loop?

```
 int product = 0;
 while(product < 50)
product *= 5;
```

Example 2: Do you know why the following is an infinite loop?

```
int counter = 1;
 while(counter != 10)
      counter +=2
```

In the first example, since the product is initialized with zero, the expression "product*=5" will always give us zero which will always be less than 50.

In the second example, the variable counter is initialized to 1 and increment is 2, the counter will never be equal to 10 as the counter only assumes odd values. In theory, this while loop is an infinite loop, but in practice, this loop eventually terminates because of an overflow error as counter is an integer it will have a maximum limit.

Off-By-One Bugs (OBOB): another thing for which you have to watch out in writing a loop is the so-called *Off-By-One Bugs or errors*. Suppose we want to execute the loop body 10 times. Does the following code work?

Example 1

```
  count = 1;
  while(count < 10)
    {
…
      count++;
   }
```

No, the loop body is executed nine times. How about the following?

Example 2

```
count = 0;
while(count <= 10)
{
      …
      count++;
}
```

No, this time the loop body is executed eleven times. The correct is

Example 3

```
count = 0;
while(count < 10)
{
 …
 count++;
}
```

OR

```
count = 1;
while(count <= 10)
{
 …
 count++;
}
```

6.5. Other Statements

6.5.1. The 'continue' Statement

The continue statement terminates the current iteration of a loop and instead jumps to the next iteration. It applies to the loop immediately enclosing the continue statement. It is an error to use the continue statement outside a loop.

In a while and do loops, the next iteration commences from the loop condition. In a for loop, the next iteration commences from the loop's third expression. For example, a loop which repeatedly reads in a number, processes it but ignores negative numbers, and terminates when the number is zero, may be expressed as:

```
do {

       cin >> num;

       if (num < 0) continue;

       // process num here...

} while (num != 0);
```

This is equivalent to:

```
do {

       cin >> num;

       if (num >= 0) {

              // process num here...

       }

} while (num! = 0);
```

A variant of this loop which reads in a number exactly n times (rather than until the number is zero) may be expressed as:

```
for (i = 0; i < n; ++i) {

      cin >> num;

      if (num < 0) continue;                // causes a jump to: ++i

      // process num here...

}
```

When the continue statement appears inside nested loops, it applies to the loop immediately enclosing it, and not to the outer loops. For example, in the following set of nested loops, the continue applies to the for loop, and not the while loop:

```
while (more) {
      for (i = 0; i < n; ++i) {
            cin >> num;
            if (num < 0) continue;                // causes
a jump to: ++i
            // process num here...
      }
      //etc...
}
```

6.5.2. *The 'break' Statement*

A break statement may appear inside a loop (while, do, or for) or a switch statement. It causes a jump out of these constructs and hence terminates them. Like the continue statement, a break statement only applies to the loop or switch immediately enclosing it. It is an error to use the break statement outside a loop or a switch.

For example, suppose we wish to read in a user password, but would like to allow the user a limited number of attempts:

```
for (i = 0; i < attempts; ++i) {
```

```
      cout << "Please enter your password: ";
      cin >> password;
      if (Verify(password)) // check password for
correctness
            break;                      // drop out of the
loop
      cout << "Incorrect! \n";
}
```

Here we have assumed that there is a function called Verify which checks a password and returns true if it is correct, and false otherwise.

Rewriting the loop without a break statement is always possible by using an additional logical variable (verified) and adding it to the loop condition:

```
verified = 0;
for (i = 0; i < attempts &&! verified; ++i) {
      cout << "Please enter your password: ";
      cin >> password;
      verified = Verify(password));
      if (! verified)
            cout << "Incorrect! \n";
}
```

The break version is arguably simpler and therefore preferred.

6.5.3. The 'goto' Statement

The goto statement provides the lowest-level of jumping. It has the general form:

```
goto label;
```

where label is an identifier which marks the jump destination of goto. The label should be followed by a colon and appear before a statement within the same function as the goto statement itself. For example, the role of the break statement in the for loop in the previous section can be emulated by a goto:

```
for (i = 0; i < attempts; ++i) {
```

```
      cout << "Please enter your password: ";
      cin >> password;
      if (Verify(password)) // check password for
correctness
            goto out;             // drop out of the loop
      cout << "Incorrect!\n";
}
out:
//etc...
```

Because goto provides a free and unstructured form of jumping (unlike break and continue), it can be easily misused. Most programmers these days avoid using it altogether in favor of clear programming. Nevertheless, goto does have some legitimate (though rare) uses.

6.5.4. The 'return' Statement

The return statement enables a function to return a value to its caller. It has the general form:

```
      return expression;
```

where expression denotes the value returned by the function. The type of this value should match the return type of the function. For a function whose return type is void, the expression should be empty:

```
      return;
```

The only function we have discussed so far is main, whose return type is always int. The return value of main is what the program returns to the operating system when it completes its execution. Under UNIX, for example, it's conventional to return 0 from main when the program executes without errors. Otherwise, a non-zero error code is returned. For example:

```
int main (void)
{
      cout << "Hello World\n";
```

```
      return 0;
}
```

When a function has a non-void return value (as in the above example), failing to return a value will result in a compiler warning. The actual return value will be undefined in this case (i.e., it will be whatever value that happens to be in its corresponding memory location at the time).

Lab

1. Write a program which inputs an integer value, checks that it is positive, and outputs its factorial, using the formulas:

```
factorial (0) = 1
factorial(n) = n × factorial(n-1)
```

2. Write a program that inputs a person's height (in centimeters) and weight (in kilograms) and outputs one of the messages: underweight, normal, or overweight, using the criteria:

```
Underweight:     weight < height/2.5
Normal:    height/2.5 <= weight <= height/2.3
Overweight:      height/2.3 < weight
```

3. Assuming that n is 20, what will the following code fragment output when executed?

```
if (n >= 0)
      if (n < 10)
            cout << "n is small\n";
else
      cout << "n is negative\n";
```

4. Write for, do-while, and while statements to compute the following sums and products

```
A. 5+10+15+…+50
```

```
B. 1+1/2+1/3+1/4+…1/15
C. 1*2*3*…*20
```

5. Write an application to print out the numbers 10 through 49 in the following manner

```
10 11 12 13 14 15 16 17 18 19

20 21 22 23 24 25 26 27 28 29

30 31 32 33 34 35 36 37 38 39
40 1 42 43 44 45 46 47 48 49
```

6. Write a weather-calculator program that asks for a list of the previous 10 days' temperatures, computes the average, and prints the results. You have to compute the total as the input occurs, then divide that total by 10 to find the average. Use a while loop for the 10 repetitions.
7. Write a C++ application that can compute the letter grade of a student after accepting the student's mid and final mark. The program should only accept mid result [0-40] and final [0- 60]. If the data entered violates this rule, the program should display that the user should enter the mark in the specified range. The program is also expected to run until the user refuses to continue.

Exercise

1. Write a program that inputs date in the format dd/mm/yy and outputs it in the format month dd, year. For example, *25/12/61* becomes:

```
December 25, 1961
```

2. Write a program that inputs an octal number and outputs its decimal equivalent. The following example illustrates the expected behavior of the program:

```
Input an octal number: 214
Octal (214) = Decimal (532)
```

3. Write a program that produces a simple multiplication table of the following format for integers in the range 1 to 9:

```
1 x 1 = 1
1 x 2 = 2
...
9 x 9 = 81
```

4. Write a C++ program that accepts marks of five students and then displays their average. The program should not accept mark which is less than 0 and mark greater than 100.
5. A prime number is an integer greater than one and divisible only by itself and one. The first seven prime numbers are 2, 3, 5, 7, 11, 13, and 17. Write a program that displays all the prime numbers between 1 and 100.
6. Write a C++ program that counts the number of digits in an integer number. For example; 23,498 has five digits.
7. Write a program in C++ to read a positive integer number n from a standard input device to display the number and digit. For example, n=5678

```
Output
5
6
7
8
```

8. Write a program in C++ to read a number n, and digit d, and check whether d is present in the number n. if it is so, count how many times it is repeated in the number n.

```
n=12567
Digit to be checked 5.
The digit is present once
```

9. Write a C++ program using for loop that can compute the following summation:

$$\sum_{i=1}^{30}[(i/3) * 2]$$

10. Write a program to read a number n and find out the sum of the integers from 1 to 2, then from 1 to 3, then 1 to 4, and so forth and to display the sum of the integers from 1 to n.

```
For example,
      From 1 to 2 =    1
      1 to 3           3
      1 to 4           6
      1 to 5           10
      1 to 6           18
```

References

[1] D. Ravichandran, Programming with C++ (3rd ed, Tata McGraw-Hill Education, 2011

[2] Ulla Kirch-Prinz, Peter Prinz, A Complete Guide to Programming in C++, Jones and Bartlett publishers, 2002

Chapter Seven

7. Function

Objectives: Statements that perform a certain task could be capsuled in a function. In C++, functions are very fundamental and students will learn about the fundamentals of functions.

At the end of this chapter, students should be able to:

- *explain why the function is important in C++ programs*
- *know how to declare functions in C++ programs*
- *know how to call a declared function in C++ programs*
- *differentiate call by value and call by reference*
- *develop C++ functions that take arguments*
- *use a local and global variable in C++ programs*
- *develop C++ functions that return values*
- *develop overloaded functions in C++ programs*
- *apply recursion in C++ programs*

7.1. Introduction to Function

A function is a group of statements that together perform a task. A function is a subprogram that can act on data and return a value. Every C++ program has at least one function, main(). When your program starts, main() is called automatically. main() might call other functions, some of which might call still others. Each function has its name, and when that name is encountered, the execution of the program branches to the body of that function. When the function returns, execution resumes on the next line of the calling function. When a program calls a function, execution switches to the function and then resumes at the line after the function call. Well-designed functions perform a specific and easily understood task. Complicated tasks should be broken down into multiple functions, and then each can be called in turn.

Functions come in two varieties: user-defined and built-in. Built-in functions are part of your compiler package--they are supplied by the manufacturer for your use. How you divide up your code among different functions is up to you, but logically the division usually is so each function performs a specific task.

A function declaration tells the compiler about a function's name, return type, and parameters. A function definition provides the actual body of the function.

The C++ standard library provides numerous built-in functions that your program can call. A function is knowing with various names like a method or a sub-routine or a procedure etc.

There are two types of functions in C++: Built-in functions (Library functions) and user-defined functions.

Built-in functions are are the functions which are declared in the C++ header files such as pow(x,y), cmath, ceil(x), cos(x), exp(x), etc. We need not declare and define these functions as they are already written in the C++ libraries. We can directly call them when needed.

Example: Here, using built-in function pow(x,y) which is x to the power y. This function is declared in cmath header file so we have included the file in our program using #include directive.

```
#include <iostream>
#include <cmath>
using namespace std;
int main(){
    /* Calling the built-in function
     * pow(x, y) which is x to the power y
     * We are directly calling this function
     */
    cout<<pow(2,5);
    return 0;
}
```

User-defined functions are the functions that are created by the programmer so that the programmer can use them many times. It reduces the complexity of a big program and optimizes the code.

Example: Here, a function which adding two numbers

```
#include <iostream>
#include <cmath>
using namespace std;
//Declaring the function sum
int sum(int,int);
int main(){
   int x, y;
   cout<<"enter first number: ";
   cin>> x;
   cout<<"enter second number: ";
   cin>>y;
   cout<<"Sum of these two :"<<sum(x,y);
   return 0;
}
//Defining the function sum
int sum(int a, int b) {
   int c = a+b;
   return c;
}
```

7.1.1. Defining a Function

The definition of a function consists of the function header and its body. The header is exactly like the function prototype, except that the parameters must be named, and there is no terminating semicolon. The body of the function is a set of statements enclosed in braces. The general form of a C++ function definition is as follows:

```
return_type function_name (parameter list)
{
body of the function
```

```
}
```

A function prototype tells the compiler the return type, name, and parameter list. Functions are not

required to have parameters, and if they do, the prototype is not required to list their names, only their types. A prototype always ends with a semicolon (;). A function definition must agree in the return type and parameter list with its prototype. It must provide names for all the parameters, and the body of the function definition must be surrounded by braces. All statements within the body of the function must be terminated with semicolons, but the function itself is not ended with a semicolon; it ends with a closing brace. If the function returns a value, it should end with a return statement, although return statements can legally appear anywhere in the body of the function. Every function has a return type. If one is not explicitly designated, the return type will be int. Be sure to give every function an explicit return type. If a function does not return a value, its return type will be void.

A C++ function definition consists of a function header and a function body. Here are all the parts of a function:

- ***Return Type:*** A function may return a value. The return_type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword void.
- ***Function Name:*** This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- ***Parameters***: A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as the actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- ***Function Body:*** The function body contains a collection of statements that define what the function does.

Function Definition Examples:

```
long Area(long l, long w)   //long return data type
{
 return l * w;
}
void PrintMessage(int whichMsg)  //void return type
{
 if (whichMsg == 0)
 cout << "Hello.\n";
 if (whichMsg == 1)
 cout << "Goodbye.\n";
 if (whichMsg > 1)
 cout << "I'm confused.\n";
}
```

Example: Following is the source code for a function called max (). This function takes two parameters num1 and num2 and returns the maximum between the two:

```
// function returning the max between two numbers
int max (int num1, int num2)
{
// local variable declaration
int result;
if (num1 > num2)
result = num1;
else
result = num2;
return result;
}
```

7.1.2. Function Declarations

There are three ways to declare a function:

- Write your prototype into a file, and then use the #include directive to include it in your program.

- Write the prototype into the file in which your function is used.
- Define the function before it is called by any other function. When you do this, the definition acts as its declaration.

Although you can define the function before using it, and thus avoid the necessity of creating a function prototype, this is not good programming practice for three reasons.

- First, it is a bad idea to require that functions appear in a file in a particular order.
- Doing so makes it hard to maintain the program as requirements change.
- Second, it has possible that function A() needs to be able to call function B(), but function B() also needs to be able to call function A() under some circumstances. It is not possible to define function A() before you define function B() and also to define function B() before you define function A(), so at least one of them must be declared in any case.
- Third, function prototypes are a good and powerful debugging technique. If your prototype declares that your function takes a particular set of parameters, or that it returns a particular type of value, and then your function does not match the prototype, the compiler can flag your error instead of waiting for it to show itself when you run the program.

A function declaration tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately. A function declaration has the following parts:

```
return_type function_name( parameter list );
```

For the above-defined function max(), the following is the function declaration:

int max(int num1, int num2); Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration:

```
int max(int, int);
```

The function declaration is required when you define a function in one source file and you call that function in another file. In such a case, you should declare the function at the top of the file calling the function.

7.1.3. Calling a Function

While creating a C++ function, you define what the function has to do. To use a function, you will have to call or invoke that function. When a program calls a function, program control is transferred to the called function. A called function performs the defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns program control to the main program.

To call a function, you simply need to pass the required parameters along with function name, and if function returns a value, then you can store returned value. For example:

```
#include <iostream>
using namespace std;
int max(int num1, int num2);
int main ()
{
// local variable declaration:
int a = 100;
int b = 200;
int ret;
// calling a function to get max value.
ret = max(a, b);
cout << "Max value is : " << ret << endl;
return 0;
}
// function returning the max between two numbers
int max(int num1, int num2)
{
// local variable declaration
int result;
if (num1 > num2)
result = num1;
else
result = num2;
return result;
```

```
}
```

While running the final executable, it would produce the following result:

```
Max value is: 200
```

7.2. Function Arguments

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the formal parameters of the function.

The formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways that arguments can be passed to a function:

Table 17: Types of function calls

Call Type	Description
Call by value	This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function do not affect the argument.
Call by reference	This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

7.2.1. Call by value

The call by value method of passing arguments to a function copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function do not affect the argument.

By default, C++ uses call by value to pass arguments. In general, this means that code within a function cannot alter the arguments used to call the function. Consider the function swap() definition as follows.

```
// function definition to swap the values.
void swap(int x, int y)
```

```
{
int temp;
temp = x; /* save the value of x */
x = y; /* put y into x */
y = temp; /* put x into y */
return;
}
```

Now, let us call the function swap() by passing actual values as in the following example:

```
# include <iostream>
using namespace std;
// function declaration
void swap(int x, int y);
int main ()
{
// local variable declaration:
int a = 100;
int b = 200;
cout << "Before the swap, the value of a:" << a << endl;
cout << "Before the swap, the value of b:" << b << endl;
// calling a function to swap the values.
swap(a, b);
cout << "After the swap, value of a:" << a << endl;
cout << "After the swap, value of b:" << b << endl;
return 0;
}
```

When the above code is put together in a file, compiled, and executed, it produces the following result:

```
Before swap, the value of a:100
Before swap, the value of b:200
After the swap, the value of a:100
After the swap, the value of b:200
```

This shows that there is no change in the values though they had been changed inside the function.

7.2.2. Call by reference

The call by reference method of passing arguments to a function copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the passed argument.

To pass the value by reference, argument reference is passed to the functions just like any other value. So accordingly you need to declare the function parameters as reference types as in the following function swap(), which exchanges the values of the two integer variables pointed to by its arguments.

```
// function definition to swap the values.
void swap(int &x, int &y)
{
int temp;
temp = x; /* save the value at address x */
x = y; /* put y into x */
y = temp; /* put x into y */
return;
}
```

For now, let us call the function swap() by passing values by reference as in the following example:

```
#include <iostream>
using namespace std;
// function declaration
void swap(int &x, int &y);
int main ()
{
// local variable declaration:
int a = 100;
```

```
int b = 200;
cout << "Before swap, the value of a:" << a << endl;
cout << "Before swap, the value of b:" << b << endl;
/* calling a function to swap the values using variable
reference.*/
swap(a, b);
cout << "After the swap, the value of a:" << a << endl;
cout << "After the swap, the value of b:" << b << endl;
return 0;
}
```

When the above code is put together in a file, compiled, and executed, it produces the following result:

```
Before swap, the value of a:100
Before swap, the value of b:200
After the swap, the value of a:200
After the swap, the value of b:100
```

By default, C++ uses call by value to pass arguments. In general, this means that code within a function cannot alter the arguments used to call the function, and the above-mentioned example while calling max() function used the same method.

7.3. Scope of variables

A variable has scope, which determines how long it is available to your program and where it can be accessed. There are two scopes Local and Global.

7.3.1. Local Variables

Not only can you pass in variables to the function, but you also can declare variables within the body of the function. This is done using local variables, so named because they exist only locally within the function itself. When the function returns, the local variables are no longer available. Local variables are defined as any other variables. The parameters passed into the function are also considered local variables and can be used exactly as if they had

been defined within the body of the function. Variables declared within the function are said to have "local scope." That means that they are visible and usable only within the function in which they are defined. In fact, in C++ you can define variables anywhere within the function, not just at its top. The scope of the variable is the block in which it is defined. Thus, if you define a variable inside a set of braces within the function, that variable is available only within that block.

Example

```
1:      #include <iostream.h>
2:
3:      float Convert(float);
4:      int main()
5:      {
6:         float TempFer;
7:         float TempCel;
8:
9:         cout << "Please enter the temperature in
Fahrenheit: ";
10:        cin >> TempFer;
11:        TempCel = Convert(TempFer);
12:        cout << "\nHere's the temperature in Celsius: ";
13:        cout << TempCel << endl;
14:                return 0;
15:     }
16:
17:     float Convert(float TFer)
18:     {
19:        float TCel;
20:        TCel = ((TFer - 32) * 5) / 9;
21:        return TCel;
22: }
```

Output:

```
Please enter the temperature in Fahrenheit: 212
```

```
Here's the temperature in Celsius: 100
Please enter the temperature in Fahrenheit: 32
Here's the temperature in Celsius: 0
Please enter the temperature in Fahrenheit: 85
Here's the temperature in Celsius: 29.4444
```

7.3.2. Global Variables

Global variables have global scope and are available anywhere within your program. Variables defined outside of any function have global scope and thus are available from any function in the program, including main(). Local variables with the same name as global variables do not change the global variables. A local variable with the same name as a global variable hides the global variable, however. If a function has a variable with the same name as a global variable, the name refers to the local variable--not the global--when used within the function.

```
1:   #include <iostream.h>
2:   void myFunction();           // prototype
3:   int x = 5, y = 7;            // global variables
5:   int main()
6:   {
7:        cout << "x from main: " << x << "\n";
9:        cout << "y from main: " << y << "\n\n";
10:       myFunction();
11:       cout << "Back from myFunction!\n\n";
12:       cout << "x from main: " << x << "\n";
13:       cout << "y from main: " << y << "\n";
14:       return 0;
15:  }
16:  void myFunction()
18:  {
19:       int y = 10;
20:       cout << "x from myFunction: " << x << "\n";
22:       cout << "y from myFunction: " << y << "\n\n";
23: }
```

```
Output: x from main: 5
y from main: 7
x from myFunction: 5
y from myFunction: 10
Back from myFunction!
x from main: 5
y from main: 7

#include <iostream.h>
void myFunc();
int main() {
int x = 5;
cout << "\nIn the main x is: " << x;
myFunc();
cout << "\nBack in main, x is: " << x;
return 0;
}
void myFunc()
{
int x = 8;
cout << "\nIn myFunc, local x: " << x << endl;
{
cout << "\nIn block in myFunc, x is: " << x;
int x = 9;
cout << "\nVery local x: " << x;
}
cout << "\nOut of block, in myFunc, x: " << x << endl;
}
```

Output:

```
In the main x is: 5
In myFunc, local x: 8
In block in myFunc, x is: 8
```

```
Very local x: 9
Out of block, in myFunc, x: 8
Back in main, x is: 5
```

7.4. Return Values

Functions return a value or return void. A void is a signal to the compiler that no value will be returned. To return a value from a function, write the keyword return followed by the value you want to return. The value might itself be an expression that returns a value. For example:

```
return 5;
return (x > 5);
return (MyFunction());
```

These are all legal return statements, assuming that the function *`MyFunction()`* itself returns a value. The value in the second statement, *`return (x > 5)`*, will be zero if x is not greater than 5, or it will be 1. What is returned is the value of the expression, 0 (false) or 1 (true), not the value of x.

When the return keyword is encountered, the expression following return is returned as the value of the function. Program execution returns immediately to the calling function, and any statements following the return are not executed. It is legal to have more than one return statement in a single function.

Example:A demonstration of multiple return statements.

```
#include <iostream.h>
int Doubler(int AmountToDouble);
int main(){
int result = 0;
int input;
cout << "Enter a number between 0 and 10,000 to double: ";
cin >> input;
cout << "\nBefore doubler is called... ";
cout<< "\ninput: " << input << " doubled: " <<result<<"\n";
```

```
result = Doubler(input);
cout << "\nBack from Doubler...\n";
cout<< "\ninput: " << input << "  doubled: "
<<result<<"\n";
return 0;
}
int Doubler(int original){
if (original <= 10000)
return original * 2;
else
return -1;
cout << "You can't get here!\n";
}
```

Output:

```
Enter a number between 0 and 10,000 to double: 9000
Before doubler is called...
input: 9000 doubled: 0
Back from doubler...
input: 9000   doubled: 18000
Enter a number between 0 and 10,000 to double: 11000
Before doubler is called...
input: 11000 doubled: 0
Back from doubler...
input: 11000  doubled: -1
```

7.5. Default Values for Parameters:

When you define a function, you can specify a default value for `*for*` each of the last parameters. This value will be used if the corresponding argument is left blank when calling the function.

This is done by using the assignment operator and assigning values for the arguments in the function definition. If a value for that parameter is not passed when the function is

called, the default given value is used, but if a value is specified this default value is ignored and the passed value is used instead. Consider the following example:

```
#include <iostream>
using namespace std;
int sum(int a, int b=20)
{
int result;
result = a + b;
return (result);
}
int main ()
{
// local variable declaration:
int a = 100;
int b = 200;
int result;
// calling a function to add the values.
result = sum(a, b);
cout << "The total value is:" << result << endl;
// calling a function again as follows.
result = sum(a);
cout << "The total value is:" << result << endl;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
The total value is:300
The total value is:120
```

7.6. Function Overloading

Unlike C, C++ lets you have more than one function with the same name. Functions with the same name are called overloaded functions. C++ requires that each overloaded function differs in its argument list. Overloaded functions enable you to have similar functions that

work on different types of data. Suppose that you wrote a function that returned the absolute value

of whatever number you passed to it:

```
int iabs(int i)
{ if(i<0)
 Return (i*-1);
 else
 Return (i);
}
float fabs(float x)
{ if(x<0.0)
 Return (x * -1.0);
 else
 Return (x);
}
```

Without using overloading, you have to call the function as:

```
int ans = iabs(weight);//for int arguments
float ans = fabs(weight);//for float arguments
```

But with overloading, the above code can be used as:

```
int abs(int i);
float abs(float x);
int main()
{
…
ians = abs(i);//calling abs with int arguments
fans = abs(p);//calling abs with float arguments
…
}
int abs(int i)
{
 if(i<0)
 Return i*-1;
```

```
 else
 Return I;
}
float abs(flaot x)
{
 if(x<0.0)
 Return x*-1.0;
 else
 Return x;
}
```

N.B: if two or more functions differ only in their return types, C++ can't overload them. Two or more functions that differ only in their return types must have different names and can't be overloaded.

Constant parameters can be pass to functions using the 'const' keyword. When a parameter or reference is const, it cannot be changed inside the function

Note that we cannot pass a const parameter to a non-const formal parameter. But we can pass const and non-const parameters to a const formal parameter.

Similarly, we can also have const return-type. In this case, also, the return type cannot be modified.

Example:

```
#include <iostream>
using namespace std;
int addition(const int &a, const int &b){
   return (a+b);
 }
int main()
 {
   int a,b;
   cout<<"Enter the two numbers to be swapped: ";
   cin>>a>>b;
   cout<<"a = "<<a;
   cout<<"\tb = "<<b;
   int res = addition(a,b);
```

```
   cout<<"\nResult of addition: "<<res;
}
```

When the above code is put together in a file, compiled, and executed, it produces the following result:

```
Enter the two numbers to be swapped: 22 33
a = 22 b = 33
Result of addition: 55
```

In the above program, we have const formal parameters. Note that the actual parameters are ordinary non-const variables that we have successfully passed. As formal parameters are const, we cannot modify them inside the function. So we just perform the addition operation and return the value.

7.7. Recursion

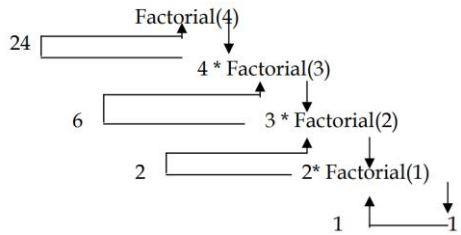
A function that calls itself is said to be recursive. Recursion is a general programming technique applicable to problems that can be defined in terms of themselves. Take the factorial problem, for instance, which is defined as:

- factorial of 0 is 1
- factorial of a positive number n is n time the factorial of n1.

The second line indicates that factorial is defined in terms of itself and hence can be expressed as a recursive function.

```
int Factorial(unsigned int n )
{
 return n = = 0 ? 1 : n * factrial(n-1);
}
```

For n set to 4, the following **Error! Reference source not found.** shows the recursive call:



The stack frames for these calls appear sequentially on the runtime stack, one after the other. A recursive function must have at least one termination condition which can be satisfied. Otherwise, the function will call itself indefinitely until the runtime stack overflows. The three necessary components in a recursive method are:

1. A test to stop or continue the recursion
2. An end case that terminates the recursion
3. A recursive call(s) that continues the recursion

Let us implement two more mathematical functions using recursion

e.g the following function computes the sum of the first N positive integers 1,2,…, N. Notice how the function includes the three necessary components of a recursive method.

```
int sum(int N)
{
 if(N==1)
 return 1;
 else
 return N+sum(N);
}
```

The last method computes the exponentiation a where A is a real number and N is a positive integer. This time, we have to pass two arguments. A and N. the value of A will not change in the calls, but the value of N is decremented after each recursive call.

```
float expo(float A, int N)
{
 if(N==1)
 return A;
```

```
 else
 return A * expo(A,N-1);
}
```

Try to use a recursive function call to solve the Fibonacci series. The Fibonacci series is :

```
0,1,1,2,3,5,8,13,21,…
```

The recursive definition of the Fibonacci series is as follows:

```
Fibonacci (0) =0
Fibonacci (1) =1
Fibonacci (n) =Fibonacci (n-1) +Fibonacci (n-2);
```

7.8. Recursion versus iteration

Both iteration and recursion are based on a control structure. Iteration uses a repetition structure (such as for, while, do…while) and recursive uses a selection structure (if, if-else, or switch).

Both iteration and recursive can execute infinitely-an infinite loop occurs with iteration if the loop continuation test becomes false and infinite recursion occurs in the recursion step doesn't reduce the problem in a manner that coverage on a base case.

Recursion has a disadvantage as well. It repeatedly invokes the mechanism, and consequently the overhead of method calls. This can be costly in both processor time and memory space. Each recursive call creates another copy of the method (actually, only the function's variables); this consumes considerable memory.

N.B: Use recursion if:

1. A recursive solution is natural and easy to understand
2. A recursive solution doesn't result in excessive duplicate computation.
3. the equivalent iterative solution is too complex and
4. of course, when you are asked to use one in the exam!!

Example 1: A program to find the sum of the given three numbers using function declaration with the *return* statement.

```
#include <iostream>
#include <iostream>
using namespace std;
float sum(float a, float b, float c){
      float sum;
      sum=a+b+c;
      return(sum);
}
int main ()
{
      float x,y,z;
      cout<<"Enter any three numbers: \n";
      cin>>x>>y>>z;
      float total;
      total =sum(x,y,z);
      cout<<"Sum="<<total<<endl;
      return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter three numbers
4     6     9
Sum=19
```

Example 2: A program to find the factorial of a given number function declaration

```
#include <iostream>
using namespace std;
int main ()
{
      int fact(int);
      int x,n;
      cout<<"Enter any integer number \n"<<endl;
```

```
	cin>>n;
	x=fact(n);
	cout<<"Value="<<n<<" and its factorial="<<x<<endl;
	return 0;
}
int fact(int a){
	int value=1;
	if (a==1){
		return (value);
	}
	else{
		for(int i=1; i<=a; i++){
			value=value*i;
		}
		return (value);
	}
}
```

When the above code is compiled and executed, it produces the following result:

```
Enter any integer number
6
Value= 6 and its factorial=720
```

Lab

1. Given the following definition of a Swap function

```
void Swap (int x, int y)
{
	int temp = x;
	x = y;
	y = temp;
}
```

what will be the value of x and y after the following call?

```
x = 10;
y = 20;
Swap(x, y);
```

2. Write a C++ program that accepts 10 integers from the user and finally displays the smallest value and the largest value. Hint using the function.
3. Write a program in C++ that reads a three-digit number and generates all the possible permutation of numbers using the above digits. For example, n = 123 then the permutations are:

```
123, 213, 312, 132, 231, 321
```

4. Write an int function cube () that returns the cube of its single int formal parameter.
5. Write a float function triangle() that computes the area of a triangle using its two formal parameters h and w, where h is the height and w is the length of the bases of the triangle.
6. Write a float function rectangle() that computes and returns the area of a rectangle using its two float formal parameters h and w, where h is the height and w is the width of the rectangle.
7. Write a program that accepts a positive integer from the user and displays the factorial of the given number. You should use a recursive function called factorial() to calculate the factorial of the number.
8. Write a function called isEven() that uses the remainder operator(%) to determine whether an integer is even or not.
9. Create simple calculator using four functions sum(),product(),quotient() and difference().

Exercise

1. Write a function which outputs all the prime numbers between 2 and a given positive integer n: A number is prime if it is only divisible by itself and 1.

```
void Primes (unsigned int n);
```

2. Write a function in C++ to generate a Fibonacci series of n numbers, where n is defined by a programmer.

   (the series should be: 1 1 2 3 5 8 12 21 32, etc)

3. The formula for a line is normally given as y = mx + b. Write a function Line() that expects three float parameters, a slop m, a y-intercept b, and an x-coordinate x. the function computes the y-coordinate associated with the line specified by m and b at x-coordinate.
4. Write a function Intersect() with four float parameters m1,b1,m2,b2. The parameters come conceptually in two pairs. The first pair contains the coefficients describing one line; the second pair contains coefficients describing a second line. The function returns 1 if the two lines intersect. Otherwise, it should return 0;
5. Write another program that accepts a number from the user and returns the Fibonacci value of that number. You should use recursion here.
6. Develop a program that uses the function factorial() of exercise 6 to compute an approximation of e (Euler's number). Base your approximation on the following formula for e: 1 + 1/1! + 1/2! + 1/3! + …
7. Write a function called isPrime() that accepts a number and determine whether the number is prime or not.
8. The distance D between two points with coordinates (x1, y1) and (x2, y2) on a plane is given by
9. Write a C++ program that calculates the letter grades of 20 students. The program should accept the mid result and the final result from the students. Use the appropriate validity control mechanism to prevent wrong inputs.
10. Write a C++ program that has two functions toBinary and toDecimal. The program should display a menu prompting the user to enter his choice. If the user selects toBinary, then the function should accept a number in base ten and displays the equivalent binary representation. The reverse should be done if the user selects toDecimal.

References

[1] D. Ravichandran, Programming with C++ (3rd ed, Tata McGraw-Hill Education, 2011

[2] Ulla Kirch-Prinz, Peter Prinz, A Complete Guide to Programming in C++, Jones and Bartlett publishers, 2002

Chapter Eight

8. Array and String

In the C++ programming language, there are different data structures used to store similar items. The array is one of those structures. There is also a string data type that stores string literals. Students need to learn these data structures as part of a C++ program learning.

At the end of this chapter, students should be able to:

- *list characteristics of an array in C++*
- *declare arrays of different type and size*
- *initialize a declared array to certain values*
- *access values of an array's elements*
- *create a two-dimensional array of different types*
- *access values of a two-dimensional array*
- *use arrays as parameters of functions*
- *declare and initialize a string*
- *write a C++ program that reads a string from the keyboard*
- *convert a string to other data types*

8.1. Introduction to Array

C++ has a data structure, called an array, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as *numbers* and use *numbers*[0], *numbers*[1], and ..., *numbers*[99] to represent individual variables. A specific element in an array is accessed by specifying the respective index of the element in the array.

All arrays consist of contiguous memory locations where the lowest address corresponds to the first element and the highest address corresponds to the last element.

Properties of arrays

- Arrays in C++ are zero-bounded; that is, the index of the first element in the array is 0 and the last element is N-1, where N is the size of the array.
- In a C++ program, it is not valid to refer to an element outside of the array bounds; otherwise, the program will crash or have unexpected results depending on the compiler.
- The array can only hold values of one type.

8.1.1. Declaring arrays

Declaring the name and type of an array and setting the number of elements in an array is called dimensioning the array. Like any other variables in C++, an array must be declared before being used. The following three aspects must be specified when declaring an array.

- The type of the array (i.e. integer, floating-point, char, etc.)
- Name of the array
- The total number of memory locations to be allocated or the maximum value of each subscript. i.e. the number of elements in the array

So, the general syntax for the declaration is:

```
type arrayName [arraySize];
```

This is called a single-dimension array. The *arraySize* must be an integer constant greater than zero and the type can be any valid C++ data type. For example, to declare a 10-element array called the balance of type double, use this statement:

```
double balance [10];
```

As another example, an integer with the size of 10 having a name of num could be declared as:

```
int num [10];
```

The above declaration can be interpreted as ten consecutive two-byte memory locations will be reserved with the name num and we can store 10 values of type *int* without having to declare 10 different variables each one with a different identifier. In other words, using an array we can store 10 different values of the same type, int for example, with a unique identifier.

Note: array size cannot be a variable whose value is set while the program is running.

```
double balance [10];
```

8.1.2. Initializing arrays

- When declaring an array of local scope (within a function), if we do not specify the elements of the array, the array variable will not be initialized to any value by default, so its content is undetermined until we store some values in it.
- If we declare a global array (outside any function), its content will be initialized with all its elements filled with zeros. Thus, if we declare the following in the global scope,

```
int day [5];
```

- Every element of the day will be set initially to 0:

	0	1	2	3	4
Day	0	0	0	0	0

You can initialize C++ array elements either one by one or using a single statement as follows:

```
double balance [5] = {1000.0, 2.0, 3.4, 17.0, 50.0};
```

The number of values between braces {} cannot be larger than the number of elements that we declare for the array between square brackets []. Following is an example to assign a single element of the array:

If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write:

```
double balance [] = {1000.0, 2.0, 3.4, 17.0, 50.0};
```

You will create the same array as you did in the previous example.

Note: The number of elements in the array that we initialized within curly brackets { } must be equal or less than the length in elements that we declared for the array enclosed within square brackets [ ]. If we have a smaller number of items for the initialization, the rest will be filled with zeros.

```
balance [4] = 50.0;
```

The above statement assigns element number 5th in the array a value of 50.0. Array with 4th index will be 5th, i.e., the last element because all arrays have 0 as the index of their first element which is also called base index.

Because this can be considered as a useless repetition, C++ allows the possibility of leaving empty the brackets [ ], where the number of items in the initialization bracket will be counted to set the size of the array.

```
int day [] = { 1, 2, 7, 4, 12,9 };
```

The compiler will count the number of initialization items which is 6 and set the size of the array day to 6 (i.e.: day [6])

You can use the initialization form only when defining the array. You cannot use it later, and cannot assign one array to another once. Let me have a look at the following example to understand this better.

```
int arr [] = {16, 2, 77, 40, 12071};
int ar [4];
ar[]={1,2,3,4};//not allowed
arr=ar;//not allowed
```

Note: when initializing an array, we can provide fewer values than the array elements.

E.g.,

```
int a [10] = {10, 2, 3};
```

In this case the compiler sets the remaining elements to zero.

8.2. Accessing and processing array elements

The items in an array are called elements. As we noted, all the elements in an array are of the same type; only the values vary. Array elements are counted from zero indexes.

At any point of the program in which the array is visible, we can access individually anyone of its elements for reading or modifying it as if it was a normal variable. To access individual elements, an index or subscript is used. The format is the following:

```
name [index];
```

In C++ the first element has an index of 0 and the last element has an index, which is one less the size of the array (i.e. array size-1). Thus, from the above declaration, day[0] is the first element, and day[4] is the last element.

Following the previous examples where the day had 5 elements and each element is of type int, the name, which we can use to refer to each element, is the following one:

	Day[0]	Day[1]	Day[2]	Day[3]	Day[4]
Day	0	0	0	0	0

For example, to store the value 75 in the third element of the array variable day a suitable sentence would be:

```
day[2] = 75; //as the third element is found at index 2
```

	Day[0]	Day[1]	Day[2]	Day[3]	Day[4]
Day	0	0	75	0	0

And, for example, to pass the value of the third element of the array variable day to the variable a, we could write:

```
a = day[2];
```

Therefore, for all the effects, the expression day[2] is like any variable of type int with same properties. Thus, an array declaration enables us to create a lot of variables of the same type with a single declaration and we can use an index to identify individual elements.

Notice that the third element of the day is specified day[2], since first is the day[0], second day[1], and therefore, third is the day[2]. For this same reason, its last element is the day [4]. If we wrote day [5], we would be acceding to the sixth element of the day and, therefore, exceeding the size of the array. This might give you either error or an unexpected value depending on the compiler.

In C++, it is perfectly valid to exceed the valid range of indices for an Array, which can cause certain detectable problems since they do not cause compilation errors but they can cause unexpected results or serious errors during execution. At this point, it is important to be able to clearly distinguish between the two uses the square brackets [ ] have for arrays. One is to set the size of arrays during declaration. The other is to specify indices for a specific array element when accessing the elements of the array. One must take care of not confusing these two possible uses of brackets [ ] with arrays.

E.g.:

```
int day[5]; // declaration of a new Array (begins with a
type name)
day[2] = 75; // access to an element of the Array.
```

Other valid operations with arrays in accessing and assigning:

```
int a=1;
day [0] = a;
day[a] = 5;
b = day [a+2];
day [day[a]] = day [2] + 5;
```

Eg: Arrays example, display the sum of the numbers in the array

```
#include <iostream.h>
int day [ ] = {16, 2, 77, 40, 12071};
int n, result=0;
void main ()
{
for ( n=0 ; n<5 ; n++ )
{
```

```
result += day[n];
}
cout << result;
getch(); }
```

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example:

```
double salary = balance [9];
```

The above statement will take 10th element from the array and assign the value to salary variable. Following is an example, which will use all the above-mentioned three concepts viz. declaration, assignment and accessing arrays:

```
#include <iostream>
using namespace std;
#include <iomanip>
using std::setw;
int main () {
int n[ 10 ]; // n is an array of 10 integers
// initialize elements of array n to 0
for ( int i = 0; i < 10; i++ )  {
n[ i ] = i + 100; // set element at location i
to i + 100
}
cout << "Element" << setw( 13 ) << "Value" <<
endl;
// output each array element's value
for ( int j = 0; j < 10; j++ )  {
cout << setw( 7 )<< j << setw( 13 ) << n[ j ] << endl;
}
return 0;
}
```

Element	Value
0	100
1	101
2	102
3	103
4	104
5	105
6	106
7	107
8	108
9	109

This program makes use of the setw() function to format the output. When the above code is compiled and executed, it produces the right-side list results:

Arrays are important to C++ and should need lots of more detail. There are following few important concepts, which should be clear to a C++ programmer:

8.3. Two-dimensional arrays

The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, in essence, a list of one-dimensional arrays. To declare a two-dimensional integer array of size x,y, you would write something as follows:

```
type arrayName [x][y];
```

Where type can be any valid C++ data type and arrayName will be a valid C++ identifier.

A two-dimensional array can be thinking as a table, which will have x number of rows and y number of columns. A 2-dimensional array a, which contains three rows and four columns can be shown as below:

Table 18: Two-dimensional array

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Thus, every element in array a is identified by an element name of the form a[i][j], where a is the name of the array, and i and j are the subscripts that uniquely identify each element in a. Multidimensional arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row have 4 columns.

```
int a[3][4] = {
{0, 1, 2, 3} , /* initializers for row indexed by 0 */
{4, 5, 6, 7} , /* initializers for row indexed by 1 */
{8, 9, 10, 11} /* initializers for row indexed by 2 */
};
```

The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to the previous example:

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

8.3.1. Accessing two-dimensional array elements

An element in the 2-dimensional array is accessed by using the subscripts ie. row index and column index of the array. For example:

```
int value = a[2][3];
```

The above statement will take the 4th element from the 3rd row of the array. You can verify it in the above diagram.

```
#include <iostream>
using namespace std;
int main ()
{
// an array with 5 rows and 2 columns.
int a [5][2] = {{0,0}, {1,2}, {2,4}, {3,6},
{4,8}};
// output each array element's value
for (int i = 0; i < 5; i++)
for (int j = 0; j < 2; j++)
{
     cout << "a [" << i << "][" << j << "]: ";
     cout << a[i][j] << endl;
}
return 0;
}
```

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

As explained above, you can have arrays with any number of dimensions although it is likely that most of the arrays you create will be of one or two dimensions.

Example 1:

```
#include <iostream.h>
float data[5]; // data to average and total
float total; // the total of the data items
float average; // average of the items
main ()
{
```

```
data[0] = 34.0;
data[1] = 27.0;
data[2] = 46.5;
data[3] = 82.0;
data[4] = 22.0;
total = data[0] + data[1] + data[2] + data[3] + data[4];
average = total / 5.0;
cout << "Total "<< total << " Average " << average << '\n';
return (0);
}
```

This program outputs:

```
Total 211.5 Average 42.3
```

Example 2: Write a C++ program that adds two initialized 3×4 matrices A and B and then stores the result in a matrix C.

```
#include <iostream.h>
using namespace std;
int main()
{
int A[3][4] = { {1, 4, 3, 2}, {5, 6, 7, 8}, {9, 10, 11, 12}
};
int B[3][4] = { {3, 4, 3, 1}, {8, 7, 5, 6}, {12, 9, 11, 8}
};
int C[3][4];
for (int i=0; i<3; i++)
{
for (int j=0; j<4; j++)
{
C[i][j] = A[i][j] + B[i][j];
cout << C[i][j] << "\t";
}
cout << endl;
}
```

```
return 0;
}
```

C++ allows multidimensional arrays. Here is the general form of a multidimensional array declaration:

```
type name[size1] [size2] ...[sizeN];
```

For example, the following declaration creates a three dimensional 5 . 10 . 4 integer array:

```
int threedim[5][10][4];
```

Multidimensional arrays are nothing else than an abstraction since we can simply obtain the same results with a simple array by putting a factor between its indices:

```
int matrix [3][5]; is equivalent to
int matrix [15]; (3 * 5 = 15)
```

The only difference that with multidimensional arrays the compiler remembers the depth of each imaginary dimension for us. Take for an example these two pieces of code, with both the same result. One uses a bi-dimensional array and the other one uses a simple array:

multidimensional-array

```
#include <iostream.h>
#define WIDTH 5
#define HEIGHT 3
int matrix
[HEIGHT][WIDTH];
int n,m;
int main ()
{
for (n=0;n<HEIGHT;n++)
for (m=0;m<WIDTH;m++)
{
matrix
[n][m]=(n+1)*(m+1);
}
```

Pseudo-multidimensional array

```
#include <iostream.h>
#define WIDTH 5
#define HEIGHT 3
int matrix [HEIGHT][WIDTH];
int n,m;
int main ()
{
for (n=0;n<HEIGHT;n++)
for (m=0;m<WIDTH;m++)
{
matrix
[n*WIDITH+m]=(n+1)*(m+1);
}
return 0;
```

```
return 0;
}
```

```
}
```

Few Things to Remember:

The array indices start with 0. Meaning x[0] is the first element stored at index 0. If the size of an array is n, the last element is stored at index (n-1). In this example, x[5] is the last element.

Elements of an array have consecutive addresses. For example, suppose the starting address of x[0] is 2120d. Then, the address of the next element x[1] will be 2124d, the address of x[2] will be 2128d, and so on.

Here, the size of each element is increased by 4. This is because the size of int is 4 bytes.

Example-: A program to manipulate a single dimensional array

```
#include<iostream>
using namespace std;
int main()
{ int mark[5] = {19, 10, 8, 17, 9};
// change 4th element to 9
 mark[3] = 9;
// take input from the user
// store the value at third position
cout<<"enter the value to be entered at the third
position\n";
cin>>mark[2];
int i;
// take input from the user
// insert at ith position
cout<<"at what position to insert from first to fifth\n ";
cin>>i;
cout<<"enter the value:\n";
```

```
cin>>mark[i-1];
// print first element of the array
cout<<mark[0]<<"\n";
// print ith element of the array
cout<<mark[i-1];
return 0;
}
```

Output:

```
enter the value to be entered at the third position
30
at what position to insert from first to fifth
 5
enter the value:
50
19
50
```

Example-2 A program to Printing array elements

```
#include <iostream>
using namespace std;
int main() {
    int numbers[5] = {7, 5, 6, 12, 35};
    cout << "The numbers are: ";
    //  Printing array elements
    // using range based for loop
    for (const int &n : numbers) {
        cout << n << "  ";
    }
    cout << "\nThe numbers are: ";
    //  Printing array elements
    for (int i = 0; i < 5; ++i) {
        cout << numbers[i] << "  ";
    }
    return 0;
```

```
}
```

Output

```
The numbers are: 7  5  6  12  35
The numbers are: 7  5  6  12  35
```

Example-3 A program to store input from user to array ang display elements.

```
#include <iostream>
using namespace std;
int main() {
    int numbers[5];
    cout << "Enter 5 numbers: " << endl;
    //  store input from user to array
    for (int i = 0; i < 5; ++i) {
        cin >> numbers[i];
    }
    cout << "The numbers are: ";
    //  print array elements
    for (int n = 0; n < 5; ++n) {
        cout << numbers[n] << "  ";
    }
    return 0;
}
```

Output

```
Enter 5 numbers:
11
12
13
14
15
The numbers are: 11  12  13  14  15
```

Once again, we have used a for loop to iterate from i = 0 to i = 4. In each iteration, we took input from the user and stored it in numbers[i]. Then, we used another for loop to print all the array elements.

Example-4: A program to initialize an array element without specifying size, print array elements, calculate the sum and average of elements and display sum and average

```
#include <iostream>
using namespace std;
int main() {
    // initialize an array without specifying size
    double numbers[] = {7, 5, 6, 12, 35, 27};
    double sum = 0;
    double count = 0;
    double average;
    cout << "The numbers are: ";
    //  print array elements
    // use of range-based for loop
    for (const double &n : numbers) {
        cout << n << "  ";
        //  calculate the sum
        sum += n;
        // count the no. of array elements
        ++count;
    }
    // print the sum
    cout << "\nTheir Sum = " << sum << endl;
    // find the average
    average = sum / count;
    cout << "Their Average = " << average << endl;
    return 0;
}
```

Output

```
The numbers are: 7  5  6  12  35  27
```

```
Their Sum = 92
Their Average = 15.3333
```

In this program:

- We have initialized a double array named numbers but without specifying its size. We also declared three double variables sum, count, and average. Here, sum =0 and count = 0.
- Then we used a range-based for loop to print the array elements. In each iteration of the loop, we add the current array element to the sum.
- We also increase the value of count by 1 in each iteration, so that we can get the size of the array by the end of the for a loop.
- After printing all the elements, we print the sum and the average of all the numbers. The average of the numbers is given by average = sum/count;

Note: If we declare an array of size 10, then the array will contain elements from index 0 to 9. However, if we try to access the element at index 10 or more than 10, it will result in Undefined Behavior.

Example-5 C++ Program to display all elements of an initialized two-dimensional array

```
#include <iostream>
using namespace std;
int main() {
    int test[3][2] = {{2, -5},{4, 0},{9, 1}};
    // use of nested for loop
    // access rows of the array
    for (int i = 0; i < 3; ++i) {
        // access columns of the array
        for (int j = 0; j < 2; ++j) {
            cout << "test[" << i << "][" << j << "] = " <<
test[i][j] << endl;
        }
    }
    return 0;
```

```
}
```

Output

```
test[0][0] = 2
test[0][1] = -5
test[1][0] = 4
test[1][1] = 0
test[2][0] = 9
test[2][1] = 1
```

In the above example, we have initialized a two-dimensional int array named test that has 3 "rows" and 2 "columns". Here, we have used the nested for loop to display the array elements.

- The outer loop from i = 0 to i = 2 access the rows of the array
- The inner loop from j = 0 to j = 1 access the columns of the array

Finally, we print the array elements in each iteration.

Example-6 A C++ Program to display elements of a two-dimensional array entered by the user.

```
#include <iostream>
using namespace std;
int main() {
    int numbers[2][3];
    cout << "Enter 6 numbers: " << endl;
    // Storing user input in the array
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            cin >> numbers[i][j];
        }
    }
    cout << "The numbers are: " << endl;
    // Printing array elements
    for (int i = 0; i < 2; ++i) {
```

```
        for (int j = 0; j < 3; ++j) {
            cout << "numbers [" << i << "][" << j << "]: "
<< numbers[i][j] << endl;
        }
    }
    return 0;
}
Output
Enter 6 numbers:
1
2
3
4
5
6
The numbers are:
numbers[0][0]: 1
numbers[0][1]: 2
numbers[0][2]: 3
numbers[1][0]: 4
numbers[1][1]: 5
numbers[1][2]: 6
```

Programming examples-B

1. Write C++ program that loads an integer array with the numbers 0 through 9 and display the array values.

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{int a[10];
for (int i=0;i<10;i++)
a[i] =i;
cout<<"Array is:"<<endl;
```

```
for (int i=0;i<10;i++)
cout<<a[i] <<" ";
getch ();
return 0;
}
```

Output:

```
Array is:
0 1 2 3 4 5 6 7 8 9
```

2. Write C++ program that read 10 numbers and print their sum, average, maximum and minimum.

```
#include<iostream.h>
#include<conio.h>
using namespace std;
int main()
{
const int n=10;
int min, max, sum;
float average;
int numbers[n];
cout<<"enter the 10 numbers \n";
for(int i=0;i<n;i++)
cin>>numbers[i];
min=max=sum=numbers[0]; /* the minimum equals the maximum
equals the summation
equals the first number at the start */
for(int i=1;i<n;i++) // we consider starting from the next
element in the array
{
if(numbers[i]<min)
min=numbers[i];
if(numbers[i]>max)
max=numbers[i];
sum+=numbers[i];
```

```
}
average=((float) sum)/n;
cout<<"sum =" <<sum<<", average= "<< average<<"\n";
cout<<" maximum =" <<max<<", minimum= "<<min;
getch ();
return 0;
}
```

Output:

```
enter the 10 numbers
2
5
7
5
8
9
11
5
6
10
sum =68, average= 6.8
maximum =11, minimum= 2
```

3. Write a C++ program that computes the number of even integer numbers in an array entered by the user.

```
#include <iostream.h>
#include<conio.h>
int main()
{
const int size = 10;
int a[size] , count = 0;
cout<<"Enter ten integer numbers: ";
for(int i=0 ; i<10 ; i++)
{
cin >> a[i];
```

```
if(a[i] % 2 == 0)
count++;
}
cout<<"The number of even numbers is " << count;
getch ();
return 0;
}
```

Output:

```
Enter ten integer numbers:
1
2
3
4
5
6
7
8
9
10
The number of even numbers is 5
```

4. Write a C++ program that inputs an integer array a[10] and arranges it in an ascending order.

```
#include <iostream.h>
#include<conio.h>
int main()
{ const int size = 10;
int a[size];
cout<<"Enter ten integer array values: ";
for (int i=0; i<size; i++)
cin>>a[i];
for (i=0; i<size-1; i++)
for(int j=i+1; j<size; j++)
if(a[i] > a[j])
```

```
{
int temp = a[i];
a[i] = a[j];
a[j] = temp;
}
cout << "Array in ascending order: " <<endl;
for(i=0; i<size ;i++)
cout<<a[i]<<" ";
getch ();
return 0;
}
```

Output:

```
Enter ten integer array values: 4
2
8
7
6
3
2
44
12
45
Array in ascending order:
 2  3  4  6  7  8  12  44  45
```

5. Write a program that accepts elements of a maximum 3x3 matrix from the keyboard and display it.

```
#include<iostream.h>
#include<conio.h>
int main()
{int matrix[3][3];
int i, j;
cout<<”enter elements of the matrix\n”;
```

```
for(i=0;i<3;i++)
for(j=0;j<3;j++)
cin>>matrix[i][j];
cout<<"The matrix is:\n";
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
cout<<matrix[i][j]<<” “;
cout<<endl;
}
getch ();
return 0;
}
Output:
enter elements of the matrix
1
2
3
4
5
6
7
8
9
The matrix is:
1   2   3
4   5   6
7   8   9
```

6. Write a program that performs matrix addition using 3x3 matrixes.

```
#include<iostream.h>
#include<conio.h>
int main()
{int A[3][3];
int B[3][3]; //declaration
```

```
int C[3][3];
int i, j;
cout<<”enter elements of the first matrix\n”;
for(i=0;i<3;i++)
for(j=0;j<3;j++)
cin>>A[i][j]; //accepting inputs
cout<<”enter elements of the second matrix\n”;
for(i=0;i<3;i++)
for(j=0;j<3;j++)
cin>>B[i][j];
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
C[i][j]=A[i][j]+B[i][j]; //matrix addition
}
cout<<" The result of matrix addition is:\n";
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
cout<<C[i][j]<<” “; //display addition result
cout<<endl;
}
getch ();
return 0;
}
```

Output:

```
enter elements of the first matrix
1
2
3
4
5
6
7
```

```
8
9
enter elements of the second matrix
8
5
4
2
7
11
10
12
5
 The result of the matrix addition is:
9  7  7
6  12  17
17  20  14
```

7. Write a C++ program that exchanges row3 with row1 in a 4× 4integer matrix input by the user.

```
#include <iostream.h>
int main()
{
int a[4][4];
cout<<"Enter 4x4 integer matrix: ";
for (int i=0; i<4; i++)
for (int j=0; j<4; j++)
cin>>a[i][j];
for (int i=0; i<4; i++)
{
int temp = a[0][i];
a[0][i] = a[2][i];
a[2][i] = temp;
}
cout<<"Matrix after exchanging row3 with row1:"<<endl;
```

```
for (int i=0; i<4; i++)
{
for (int j=0; j<4; j++)
cout<<a[i][j]<<" ";
cout<<endl;
}
return 0;
}
```

Output:

```
Enter 4x4 integer matrix: 1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
Matrix after exchanging row3 with row1:
9  10  11  12
5  6  7  8
1  2  3  4
13  14  15  16
```

8. Write a program that displays an identity matrix of 3x3.

```
#include<iostream.h>
#include<conio.h>
```

```
int matrix[3][3];
int main()
{
int i, j;
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
if(i==j)
matrix[i][j]=1;
}
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
cout<<matrix[i][j]<<" ";
cout<<endl;
}
getch ();
return 0;
}
```

Output:

```
1 0 0
0 1 0
0 0 1
```

9. Write a C++ program that multiplies a 3×4 matrix by the 4×3 matrix both are entered by the user. Then the program should store the result in a third matrix.

```
#include <iostream.h>
void main()
{
const int row_a=3 , col_a=4 , row_b=4, col_b=3;
int a[row_a][col_a];
int b[row_b][col_b];
int c[row_a][col_b];
```

```
cout<<"Enter "<<row_a<<"x"<<col_a<<" integer matrix: " <<
endl;
for (int i=0; i<row_a; i++)
for (int j=0; j<col_a; j++)
cin>>a[i][j];
cout<<"Enter "<<row_b<<"x"<<col_b<<" integer matrix: " <<
endl;
for (i=0; i<row_b; i++)
for (j=0; j<col_b; j++)
cin>>b[i][j];
for(i=0; i<row_a; i++)
for(j=0; j<col_b ; j++)
{
c[i][j] = 0;
for(int k=0; k<col_a ; k++)
c[i][j] += a[i][k] * b[k][j];
}
cout<<"Resulted Matrix is " << endl;
for (i=0; i<row_a; i++)
{
for (j=0; j<col_b; j++)
cout<<c[i][j]<<" ";
cout<<endl;
}
}
```

Output:

```
Enter 3x4 integer matrix:
1
2
3
4
5
6
7
```

```
8
9
10
11
12
Enter 4x3 integer matrix:
13
14
15
16
17
18
19
20
21
22
23
24
Resulted Matrix is
190  200  210
470  496  522
750  192  834
```

8.4. Arrays as parameters

At some moment we may need to pass an array to a function as a parameter. In C++, it is not possible to pass by value a complete block of memory as a parameter even if it is ordered as an array, to a function. Rather, it is allowed to pass its address, which has almost the same practical effect and is a much faster and more efficient operation.

To admit arrays as parameters the only thing that we must do when declaring the function is to specify in the argument the base type for the array that it contains, an identifier, and a pair of void brackets [ ]. For example, the following function:

```
void procedure (int arg[])
```

admits a parameter of type "Array of int " called arg. To pass to this function an array declared as:

```
int myarray [40];
```

it would be enough with a call like this:

```
procedure (myarray);
```

Here you have a complete example:

```
// arrays as parameters
#include <iostream.h>
#include <conio.h>
void print array (int arg[], int length)
{
for (int n=0; n<length; n++)
cout << arg[n] << " ";
cout << "\n";
}
void mult(int arg[], int length)
{
for (int n=0; n<length; n++)
arg[n]=2*arg[n];
}
void main ()
{
int firstarray[] = {5, 10, 15};
int secondarray[] = {2, 4, 6, 8, 10};
printarray (firstarray,3);
printarray (secondarray,5);
mult(firstarray,3);
cout<<”first array after being doubled is\n”;
printarray (firstarray,3);
getch()
}
```

As you can see, the first argument (int arg[]) admits any array of type int, whatever its length is, for that reason, we have included a second parameter that informs the function of the length of each array that we pass to it as the first parameter so that the for loop that prints out the array can have the information about the size we are interested about. The function *mult* doubles the value of each element and the first array is passed to it. After that, the display function is called. The output is modified showing that arrays are passed by reference. To pass an array by value, pass each element to the function

8.5. Introduction to Strings

8.5.1. What are the Strings?

In all programs and concepts, we have seen so far, we have used only numerical variables, used to express numbers exclusively. But, in addition to numerical variables, there exist also strings of characters that allow us to represent successive characters, like words, sentences, names, and texts. Until now, we have only used them as constants, but we have never considered variables able to contain them.

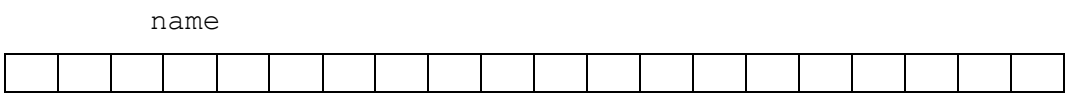
A string is a series of characters stored in consecutive bytes of memory. This implies that a string can be stored in an array of char, with each character kept in its array elements. Strings provide a convenient way to store text information such as messages to the user.

A string can also be defined as a character array that is terminated by a null. A null character is specified using '\0' and is zero. Because of the null terminator, it is necessary to declare a character array to be one character longer than the largest string that it will hold.

For example, if you want to declare an array name that could hold a 20-character string, you would write:

```
char name [20];
```

You may imagine it thus:



This maximum size of 20 characters is not required to be always fully used. For example, the name could store at some moment in a program either the string of characters "Hello" or the string "studying C++". Therefore, since the array of characters can store shorter strings than its total length, there has been reached a convention to end the valid content of a string with a null character, whose constant can be written as '\0'. We could represent name (an array of 20 elements of type char) storing the strings of characters "Hello" and "Studying C++" in the following way:

H	e	L	L	O	\0														

S	t	U	D	i	n	g		C	+	+	\0								

Notice how after the valid content it is included a null character ('\0') to indicate the end of the string. The empty cells (elements) represent indeterminate values.

8.5.2. Initialization of Strings

Since strings of characters are ordinary arrays, they fulfill the same rules as any array. For example, if we want to initialize a string of characters with predetermined values, we can do it in a similar way to any other array:

```
char mystring[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
```

In this case, we would have declared a string of characters (array) of 6 elements of type char initialized with the characters that compose Hello plus a null character '\0'. A string of characters can be initialized using constant strings. To initialize a string, follow the char array declaration with the equal character and a string of characters enclosed in the double quotes (“ ”). For example,

```
char promptMsg[] = “Enter a positive number: ”;
char errMsg[] = “Value must be positive. “;
```

prompting is stored in 25 locations, one for each character. Unlike single quotes (‘) which allow specifying single character constants, double quotes (“) are constants that specify a succession of characters. These strings enclosed between double quotes have always a null

character ('\0') automatically appended at the end. Therefore, we could initialize the string **mystring** with values by any of these two ways:

```
char mystring [] = { 'H', 'e', 'l', 'l', 'o', '\0' };
char mystring [] = "Hello";
```

In both cases the Array or string of characters mystring is declared with a size of 6 characters (elements of type char): the 5 characters that compose Hello plus a final null character ('\0') which specifies the end of the string and that, in the second case, when using double quotes (") it is automatically appended.

Before going further, you should note that the assignation of multiple constants like double-quoted constants (" ") to arrays are only valid when initializing the array, that is, at the moment when declared. The following expressions within a code are not valid for arrays

```
mystring="Hello";
mystring[] = "Hello";
```

neither would be: mystring = { 'H', 'e', 'l', 'l', 'o', '\0' };

So, remember: We can "assign" a multiple constants to an array only at the moment of initializing it. The reason will be more comprehensible when you know a bit more about pointers since then it will be clarified that an array is simply a constant pointer pointing to an allocated block of memory. And because of this constant feature, the array itself cannot be assigned any value, but we can assign values to each of the elements of the array.

At the moment of initializing an Array, it is a special case, since it is not an assignation, although the same equal sign (=) is used. Anyway, have always present the rule previously underlined.

8.5.3. Reading a String from the Keyboard

The object cin of <istream> along with extraction operator >> may be used for input of strings if there is no white space between the characters because cin takes the white space as the end of the string.

In the following program, only cin is used. The input string consists of two words Abebe Alemu, but cin extracts only the first word Abebe and stops reading on encountering white space.

```
#include <iostream.h>
#include<conio.h>
int main()
{
char S[15] ;
cout<<"Write a short name : " ;
cin >> S ;
cout <<"You have written : "<< S<<endl;
return 0 ;
}
```

The expected output is below.

```
Write a short name : Abebe Alemu
You have written: Abebe
```

From the output, it is clear that cin stopped reading after encountering the white space, i.e. After Abebe. Now if the same string is extracted by cin character by character as an array, the complete name is read but the white space is not read as space. This is illustrated in the following program. The two words Abebe and Alemu are merged.

```
#include <iostream.h>
int main()
{char S[15];
cout<<"Write a short name: ";
for (int i =0; i<=15;i++) // Use of for loop for reading
{
cin>>S[i] ; // It does not read the white space
cout<<S[i] ;
}
return 0 ;
}
```

The expected output is given below.

```
Write a short name: Abebe Alemu
AbebeAlemu
```

From the 2nd line of the output, it is clear that the white space has not been read and included in the string. So, in the output, the two parts of the name are merged. The above-mentioned problem is not there if we use function **cin.getline().** The function cin.getline() reads all the characters in a line until the limiting number of characters is reached or it encounters a delimiting character. It takes a maximum of three arguments. The syntax for a function call is as below.

```
cin.getline( identifier, int length, ch);
```

The "**identifier**" is the name of the string we want to put in. The integer **length** is used to limit the number of characters to be read or the maximum length of the size of the array. The "**ch**" is the character used to determine the end of the user input, which by default - if we do not include that parameter - will be the newline character ('\n').

The function stops reading when

a) n–1characters have been read or
b) a new line or end of file is encountered or
c) the delimiting character ch is encountered.

Example 1:

```
#include <iostream>
using namespace std;
int main()
{
char B[50];
cout <<"Enter a long name: ";
cin.getline(B,50) ; // It specifies to read up to read 50-1
// characters from string. The last character is Null (\0).
cout<<"You have written = "<<B <<"\n" ;
return 0 ;
```

```
}
```

The expected output is given below.

```
Enter a long name: I like studying C++.
You have written = I like studying C++.
```

Example 2:

```
#include <iostream>
using namespace std;
int main()
{
char B[20];
cout <<"Enter a name: ";
cin.getline(B,5) ; // read first 5 characters of B
cout<<"You have written = "<<B <<"\n" ;
return 0 ;
}
```

The name entered was Abebe Alemu but the function has read-only the first five characters, i.e. Abebe as given in the output given below, because, it was specified so in the getline() function.

```
Enter a name: Abebe Alemu
You have written = Abeb
```

In the following program, a delimiting character has been put in the cin.getline() function to stop reading on encountering the character.

Example 3:

```
#include <iostream>
using namespace std;
int main()
{
char B[20];
cout <<"Enter a name: ";
char ch = 'm';
cin.getline(B,20, ch) ; //read up to character 'S'
```

```
cout<<"You have written = "<<B <<"\n" ;
return 0 ;
}
```

The output is given below. It stopped reading on the occurrence of the letter'm'. So only Abebe Ale is read as given in the output below.

```
Enter a name: Abebe Alemu
You have written = Abebe Ale
```

8.5.4. Functions to manipulate strings

The standard C++ library contains several functions to perform some manipulation operations with C-like strings. The prototypes for these functions are contained in the file string.h. Here you have a brief with the most usual:

String length:

- Returns the length of a string, not including the null character (\0).

```
strlen (const char* string);
```

Example 1: The following program takes your name and tells the length of your name.

```
#include<iostream.h>
#include<string.h>
#include<conio.h>
void main()
{
char name[25];
cout<<"Enter your name: ";
cin>>name;
cout<<"your name is "<<strlen(name)<<" characters long";
getch();
}
```

Output:

```
Enter your name: abebe
```

```
your name is 5 characters long
```

Example 2:

The following C++ program reads your name from keyboard and display it in reverse order.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
#include<iomanip.h>
void main ( )
{ char name[20];
cout<<"Enter your name \t";
cin>>setw(20)>>name;
for(int i=(strlen(name)-1);i>=0;i--)
cout<<name[i];
getch();
}
```

Output:

```
Enter your name         Abebe
ebebA
```

String Copy:

- The string copy can have two forms, where the first one is to copy the whole content of the source to the destination and the other will copy only part of the source to the destination.
- Overwrites the content of the dest string by the src string. Returns dest.
- Copy the whole content of the source

```
strcpy (char* dest , const char* src );
```

Appending part of the source

```
strncpy (char* dest , const char* src, int size );
```

Where size is the number of characters to be copied

Example 3: A program on the use of **strcpy (**Copy the whole content of the source**)**

```
#include <iostream.h>
#include <string.h>
int main ()
{
char src[10],dest[10];
cout<<"\n enter your name :";
cin>>src;
strcpy (dest,src);
cout<<"\n your name after copying :"<<dest;
return 0 ;
}
```

Output:

```
enter your name :abebe
your name after copying :abebe
```

Example 4: *A program on the use of* ***strncpy (***Appending part of the source***).***

```
#include <iostream.h>
#include <string.h>
int main ()
{
char src[10],dest[10];
cout<<"\n enter your name :";
cin>>src;
strncpy (dest, src, 3); // look strncpy not strcpy
cout<<"\n your name after copying :"<<dest;
return 0 ;
}
```

Output:

```
enter your name:Abebe
 your name after copying: Abe
```

String Compare:

- Compares the two string string1 and string2.
- The string comparison can have two forms, where the first one is to compare the whole content of the two strings and the other will compare only part of the two strings.
- Compare the whole content of the source

```
strcmp (const char* string1 , const char* string2 );
```

- Appending part of the source

```
strncmp(const char*string1,const char* string2, int size );
```

Where size is the number of characters to be compared. Both string compare functions return three different values:

- Returns 0 if the strings are equal
- Returns a negative value if the first is less than the second string
- Returns a positive value if the first is greater than the second string

Example 5:

```
#include<iostream>
# include <string.h>
using namespace std;
int main()
{
char Str1[] = "XYZ" ;
char Str2[] = "ABC";
char Str3[] = "XYZ";
cout << strcmp(Str1, Str2)<<endl;
cout << strcmp(Str2, Str1)<<endl;
cout << strcmp(Str3, Str1)<<endl;
cout << strcmp(Str2, Str3)<<endl;
return 0 ;
```

```
}
```

The expected output is given below. Str1 is lexicographically greater than Str2 and Str1 is lexicographically equal to that Str3. So, the following output is obvious.

```
1
-1
0
-1
```

String Concatenation:

- Appends src string at the end of dest string. Returns dest.
- The string concatenation can have two forms, where the first one is to append the whole content of the source to the destination the other will append only part of the source to the destination.
- Appending the whole content of the source

```
strcat (char* dest , const char* src );
```

Appending part of the source

```
strncat (char* dest , const char* src, int size );
```

Where size is the number characters to be appended

Example 6:

```
#include<iostream>
# include <string.h>
using namespace std;
int main()
{
char Str1 [10] = " Abebe" ;
char Str2 [14] = " Alemu Kebede";
char Str3 [10] = "Belay";
char Str4 [10] = "Dereje";
strncat(Str1, Str2, 5);
```

```
cout<<Str1<<endl;
cout<<Str2<<endl;
strcat( Str3, Str4);
cout << Str3<<endl;
cout<<Str4 <<endl;
return 0;
}
```

The expected output is given below.

```
Abebe Alemu
Alemu Kebede
Belay Dereje
Dereje
```

The first line of output is of Str1 after the operation of strncat(Str1, Str2, 5);.The first 5 characters of S2 (Alemu) are appended to the end of Str1. The string Str2 is not affected by this operation. The third line of output is the result of strcat(Str3, Str4);. The complete string Str4 is appended to the end of string Str3. The string Str4 remains intact.

8.5.5. Converting strings to other types

Due to that, a string may contain representations of other data types like numbers it might be useful to translate that content to a variable of a numeric type. For example, a string may contain "**1977**", but this is a sequence of 5 chars not so easily convertible to a single integer data type.

The cstdlib (**stdlib.h**) library provides three useful functions for this purpose:

- atoi: converts string to int type.
- atol: converts string to long type.
- atof: converts string to float type.

All of these functions admit one parameter and return a value of the requested type (int, long, or float). These functions combined with the getline method of **cin** are a more reliable way to get the user input when requesting a number than the classic **cin**>> method:

Example 7:

```
#include <iostream>
# include <cstdlib>
using namespace std;
void main()
{
char ch1 []= "6444";
char ch2[] = "21.66";
char ch3[] = "Absent45";
char ch4[] = "45Absent";
char chL1[] = "Delhi45676548";
char chL2[] = "45676548Delhi";
cout<<"The number represented by ch1 divided by 2 = "
<<atoi(ch1)/2 <<endl;
cout<<"Number ch2 /3= " << atof (ch2)/3<<endl;
cout<<"Number ch3 = " <<atoi (ch3)<<endl;
cout<<"The number represented by ch4 = " <<atoi
(ch4)<<endl;
cout<<"The number represented by chL1 = " <<atol
(chL1)<<endl;
cout<<"The number represented by chL2 = " <<atol
(chL2)<<endl;
}
```

The expected output is as below.

```
The number represented by ch1 divided by 2 = 3222
Number ch2 /3= 7.22
Number ch3 = 0
The number represented by ch4 = 45
The number represented by chL1 = 0
The number represented by chL2 = 45676548
```

Six strings ch1, ch2, ch3, ch4, chL1 and chL2 are declared and initialized. The functions atoi(), atof (), and atol() are called to convert the strings into corresponding digits. The strings which begin with non-digit characters are not converted and 0 is returned while

those which begin with digit characters are converted. To show that the converted digits represent a number, the numbers obtained in the first two cases have been divided by 2 and 3 respectively. The trailing ends of strings consisting of non-digital characters are ignored.

Example 8:

```
// cin and ato* functions
#include <iostream.h>
#include <stdlib.h>
#include<conio.h>
int main()
{
clrscr();
char mybuffer[100];
float price;
int quantity;
cout << "Enter price: ";
cin.getline (mybuffer, 100);
price = atof (mybuffer);
cout << "Enter a quantity: ";
cin.getline (mybuffer, 100);
quantity = atoi (mybuffer);
cout<<"\nafter conversion :\n";
cout<<"\nprice is: "<<price;
cout<<"\nquantity is: "<<quantity;
cout << "\nTotal price: " << price*quantity;
getch();
return 0;
}
```

Output:

```
Enter price: 25.5
Enter a quantity: 10
after conversion :
price is: 25.5
quantity is: 10
```

```
Total price: 255
```

Lab

1. Write a program to store the ages of six of your friends in a single array. Store each of the six ages using the assignment operator. print the ages on the screen
2. Write a C++ program that accepts 10 integers from the user and finally displays the smallest value and the largest value.
3. Write a program that accepts ten different integers from the user and display these numbers after sorting them in increasing order.
4. Write a C++ program to count the number of occurrences of a given number in a sorted array of integers in lab exercise #3.
5. Write a program to store six of your friend's ages in a single array. Assign the ages in random order. print the ages, from low to high, on-screen
6. Write a C++ program that calculates the letter grades of 20 students. The program should accept the mid result and the final result from the students. Use the appropriate validity control mechanism to prevent wrong inputs.
7. Write a program which prints the letters in a char array in reverse order. For example, if the array contains {'c', 's', 'c', '2', '6', '1'}the output (to the terminal) should be "162csc".

8. Define two functions which, respectively, input values for the elements of an array of reals and output the array elements:

```
void ReadArray (double nums[], const int size);
void WriteArray (double nums[], const int size);
```

9. Define a function that reverses the order of the elements of an array of reals:

```
void Reverse (double nums[], const int size);
```

10. The following table specifies the major contents of four brands of breakfast cereals. Define a two-dimensional array to capture this data:

	Fiber	Sugar	Fat	Salt
Top Flake	12g	25g	16g	0.4g
Cornabix	22g	4g	8g	0.3g
Oatabix	28g	5g	9g	0.5g
Ultrabran	32g	7g	2g	0.2g

11. Write a program in C++ that reads a three-digit number and generates all the possible permutation of numbers using the above digits. For example, n = 123 then the permutations are

```
123, 213, 312, 132, 231, 321
```

Review question

1. Answer the following statements as either true or false:

   i. The array data type is one of the standard data types in C++.

   ii. Arrays can have more than one dimension.

   iii. For loops are often used to display the members of an array.

   iv. When defining an array, it is preferable to specify how many members are in the array.

   v. We can use an array for elements with different data types.

2. Provide the statement(s) that declare and create an array named **a** that can hold 40 integers.

3. Can you declare an array to hold a mixture of **ints** and **doubles**?

4. What happens if you attempt to access an element of an array using a negative index?

5. Given the declaration

```
int list[100];
```

a) What expression represents the very first element of the list?

b) What expression represents the very last element of the list?

c) Write the code fragment that prints out the contents of the list.

d) Is the expression list[3.0] legal or illegal?

6. Suppose s is a string of characters

   (a) What expression represents the number of characters that make ups?

   (b) What expression represents the first character in s?

   (c) What statement would insert the character 'x' onto the front of s?

   (d) What statement would append the character 'x' onto the back of s?

7. What will the following program output when executed?

```
#include <iostream.h>
char *str = "global";
void Print (char *str)
{
cout<< str << '\n';
{
char *str = "local";
cout<<str << '\n';
cout<<::str << '\n';
}
cout<< str << '\n';
}
int main (void)
{
Print("Parameter");
return 0;
}
```

8. Write a C++ program to

   - initialize a string `s1` with the string `"As time by ..."` and a second-string `s2` with the string `"goes"`,

- insert string `s2` in front of `"by"` in string `s1`,
- erase the remainder of string `s1` after the substring `"by"`,
- replace the substring `"time"` in `s1` with `"Bill"`.

In each case, your program should determine the position of the substring. Output string s1 on screen at the beginning of the program and after every modification.

Exercise

1. Write a C++ program that has two functions toBinary and toDecimal. The program should display a menu prompting the user to enter his choice. If the user selects toBinary, then the function should accept a number in base ten and displays the equivalent binary representation. The reverse should be done if the user selects toDecimal.
2. Write a C++ program that accepts a word from the user and then displays the word after reversing it.
3. Write a program in C++ to count the total number of words in a string
4. Develop a C++ program that accepts the name of a person and then counts how many vowels the person's name has?
5. Write a program that reads a set of lines until you enter #.
6. Write a program that reads two matrixes and then prints a matrix that is the addition of these two matrixes.
7. Write a program which reads two matrices and multiply them if possible
8. Write a program which reads a 3 x 2 matrix and then calculates the sum of each row and store that in a one-dimension array.
9. Write a program in C++ to remove characters in String Except Alphabets Input a string of alphabets. Find out the number of occurrences of all alphabets in that string. Find out the alphabet with maximum occurrence.
10. Write a function named `"rotate right"` that takes as its arguments the following:
    - an array of floating-point values;

- an integer that tells the number of cells in the array;

The function should shift the contents of each cell one place to the right, except for the contents of the last cell, which should be moved into the cell with subscript 0. Thus, for example, if the array passed to the function looks like this:

5.8	2.6	9.1	3.4	7.0

then when the function returns, the array will have been changed so that it looks like this:

7.0	5.8	2.6	9.1	3.4

The function should not return a value.

11. Write a function **equalsIgnoreCase**, which receives two char arrays and their sizes, and returns true if the two char arrays contain the same characters irrespective of the case. For example, for character arrays {'a', 'B', 'c'} and {'A', 'b', 'c'}, the function returns true, but for {'a', 'B', 'c'} and {'a', B'}, or {'a', 'B', 'c'} and {'X', 'b', 'z'}, the function returns false.

12. Write a function named "location_of_target" that takes as its arguments the following:
    1) an array of integer values;
    2) an integer that tells how many integer values are in the array;
    3) an integer "target value".

    The function should determine whether the given target value occurs in any of the cells of the array and if it does, the function should return the subscript of the cell containing the target value. If more than one of the cells contains the target value, then the function should return the largest subscript of the cells that contain the target value. If the target value does not occur in any of the cells, then the function should return the sentinel value -1 Thus, for example, if the target value that's passed to the function is 34 and the array that's passed to the function looks like this:

58	26	91	34	70	34	88

then the target value occurs in cells 3 and 5, so the function should return the integer value 5.

References

1. Savitch, Walter.Problem-Solving with C++: The object programming with 4th Edition, Menlo Park, Addison-Wesley, 2016
2. Stroustrup, Bjarne: The C++ Programming Language, 3rd Edition, Addison – Wesley
3. Teach yourself C++ in 21 days, Available on http://newdata.box.sk/bx/c/
4. Lafore, Robert, Object-Oriented C++ Programming using Turbo C++
5. Dale, Nell, Chp Weems, Mark Headington, Programming and problem solving with C++, 2nd Edition, Jones and Bartlett Publishers, Inc: Massachusetts, 2010